

Hierarchical reinforcement learning

or abstraction in sequential decision making

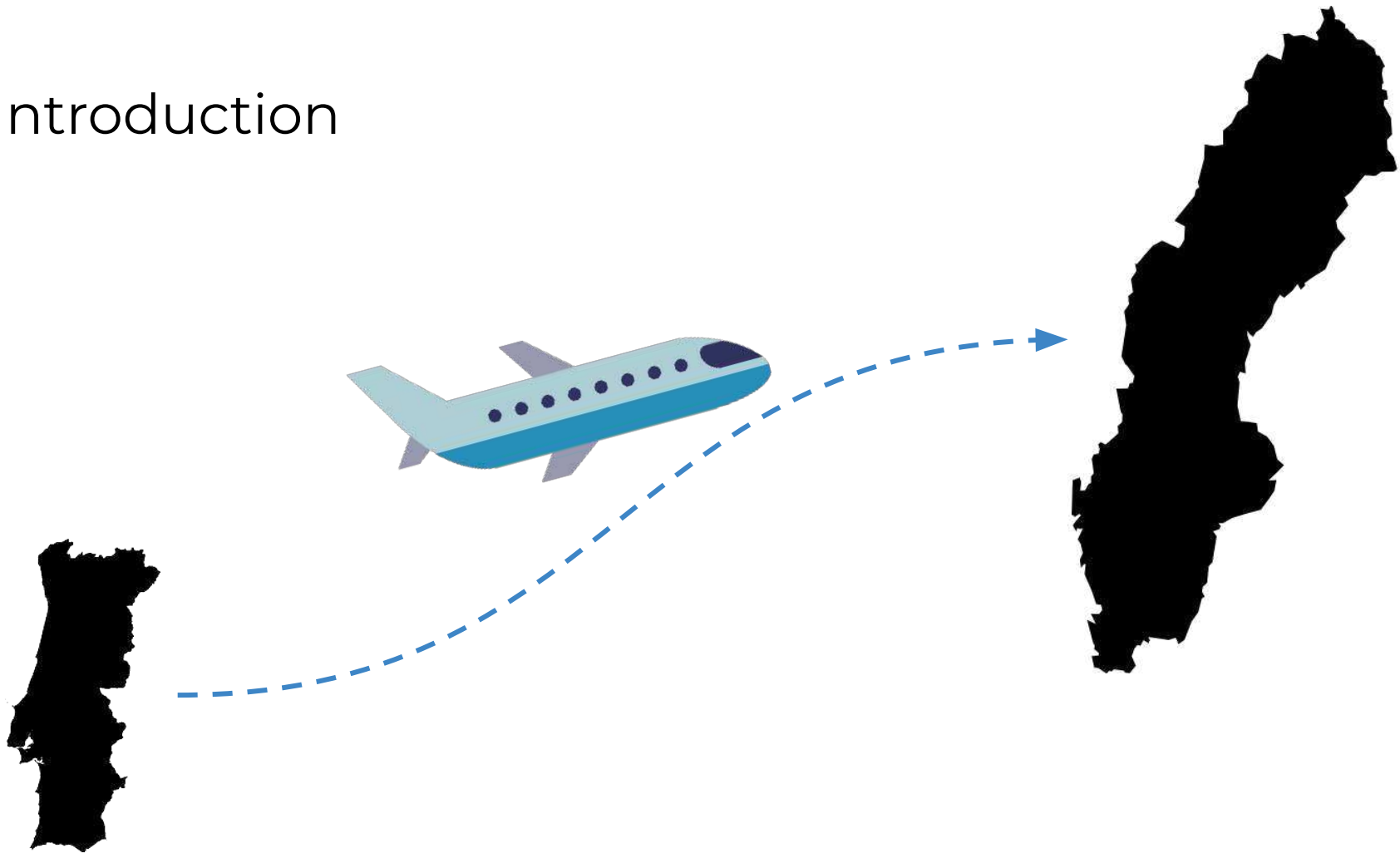
Outline

- Introduction
- Frameworks
- Task discovery
- Conclusion

Outline

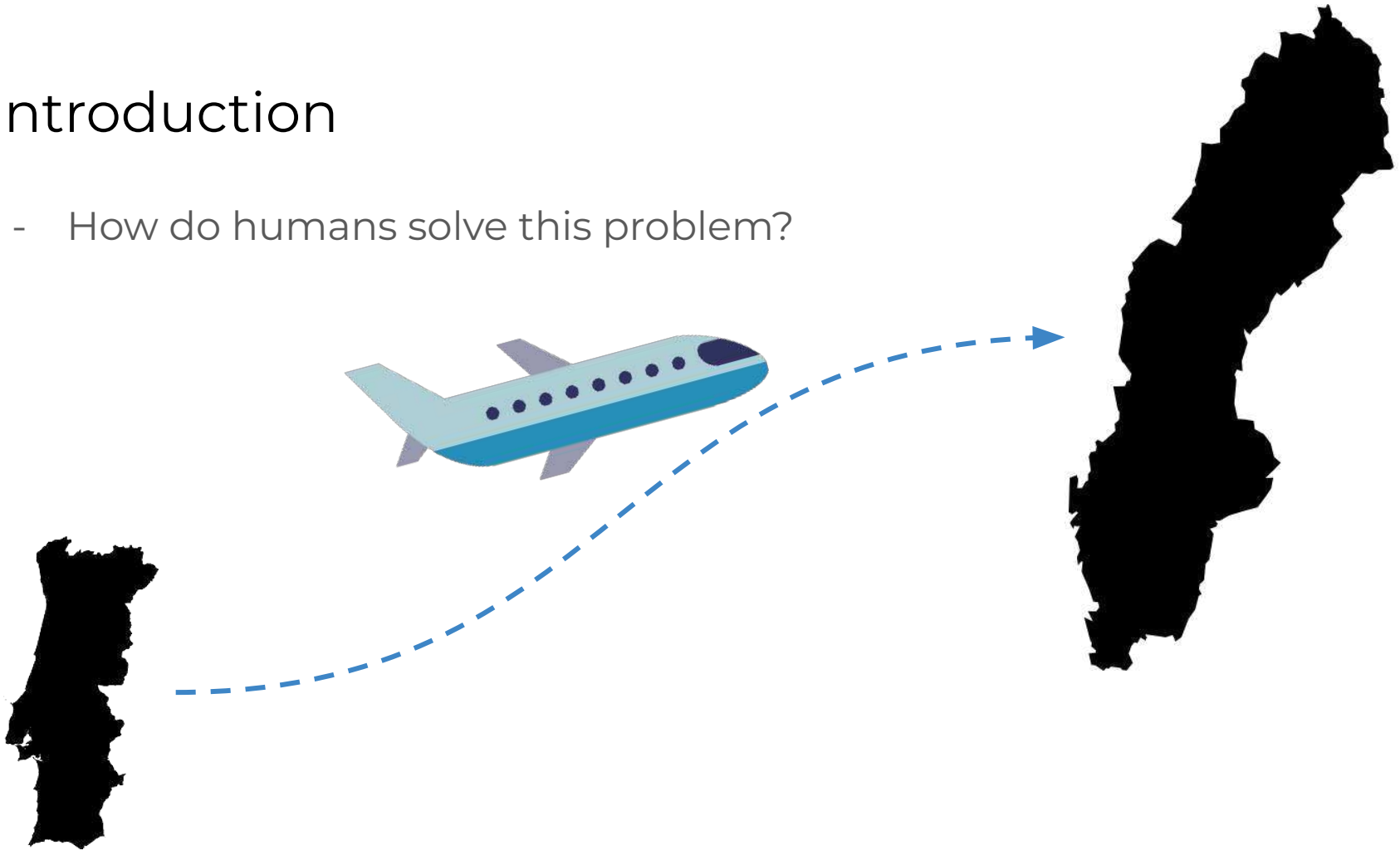
- Introduction
 - Hierarchical decision making
 - Motivation
 - Example
- Frameworks
 - Feudal hierarchies
 - Options
- Task discovery
 - Early techniques
 - Eigenoptions
- Conclusion
 - Successes
 - Limitations
 - Open problems

Introduction



Introduction

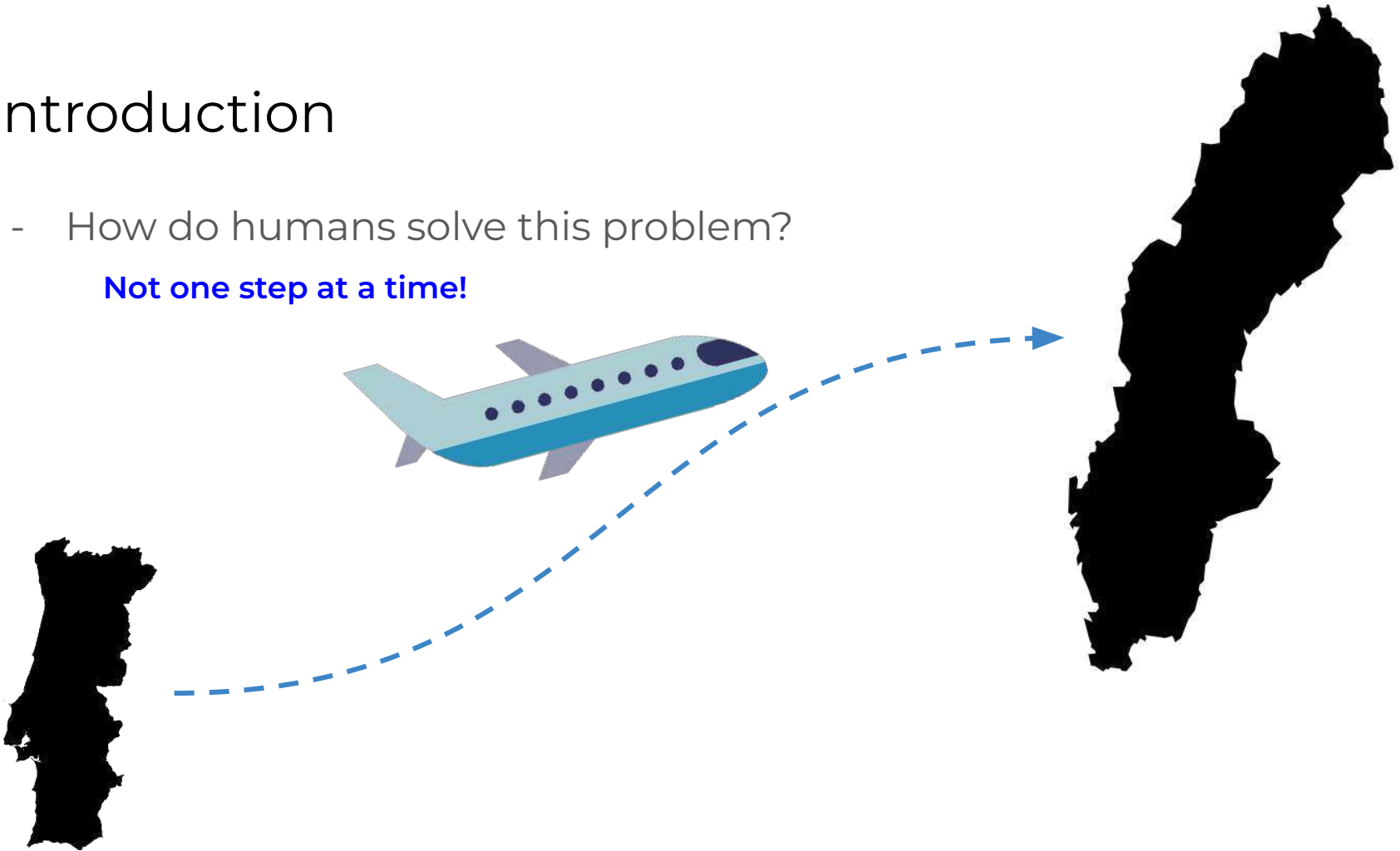
- How do humans solve this problem?



Introduction

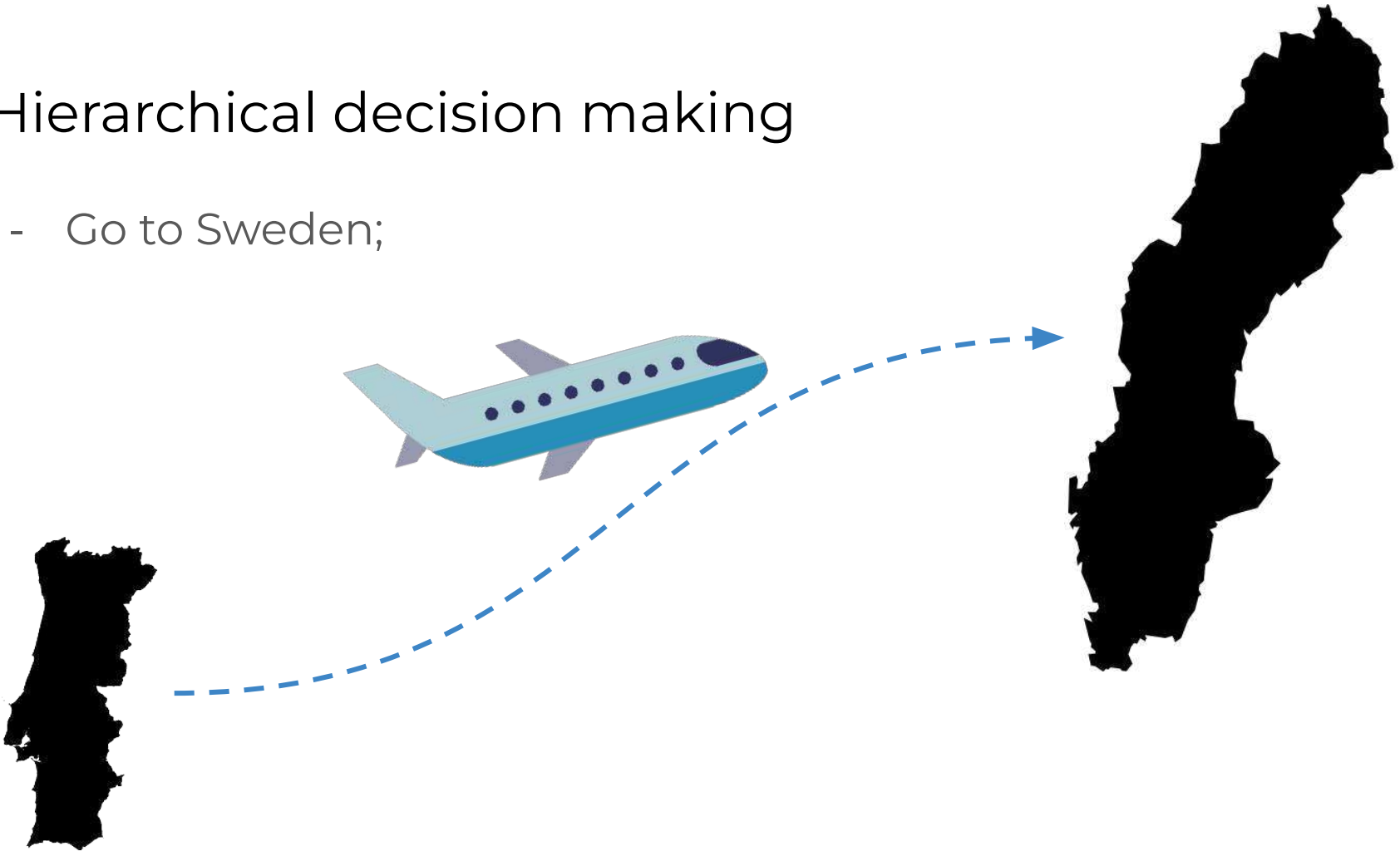
- How do humans solve this problem?

Not one step at a time!



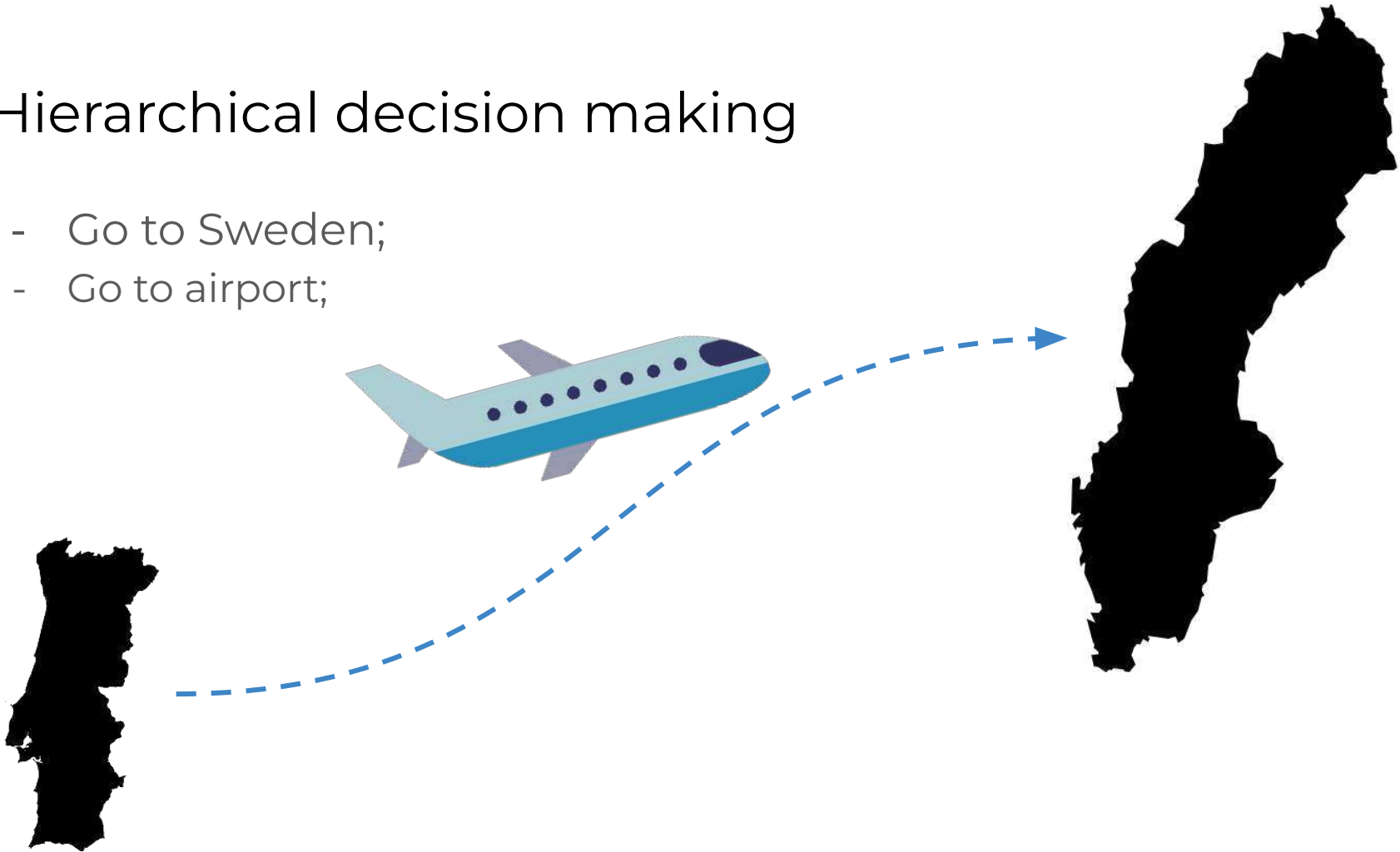
Hierarchical decision making

- Go to Sweden;



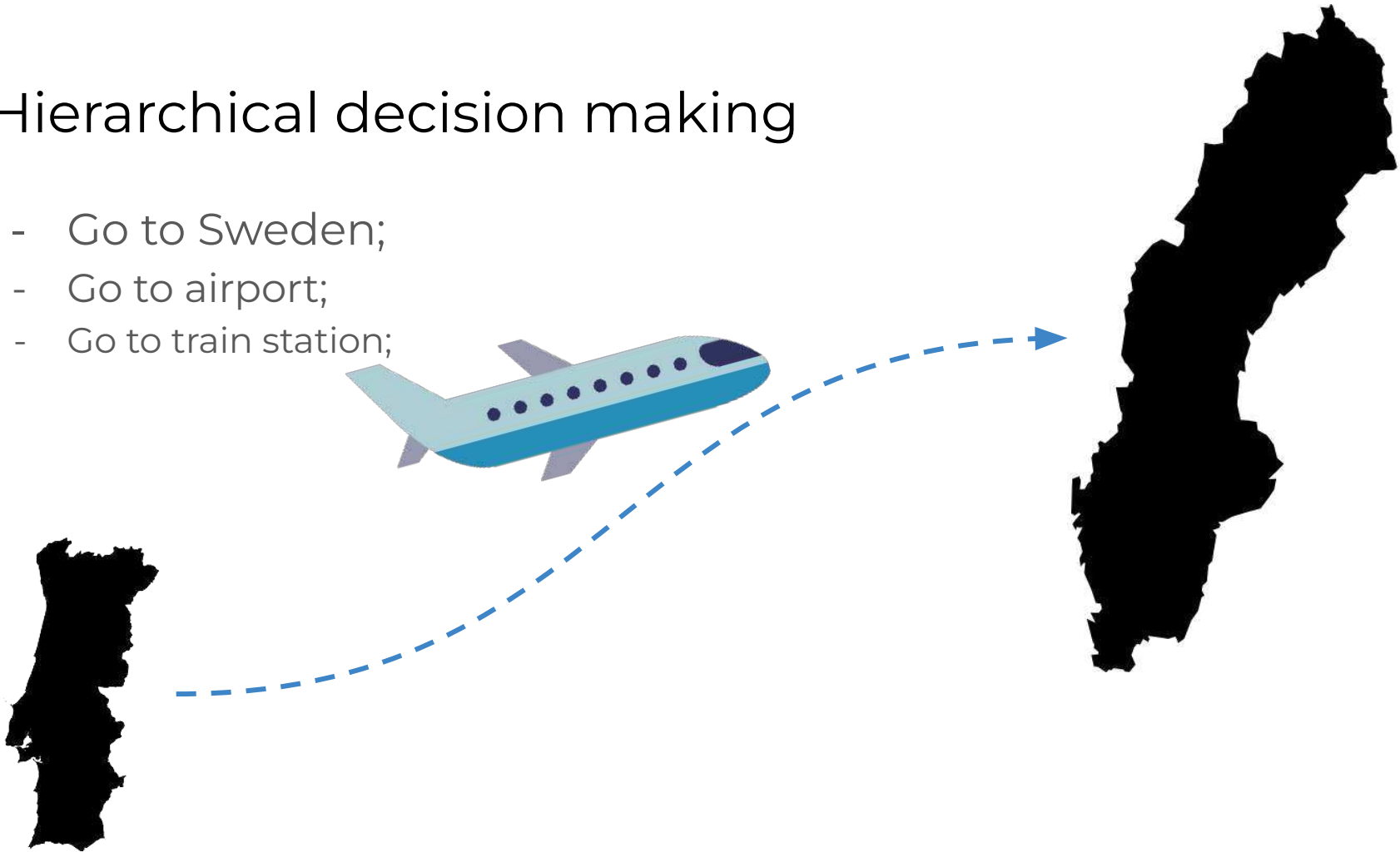
Hierarchical decision making

- Go to Sweden;
- Go to airport;



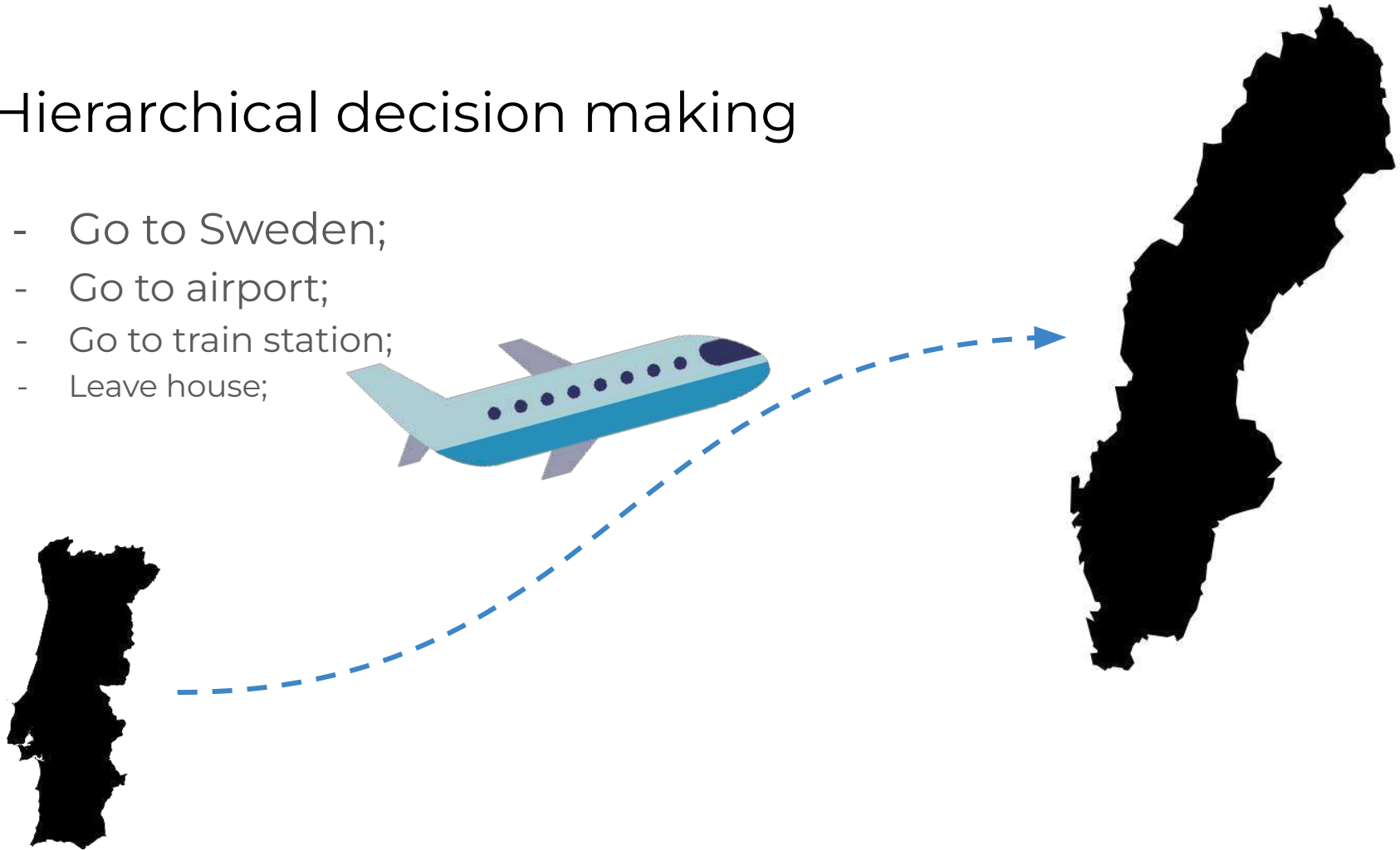
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Go to train station;



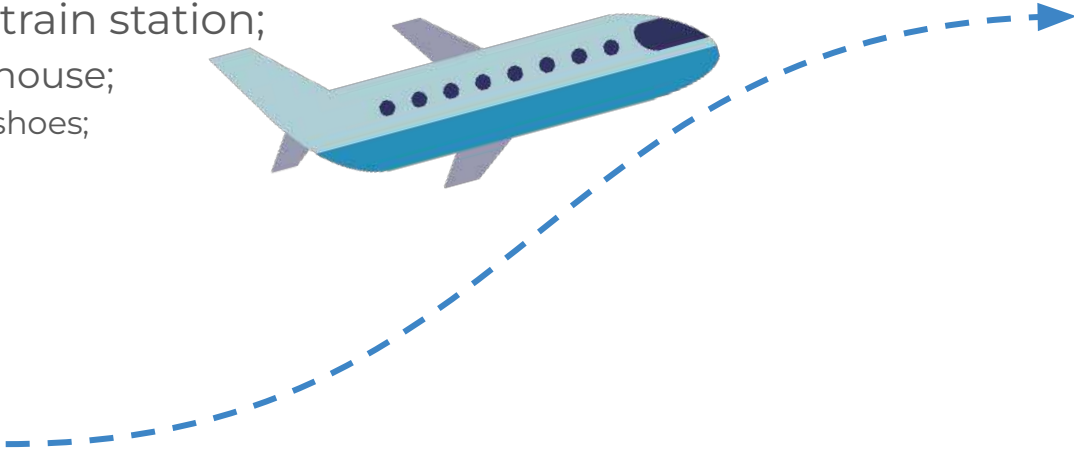
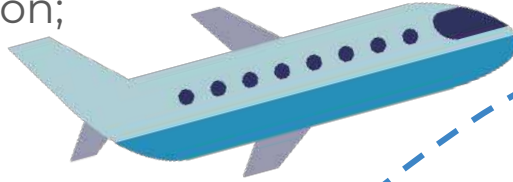
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Go to train station;
- Leave house;



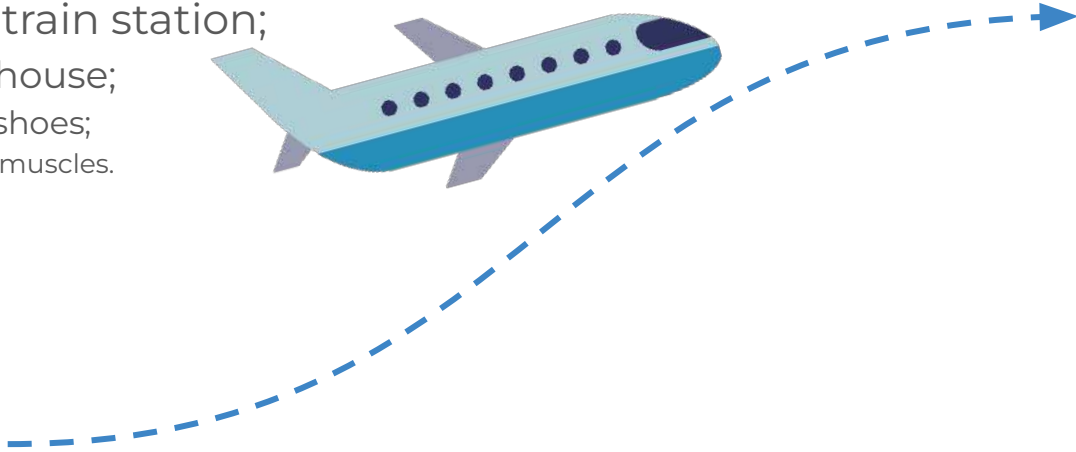
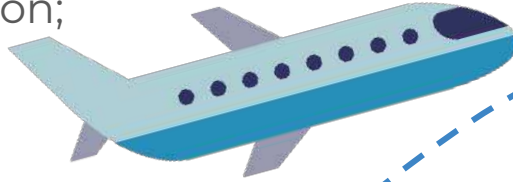
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Go to train station;
- Leave house;
- Put on shoes;



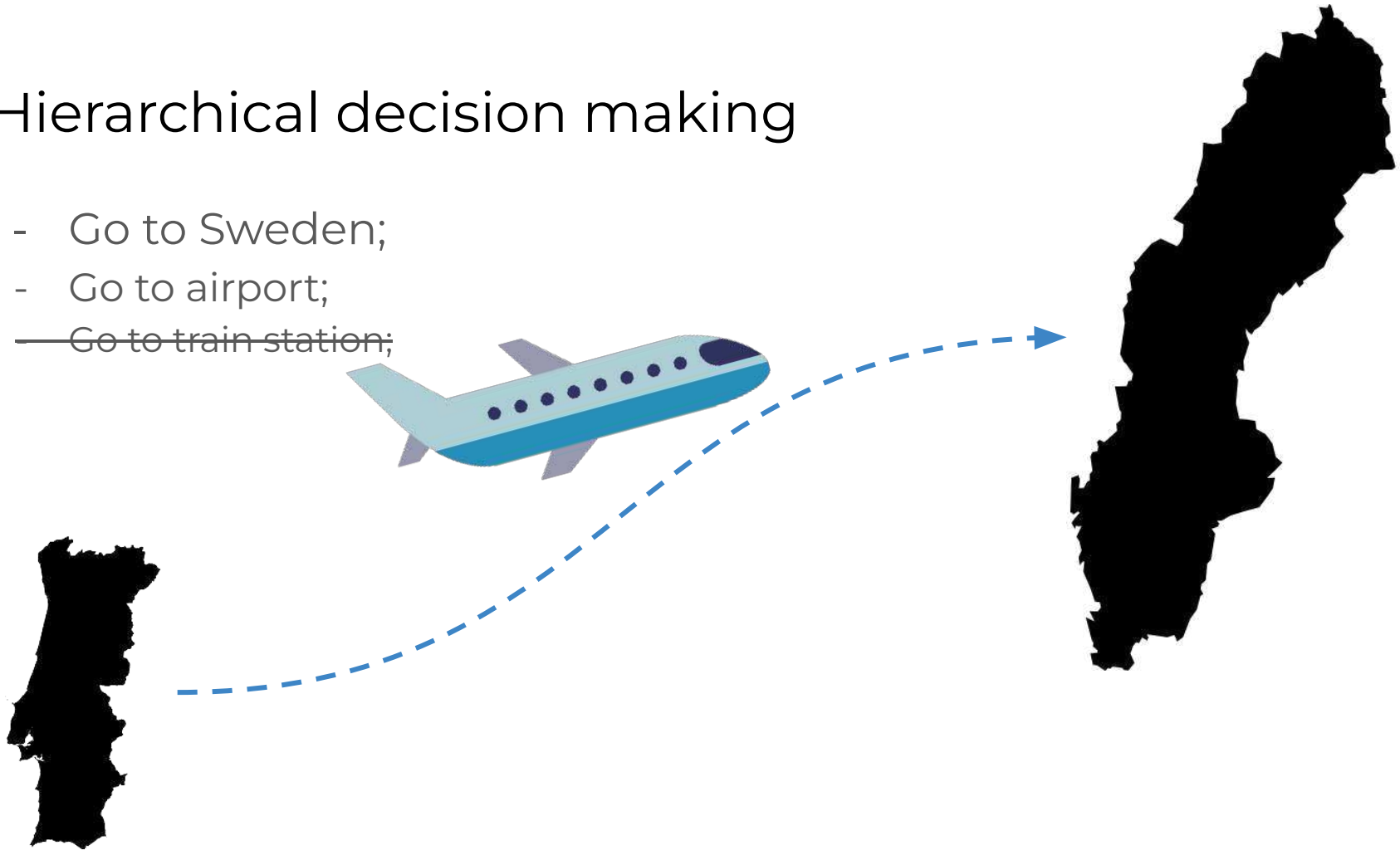
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Go to train station;
- Leave house;
- Put on shoes;
- Contract muscles.



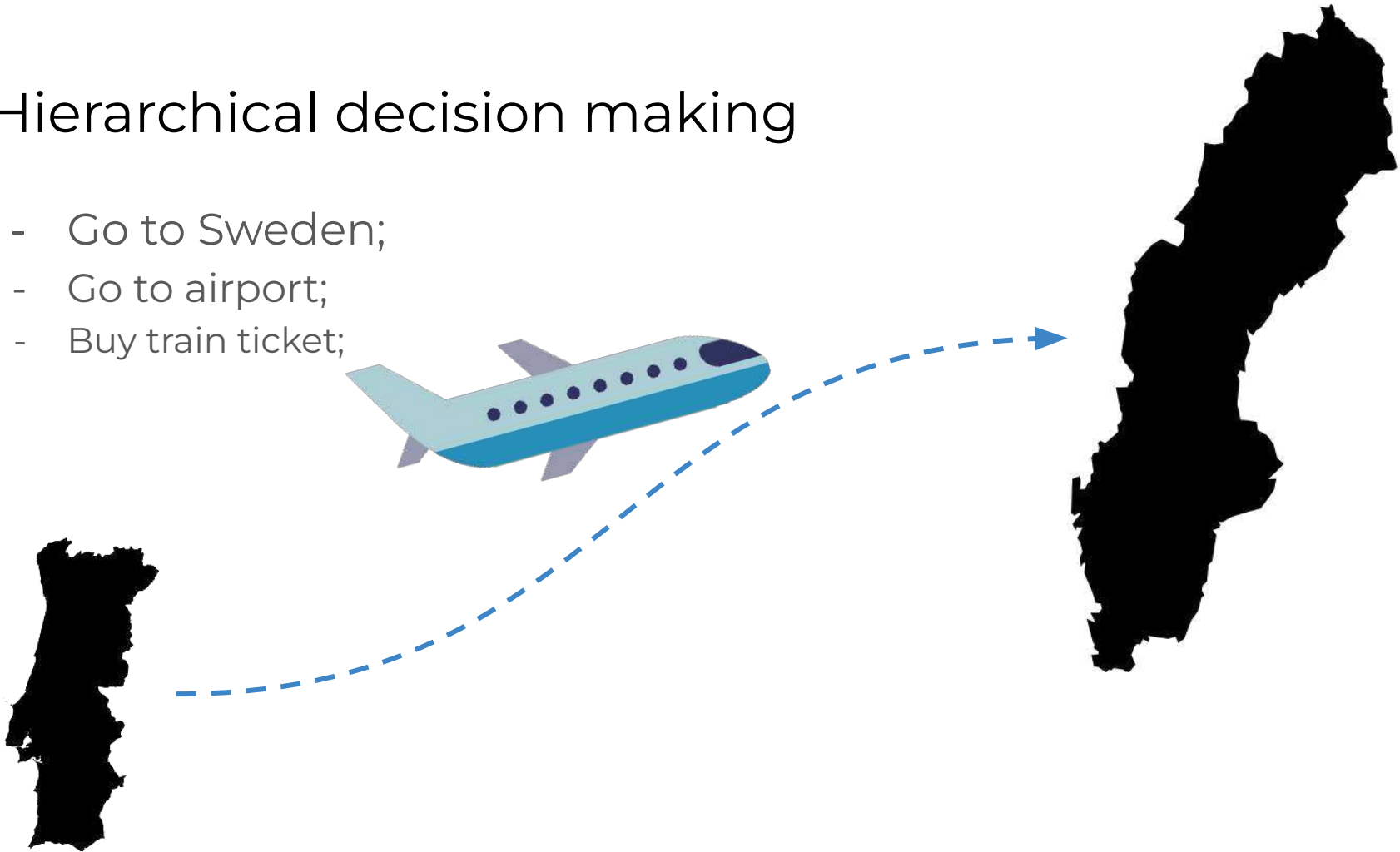
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- ~~— Go to train station;~~



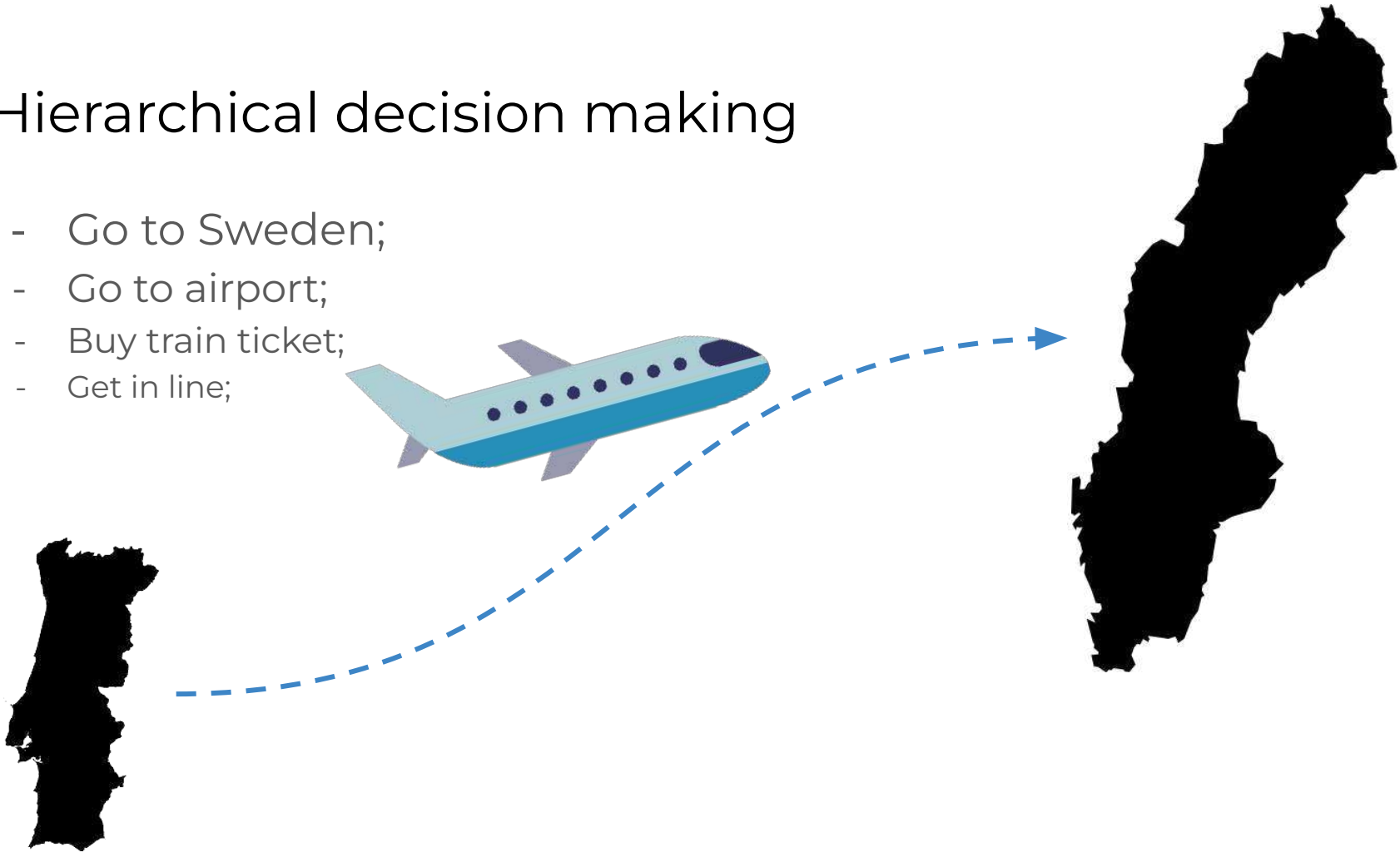
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Buy train ticket;



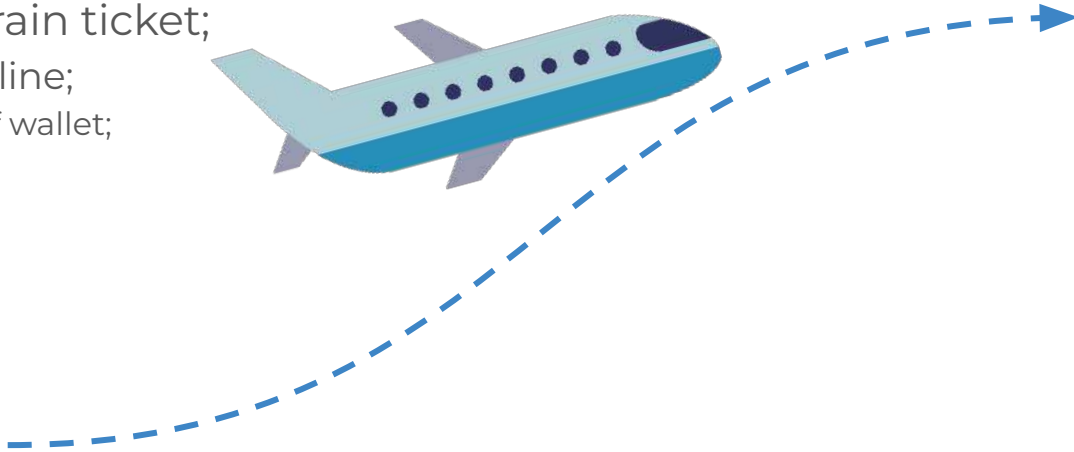
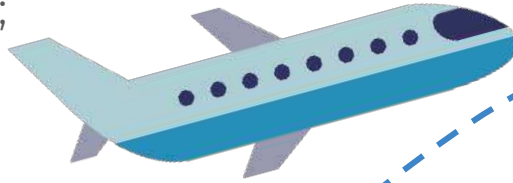
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Buy train ticket;
- Get in line;



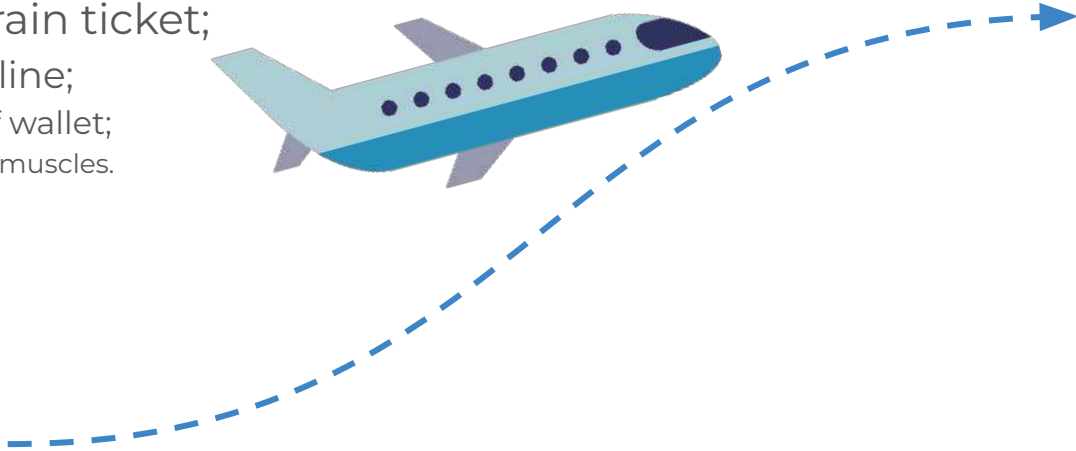
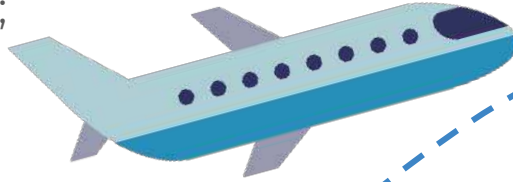
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Buy train ticket;
- Get in line;
- Take off wallet;



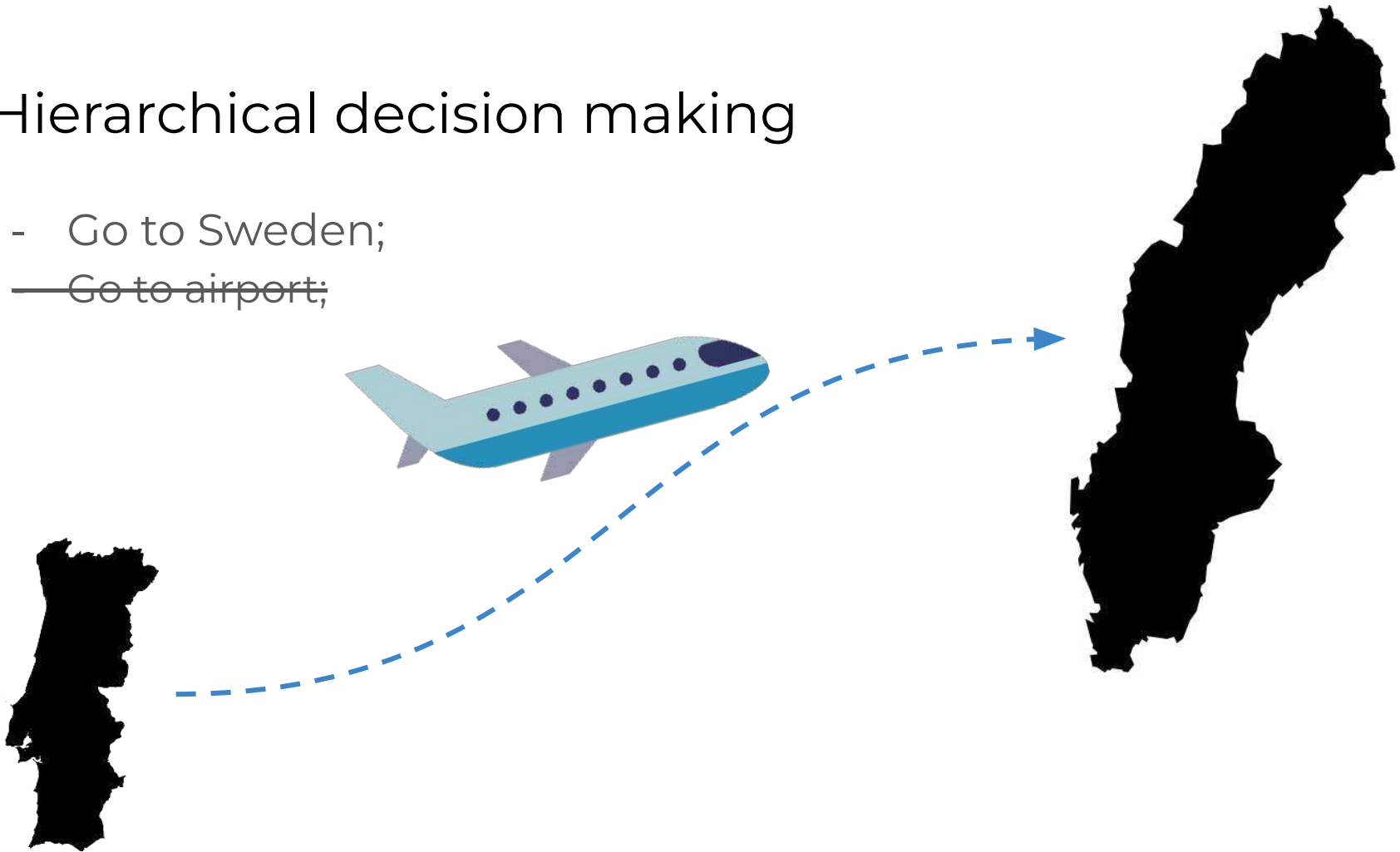
Hierarchical decision making

- Go to Sweden;
- Go to airport;
- Buy train ticket;
- Get in line;
- Take off wallet;
- Contract muscles.



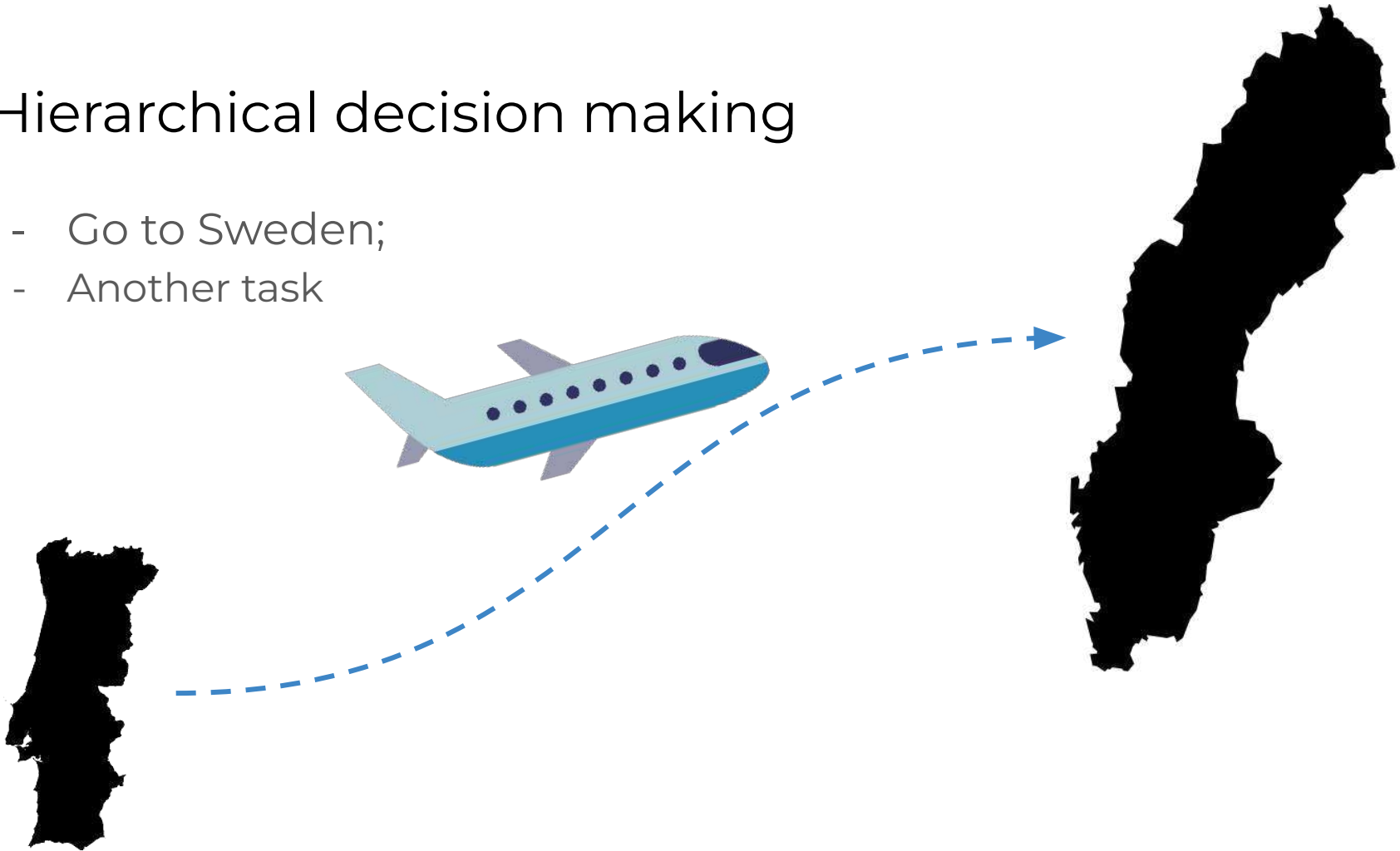
Hierarchical decision making

- Go to Sweden;
- ~~— Go to airport;~~



Hierarchical decision making

- Go to Sweden;
- Another task



Hierarchical decision making

- Divide and conquer:

Hierarchical decision making

- Divide and conquer:
 - Decompose the problem into smaller problems;

Hierarchical decision making

- Divide and conquer:
 - Decompose the problem into smaller problems;
 - Solve the problems and compose the solutions.

Motivation

- Reinforcement learning



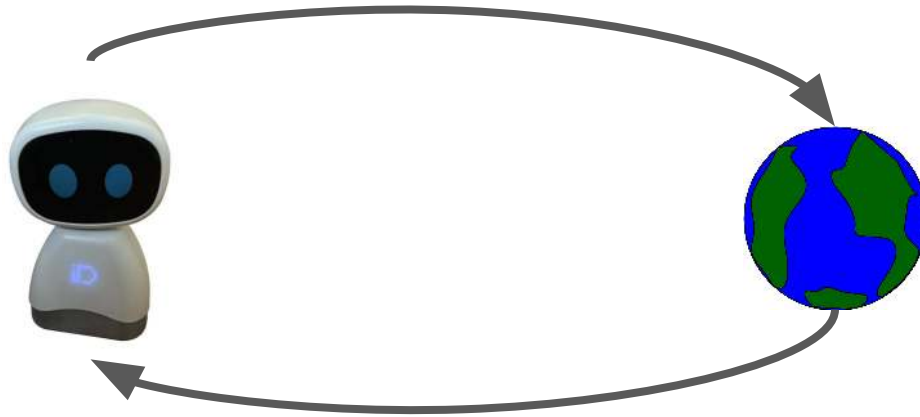
Motivation

- Reinforcement learning



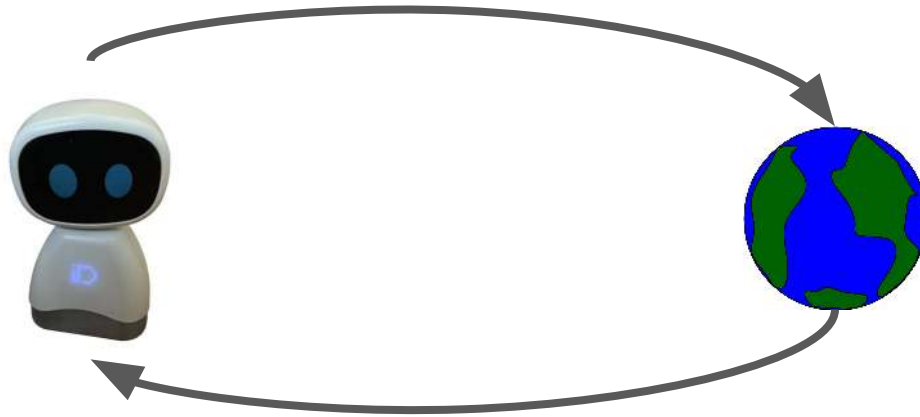
Motivation

- Reinforcement learning



Motivation

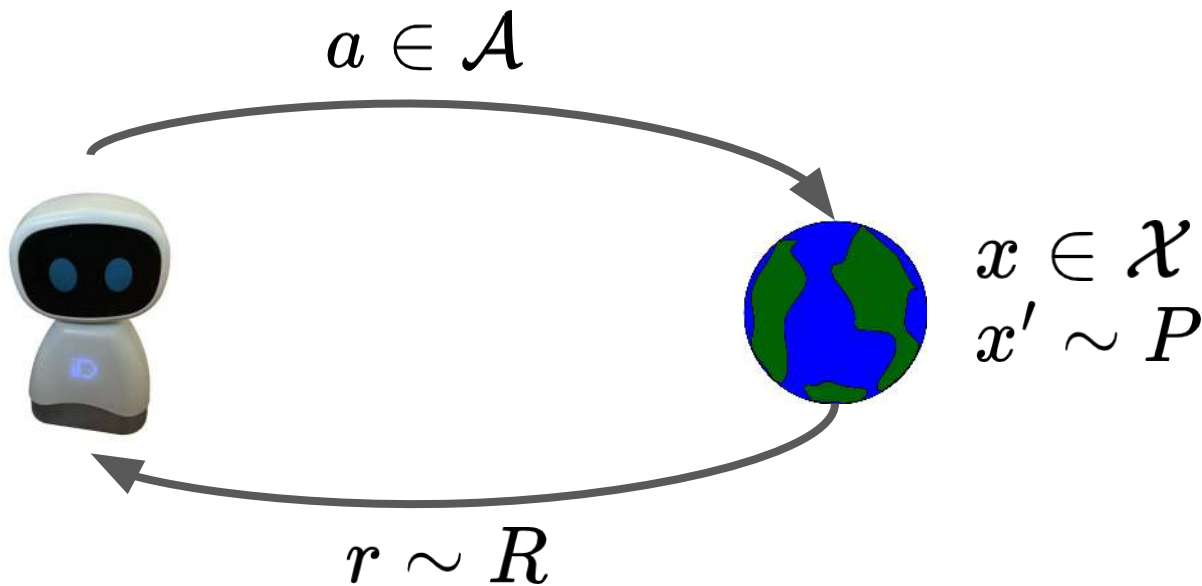
- Reinforcement learning



Motivation

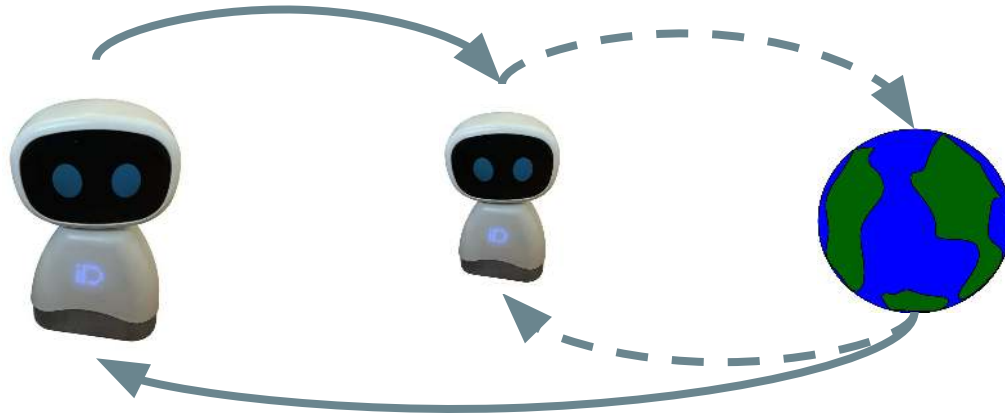
- Reinforcement learning

$$(\mathcal{X}, \mathcal{A}, R, P)$$



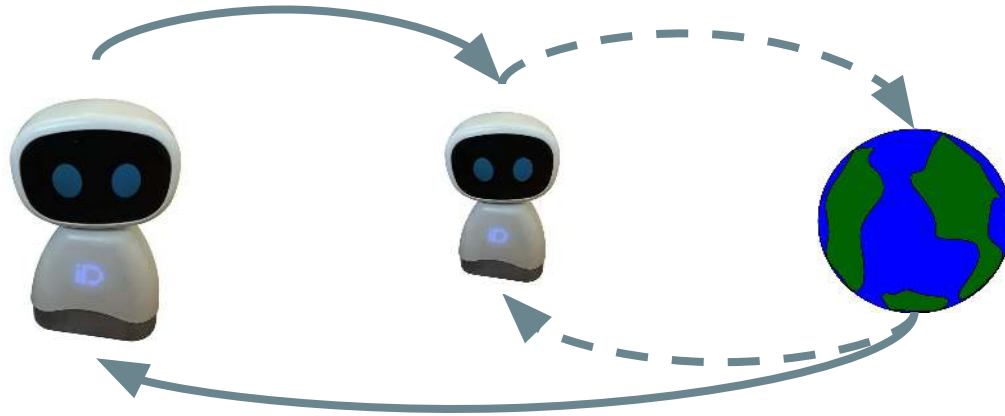
Motivation

- Hierarchical reinforcement learning



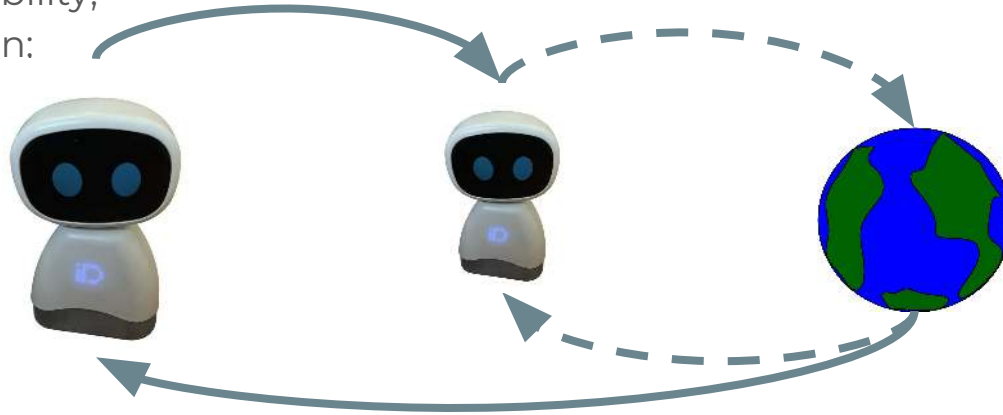
Motivation

- Several potential benefits



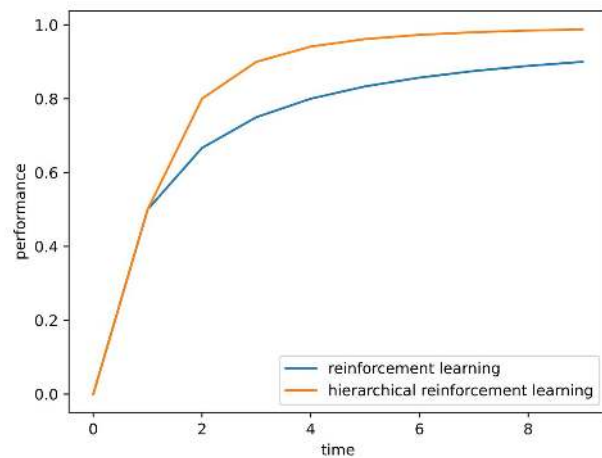
Motivation

- Several potential benefits:
 - Efficiency;
 - Simplification;
 - Interpretability;
 - Exploration:
 - Transfer.



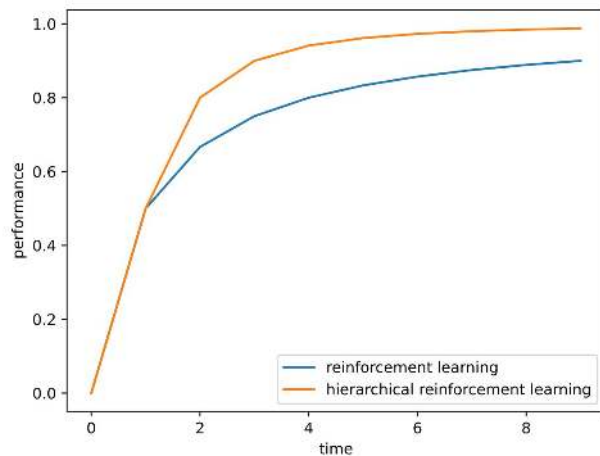
Motivation

- Learning efficiency:



Motivation

- Learning efficiency:
 - Structured credit assignment;
 - Sharing between subproblems.



Motivation

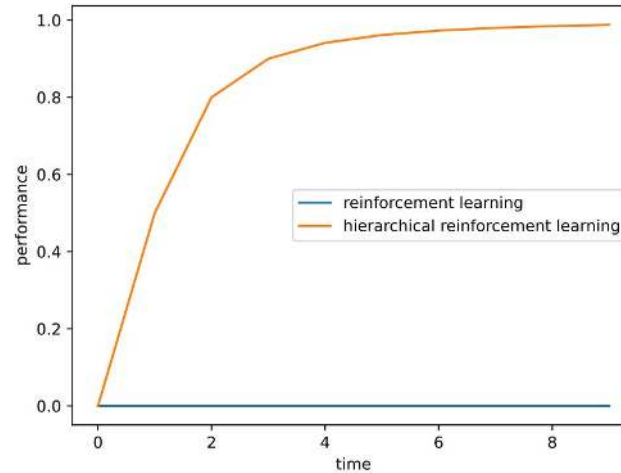
- Abstraction / simplification:

Motivation

- Abstraction / simplification:
 - Reduction of the horizon;
 - Reduction of the state space.

Motivation

- Abstraction / simplification:
 - Reduction of the horizon;
 - Reduction of the state space.

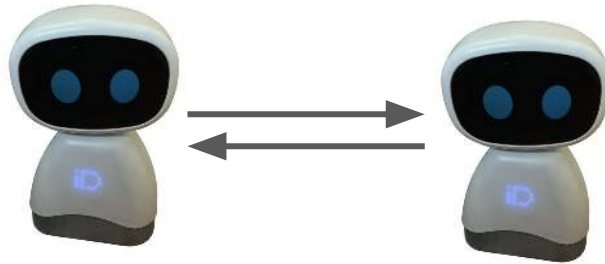


Motivation

- Interpretability:

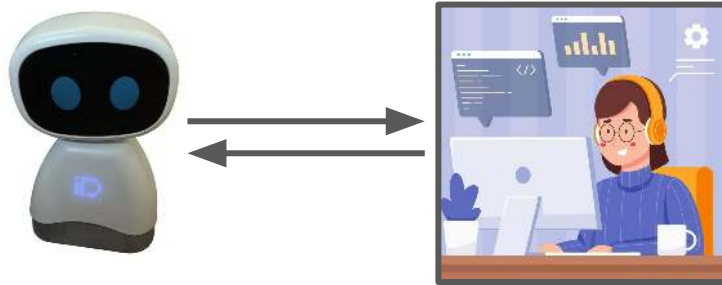
Motivation

- Interpretability:
 - Communication;
 - Flexibility.



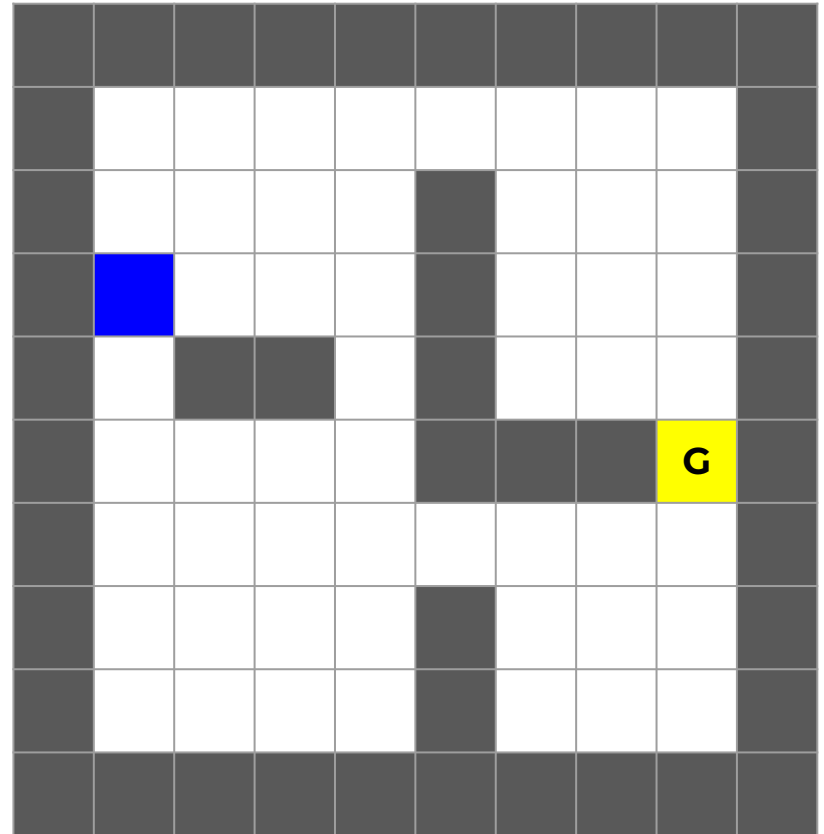
Motivation

- Interpretability:
 - Communication;
 - Flexibility.



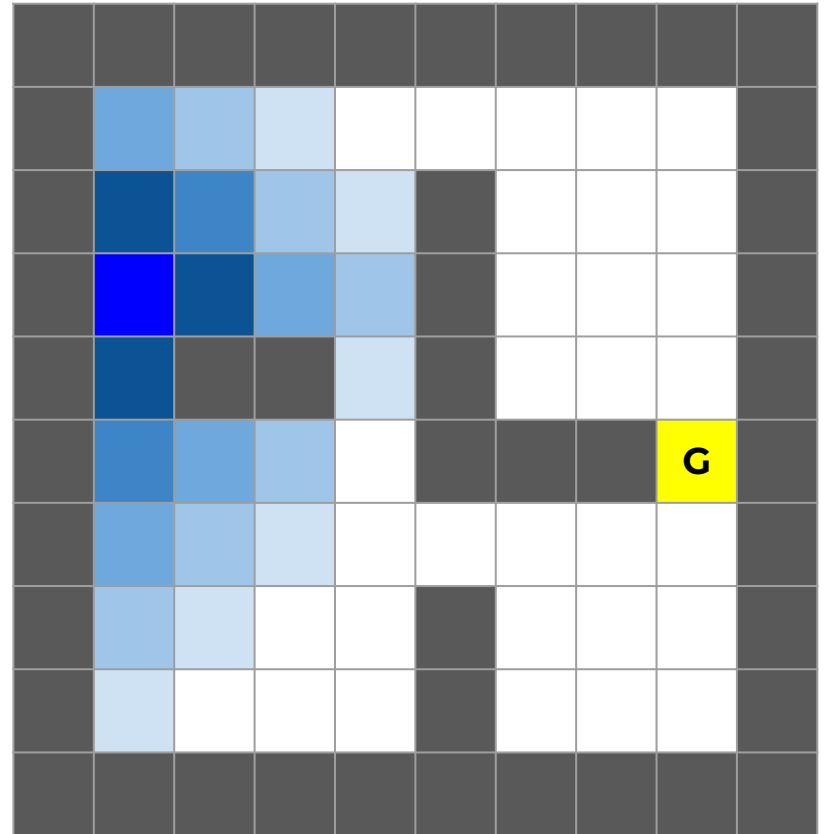
Motivation

- Exploration



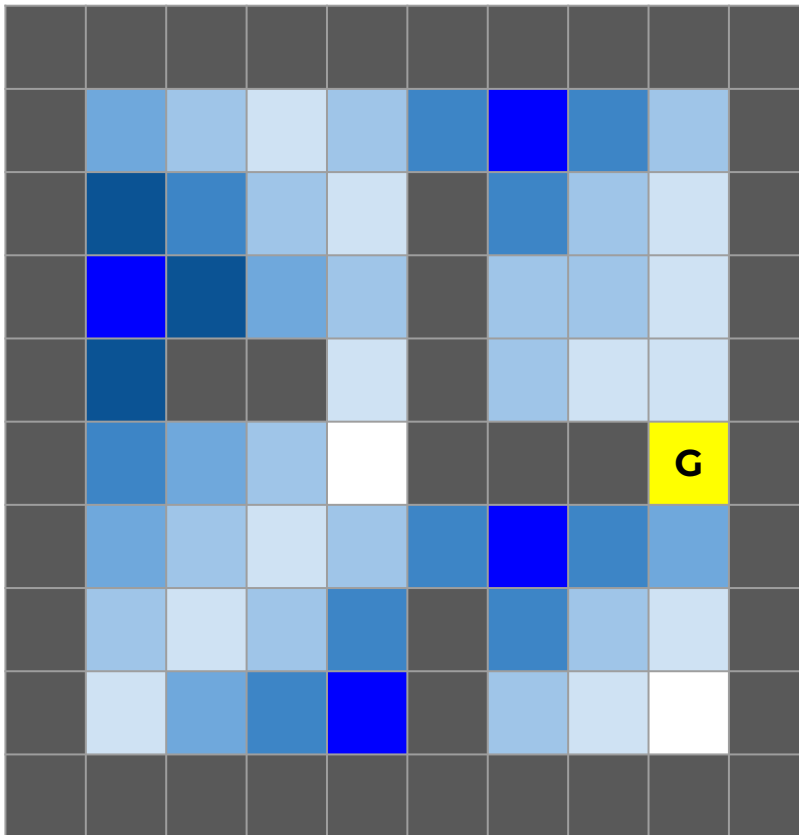
Motivation

- Exploration



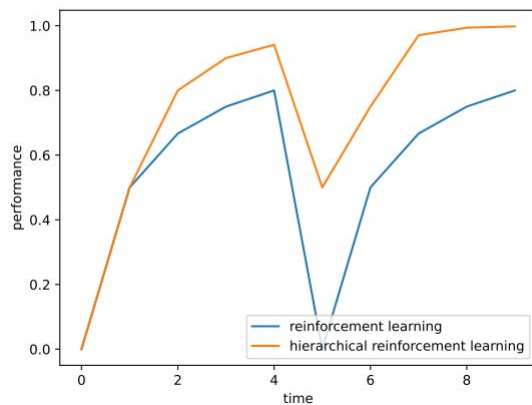
Motivation

- Exploration



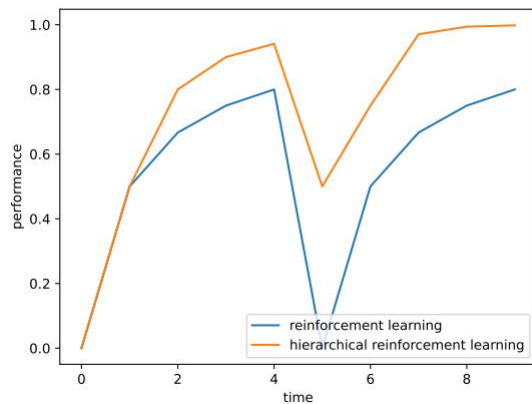
Motivation

- Transfer learning / continual learning:



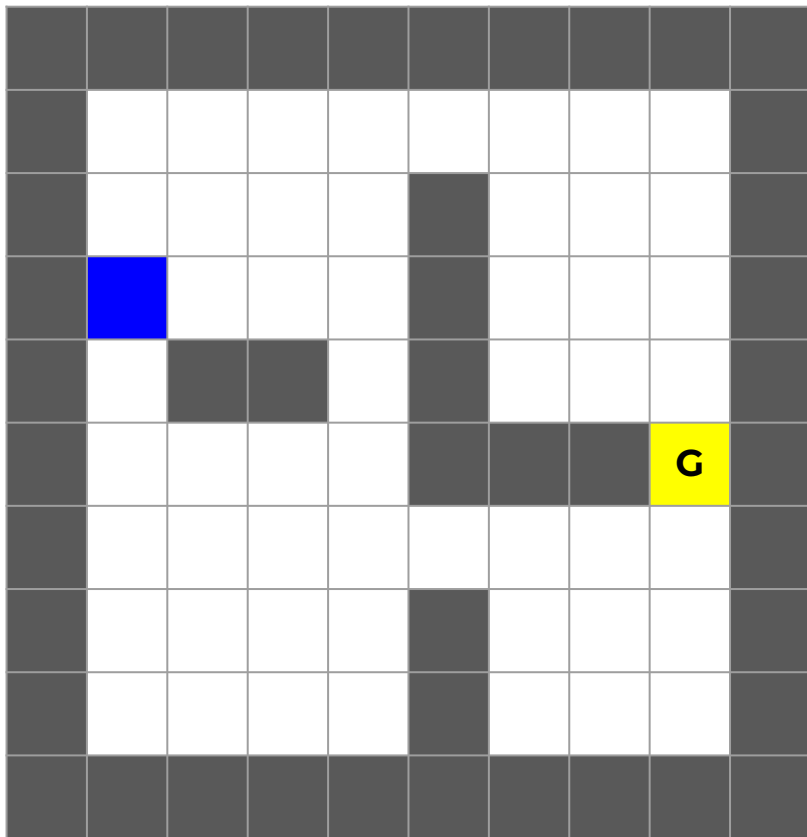
Motivation

- Transfer learning / continual learning:
 - High-level transfer;
 - Low-level transfer.

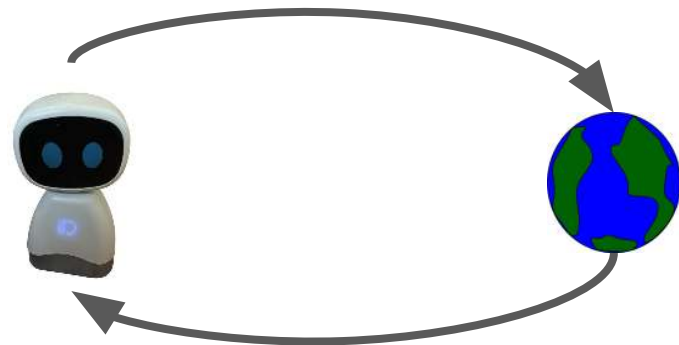
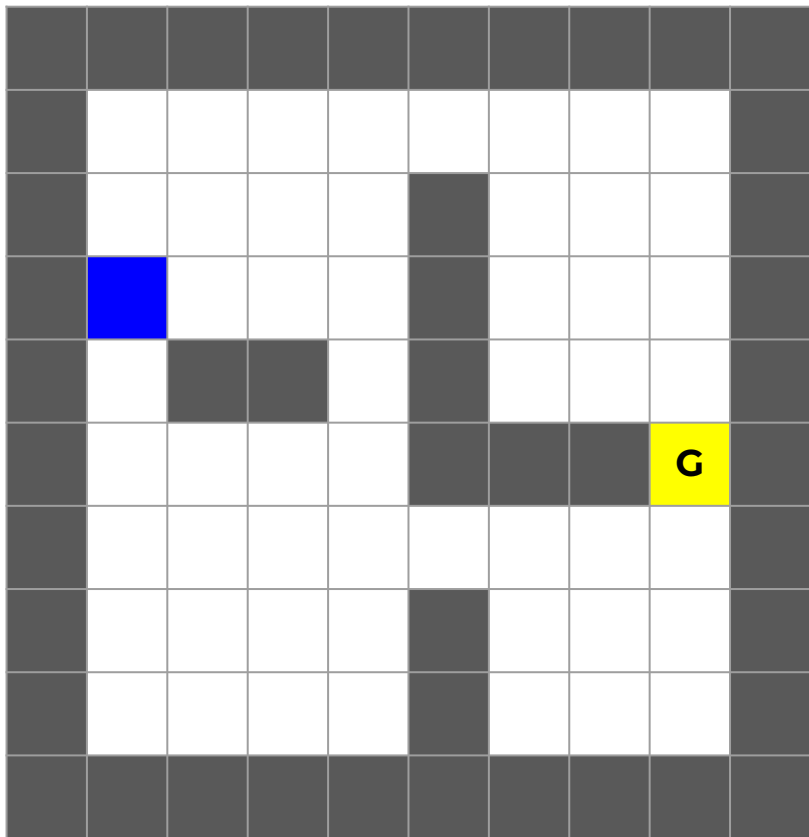


Example

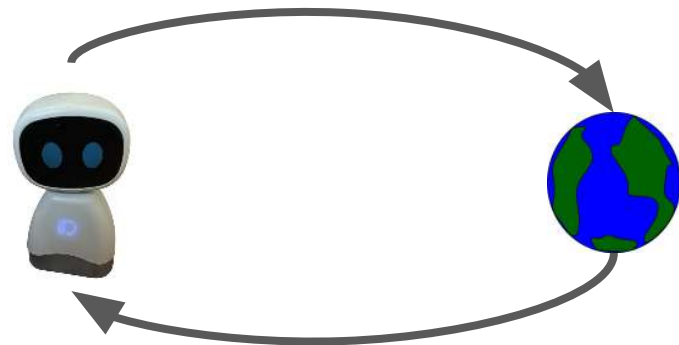
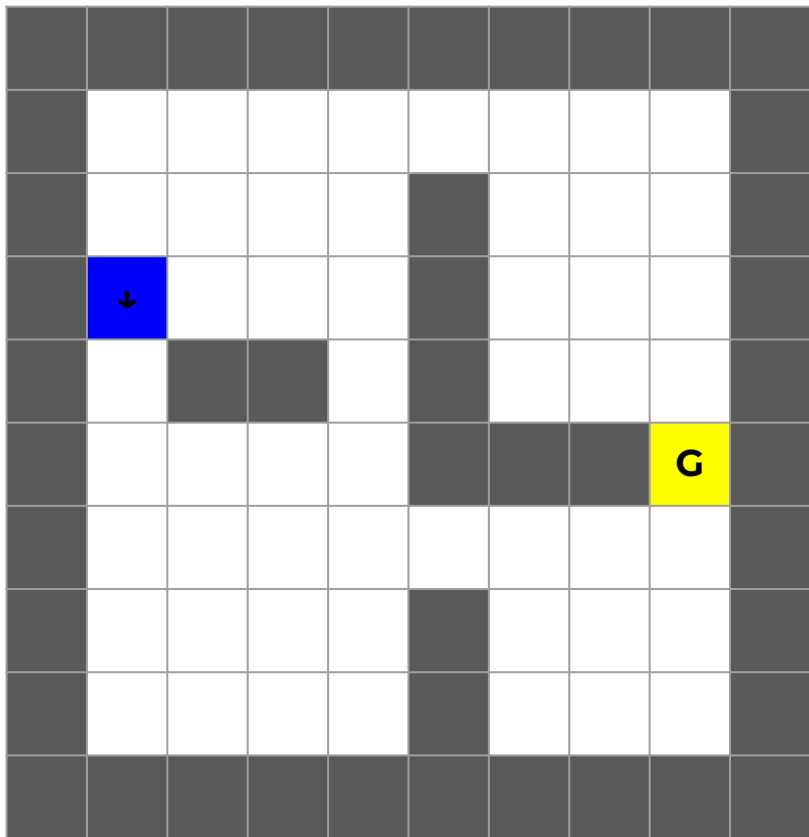
Example



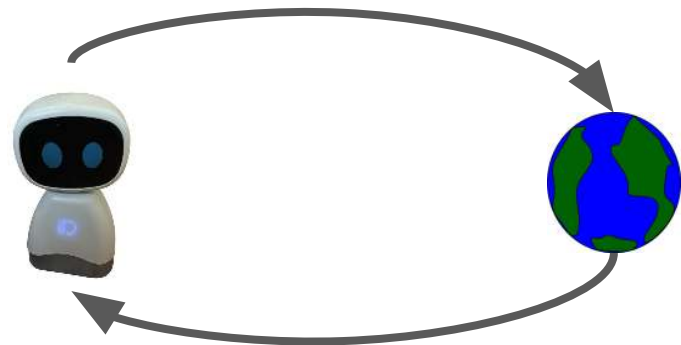
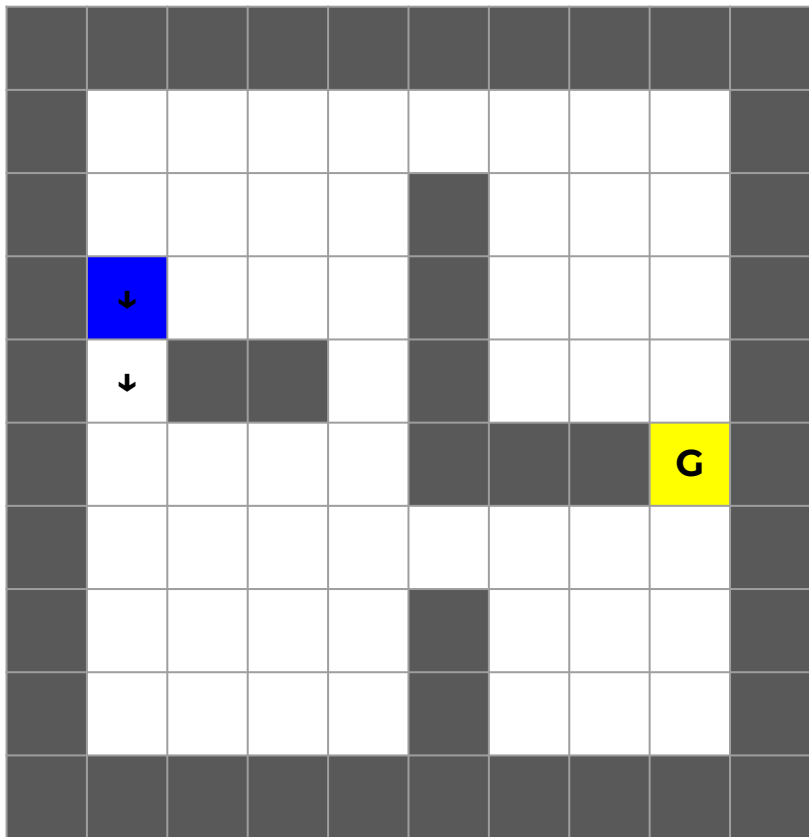
Example



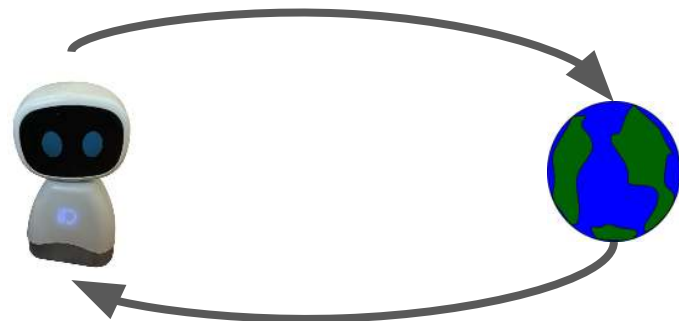
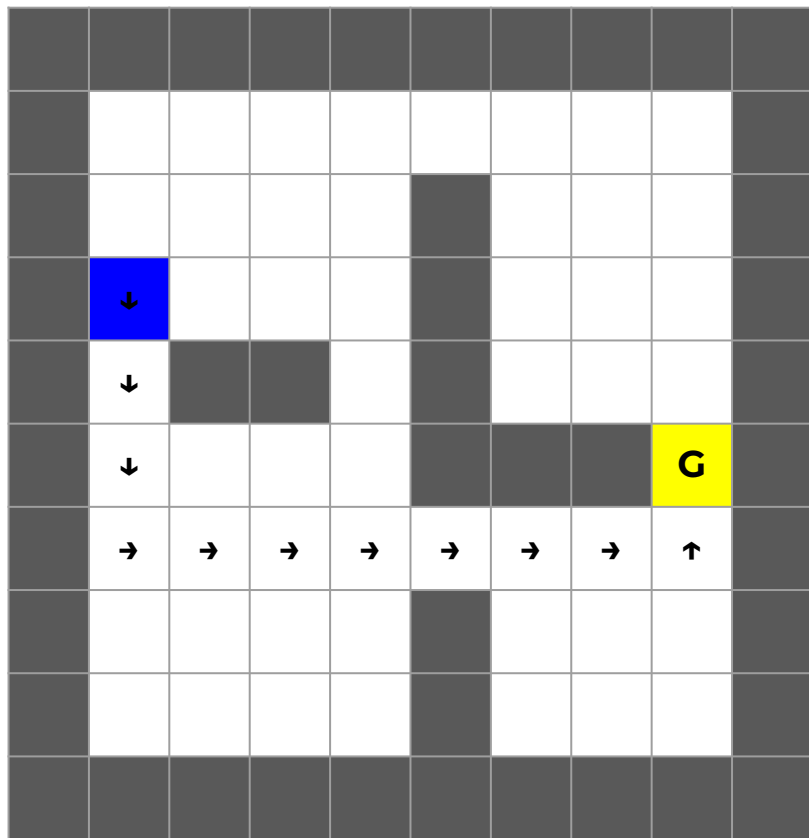
Example



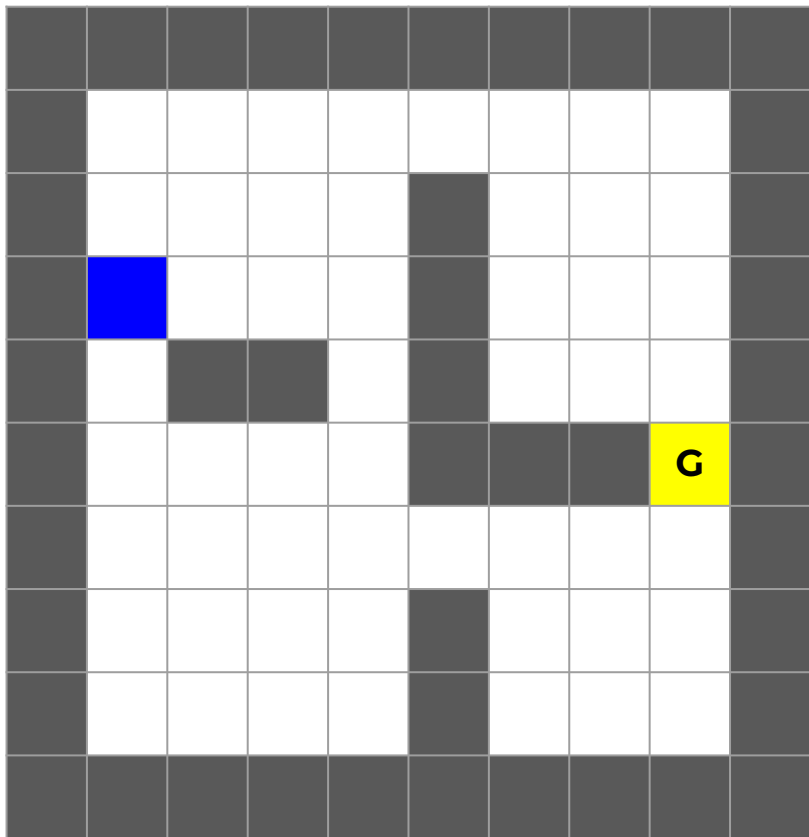
Example



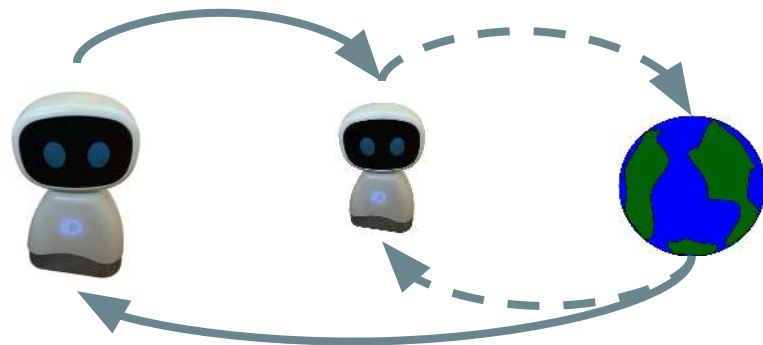
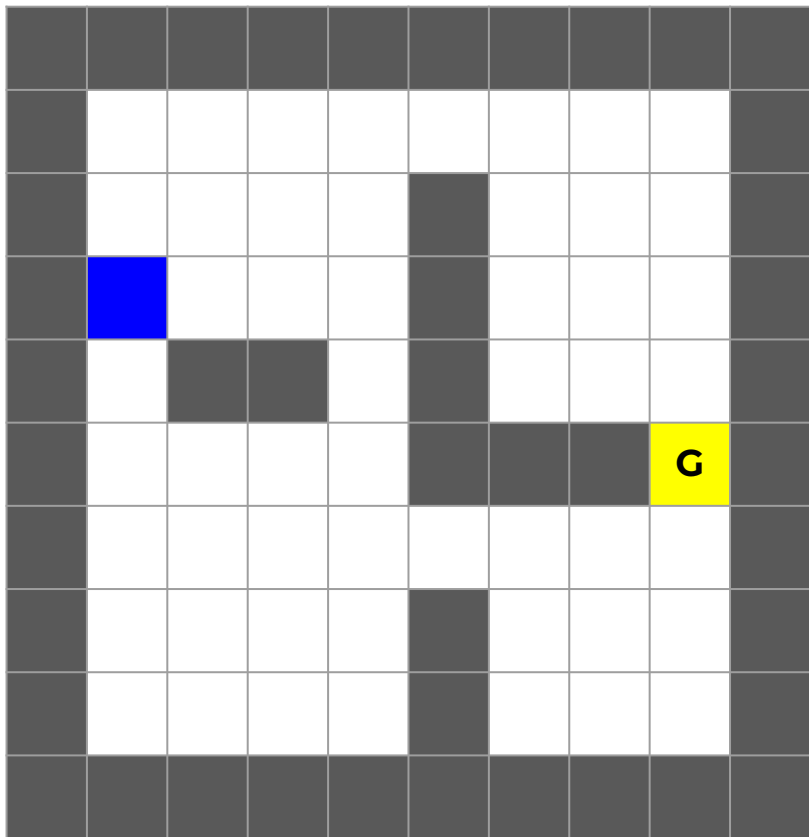
Example



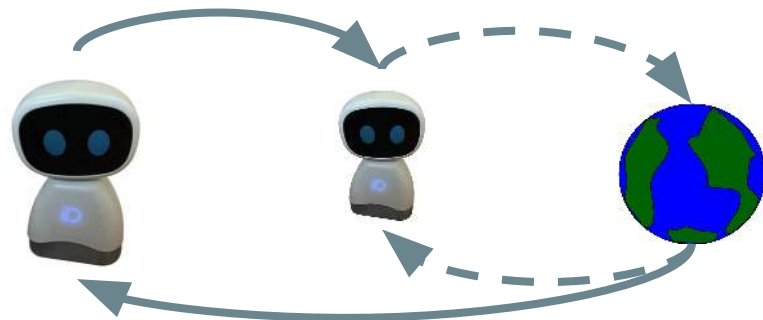
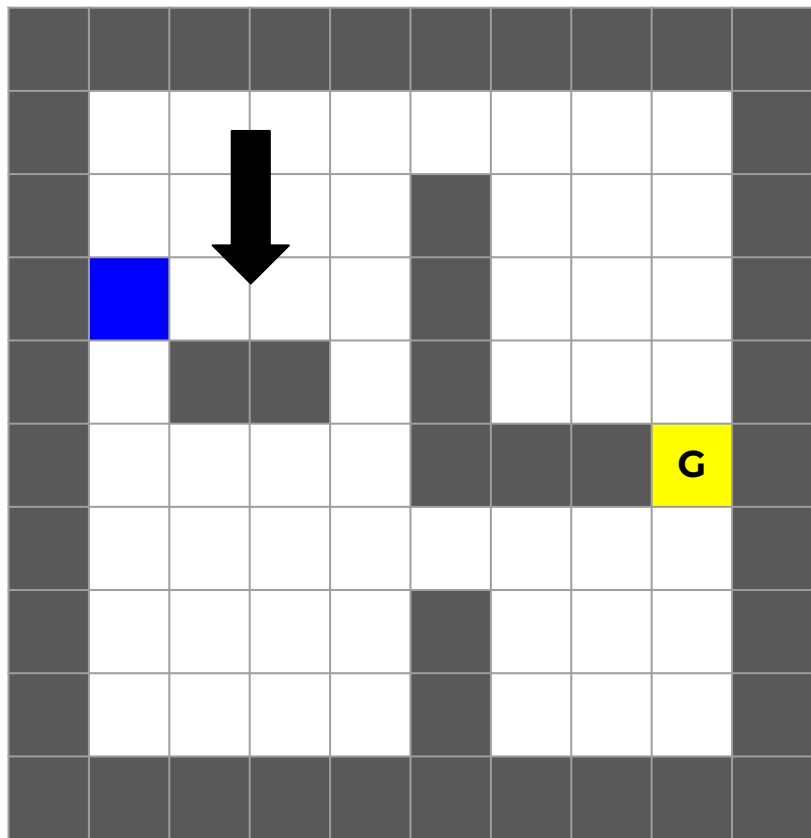
Example



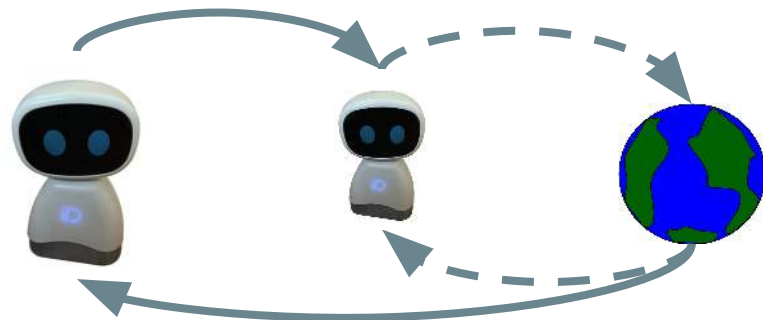
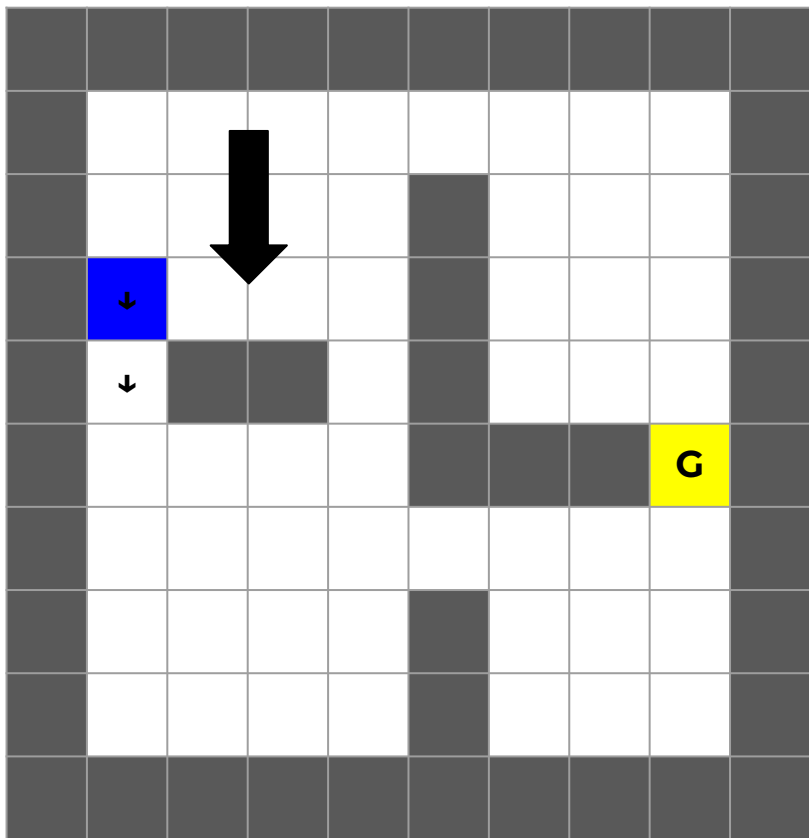
Example



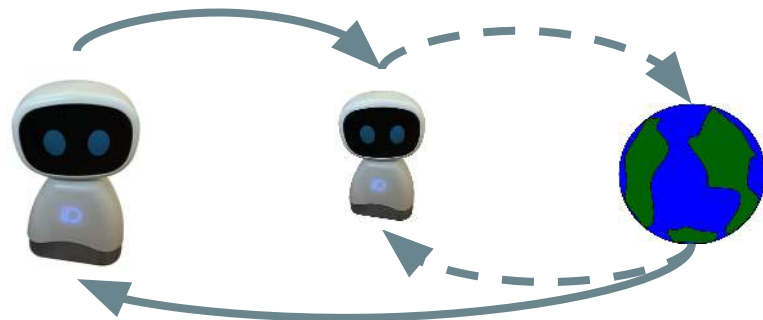
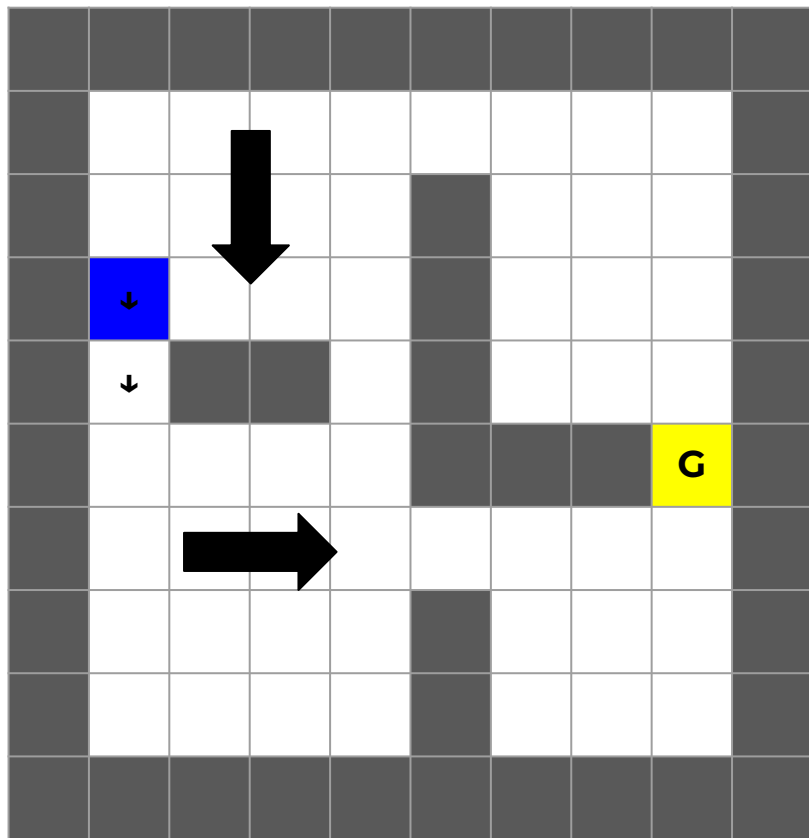
Example



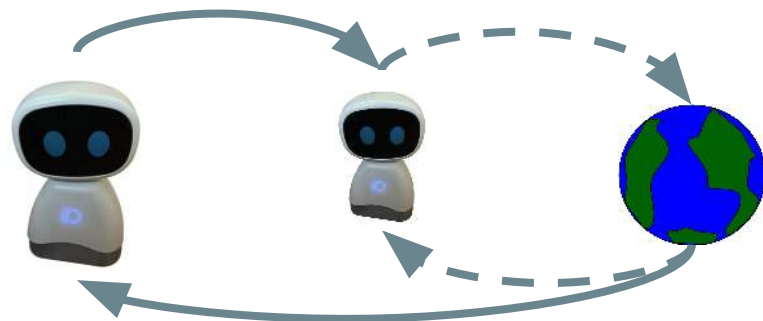
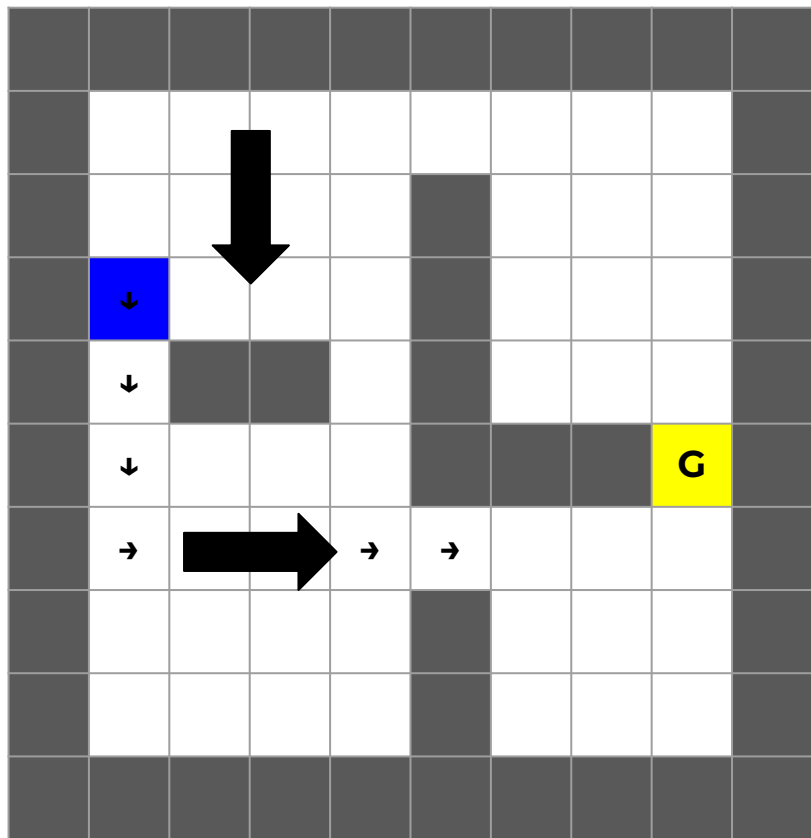
Example



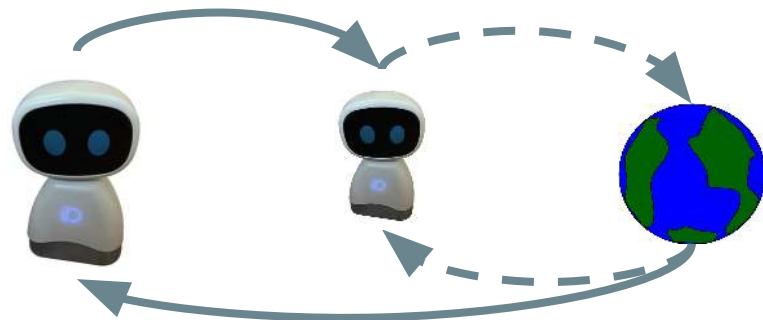
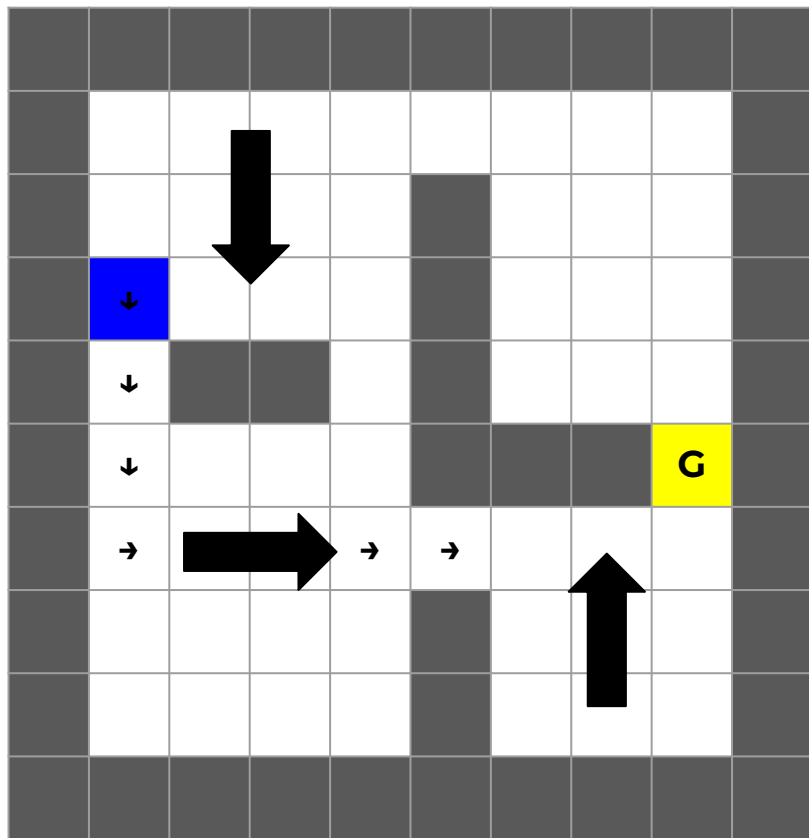
Example



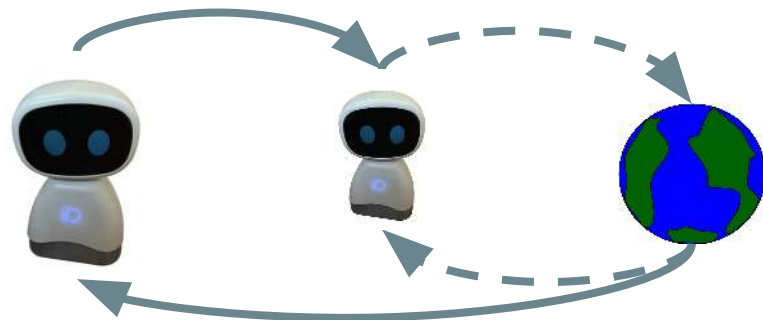
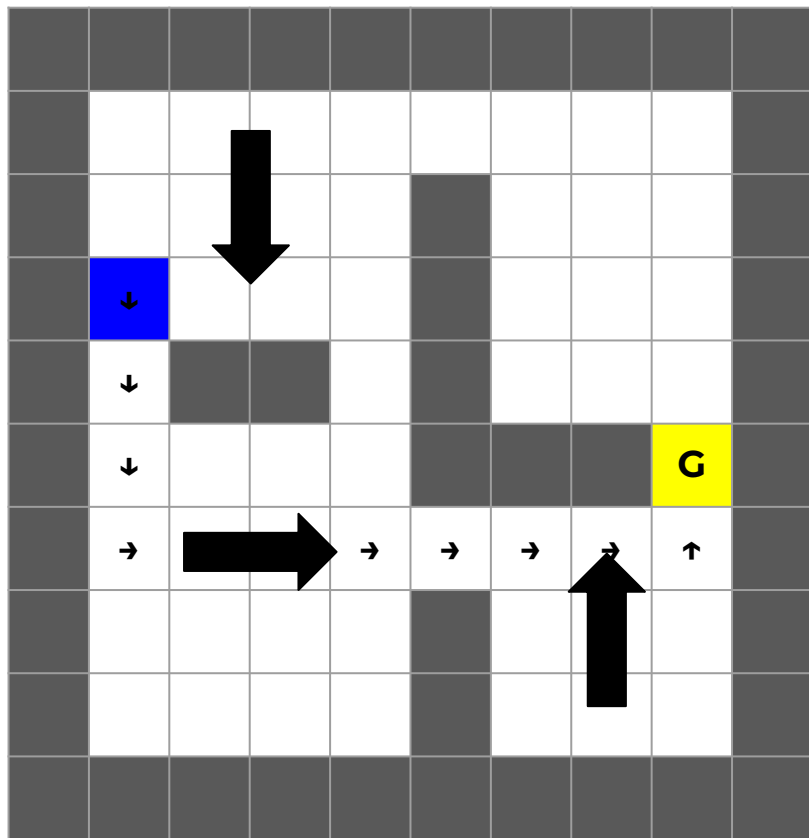
Example



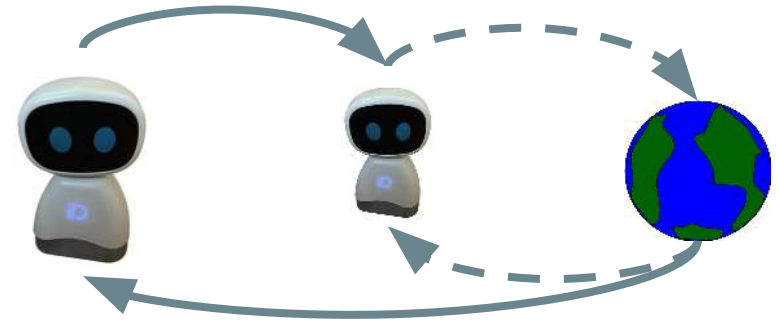
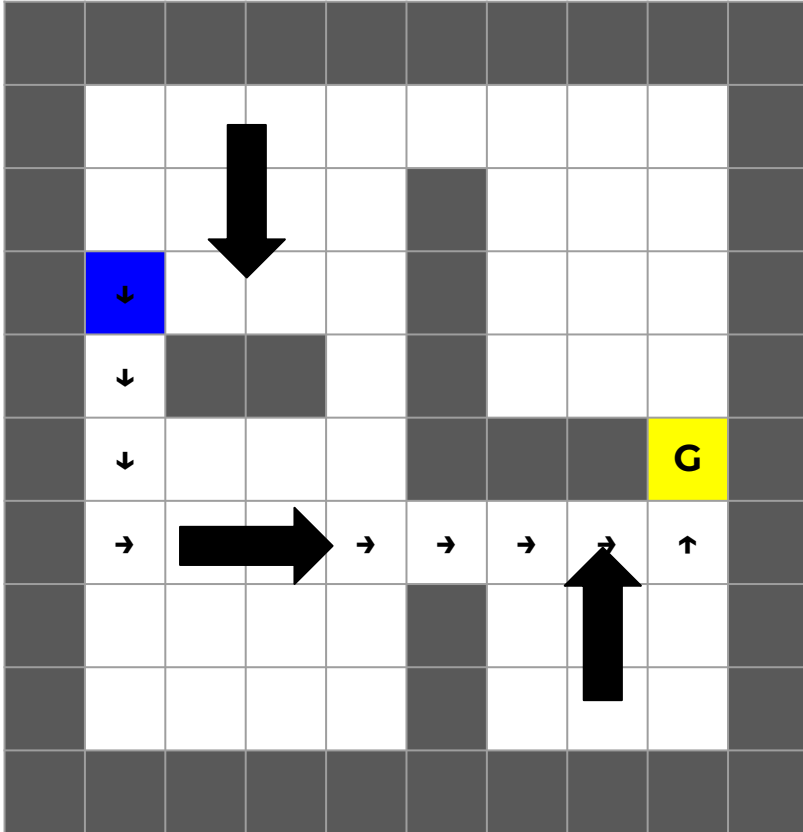
Example



Example



Example



How can we work with hierarchical decision making?

Frameworks

Frameworks

Frameworks

- Several different frameworks:

Frameworks

- Several different frameworks:
 - Feudal hierarchies (Dayan and Hinton, 1992);
 - Hierarchical abstract machines (Parr and Russell, 1997);
 - MAXQ decomposition (Dietterich, 1999);
 - Options (Sutton, Precup and Singh, 1999).

Frameworks

- Several different frameworks:
 - Feudal hierarchies (Dayan and Hinton, 1992);
 - ~~— Hierarchical abstract machines (Parr and Russell, 1997);~~
 - ~~— MAXQ decomposition (Dietterich, 1999);~~
 - Options (Sutton, Precup and Singh, 1999).

Frameworks

- Several different frameworks:
 - Feudal hierarchies (Dayan and Hinton, 1992);
 - ~~— Hierarchical abstract machines (Parr and Russell, 1997);~~
 - ~~— MAXQ decomposition (Dietterich, 1999);~~
 - Options (Sutton, Precup and Singh, 1999).

- Several different terms for similar concepts:

Frameworks

- Several different frameworks:
 - Feudal hierarchies (Dayan and Hinton, 1992);
 - ~~— Hierarchical abstract machines (Parr and Russell, 1997);~~
 - ~~— MAXQ decomposition (Dietterich, 1999);~~
 - Options (Sutton, Precup and Singh, 1999).

- Several different terms for similar concepts:
 - Task / subtask;
 - Goal / subgoal;
 - Option;
 - Skill.

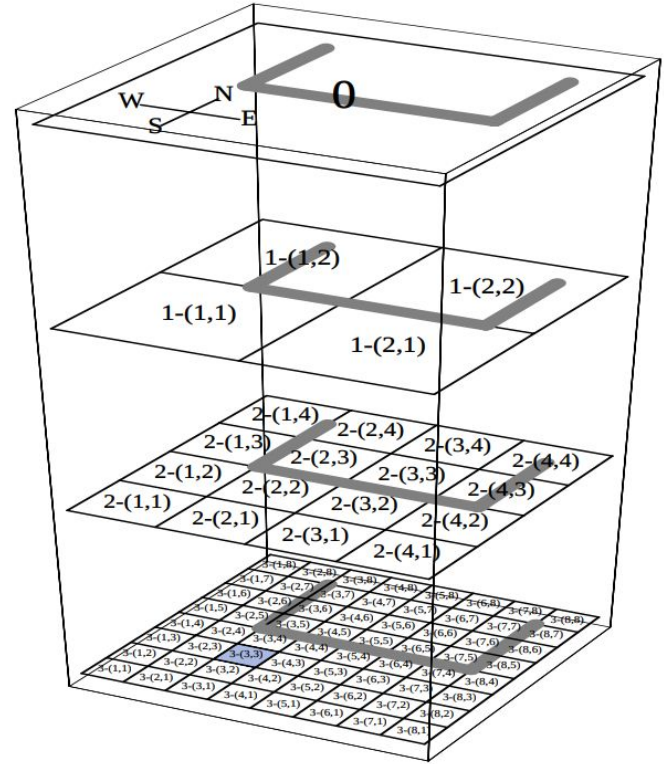
Feudal hierarchies

Feudal hierarchies

- Multiple agents operate at different levels

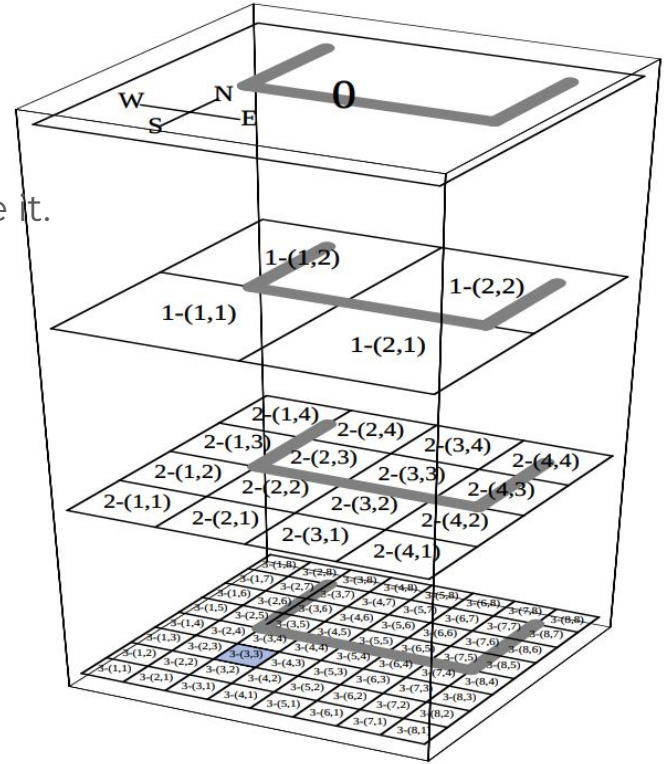
Feudal hierarchies

- Multiple agents operate at different levels



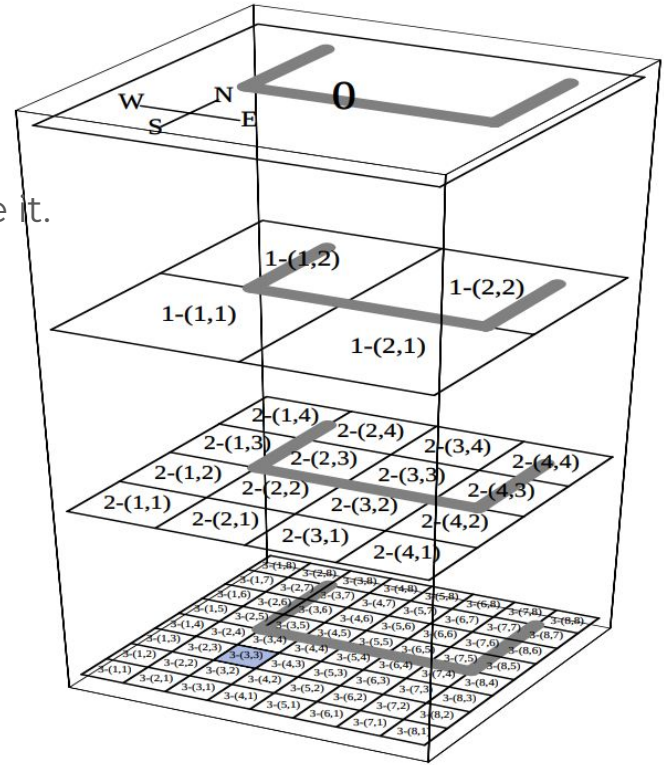
Feudal hierarchies

- Multiple agents operate at different levels:
 - High-level manager gives goal ω
 - Low-level worker observes ω and tries to achieve it.



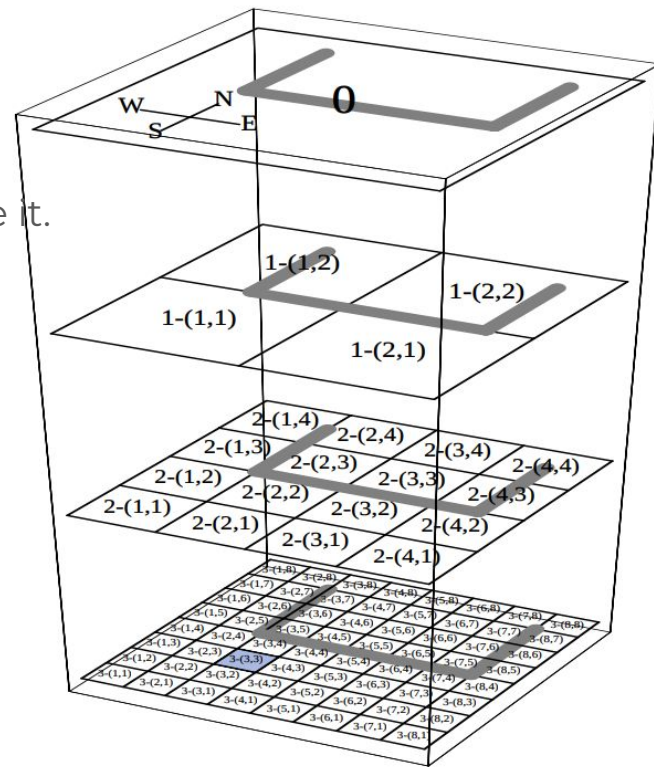
Feudal hierarchies

- Multiple agents operate at different levels:
 - High-level manager gives goal ω
 - Low-level worker observes ω and tries to achieve it.
- Important principles:



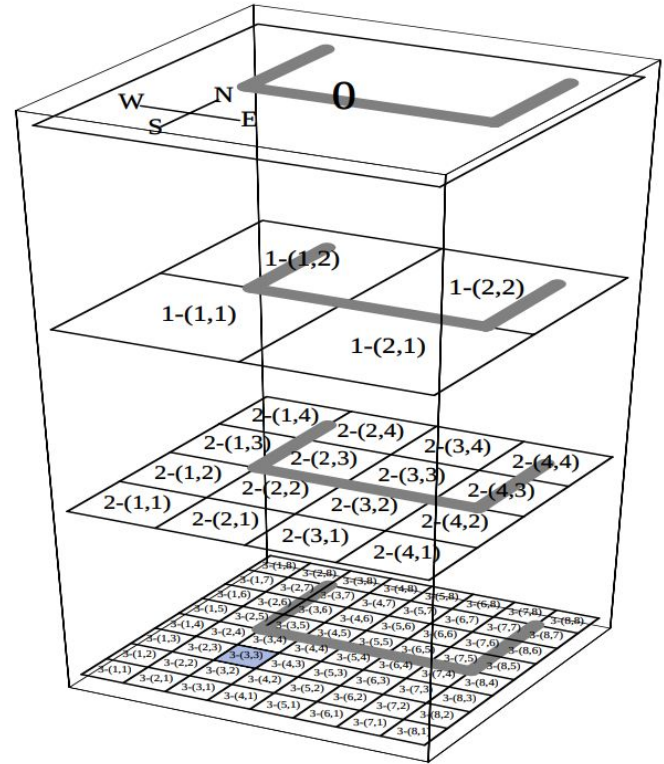
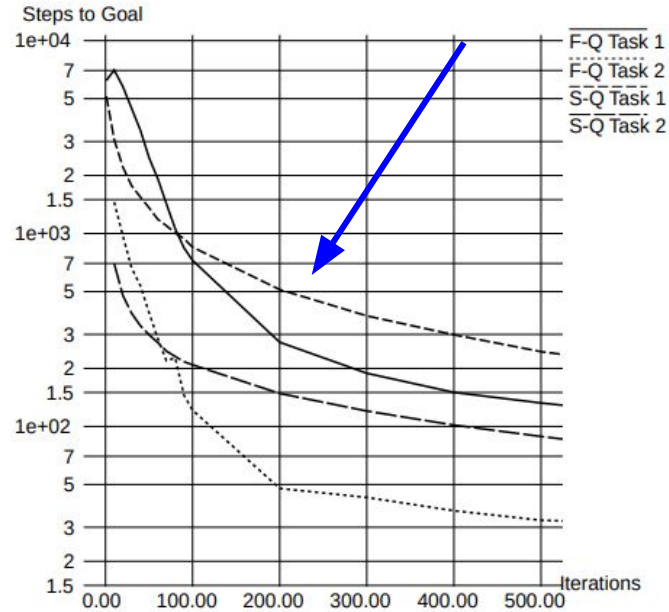
Feudal hierarchies

- Multiple agents operate at different levels:
 - High-level manager gives goal ω
 - Low-level worker observes ω and tries to achieve it.
- Important principles:
 - Reward hiding;
 - Information hiding.



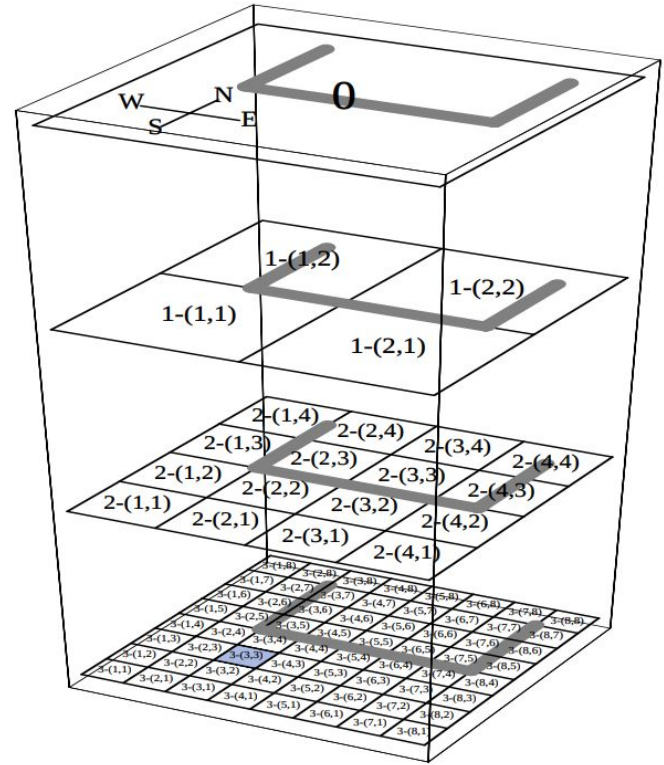
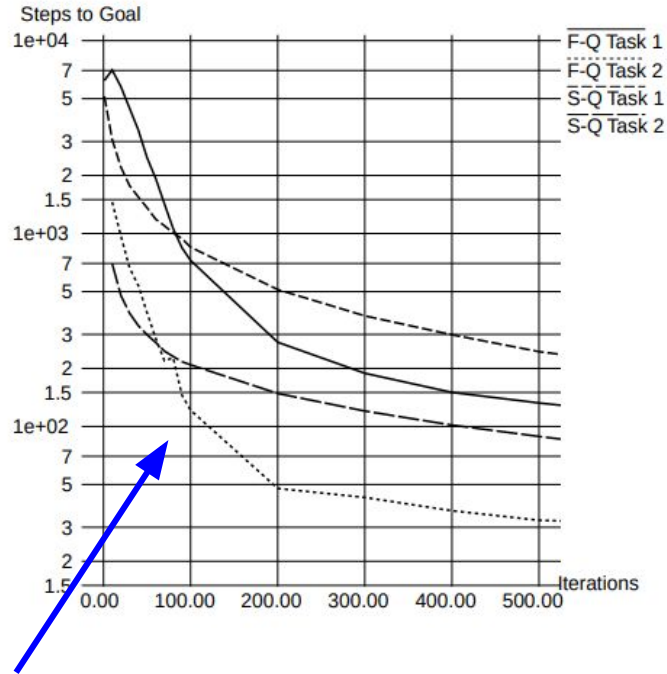
Feudal hierarchies

- Results:
 - Efficiency;



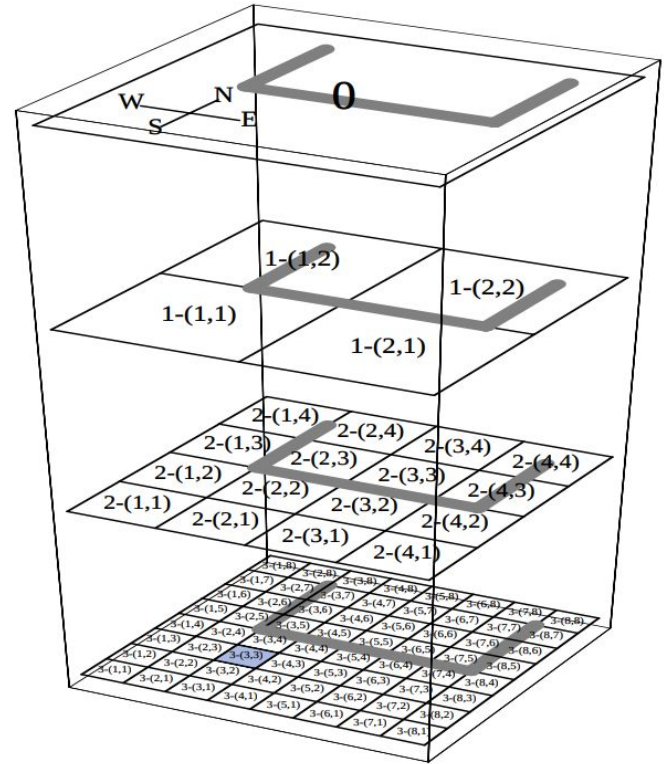
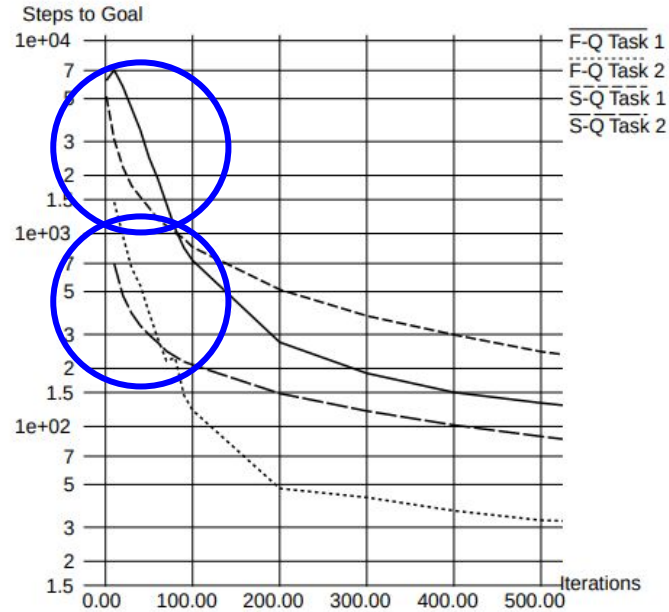
Feudal hierarchies

- Results:
 - Efficiency;
 - Transfer.



Feudal hierarchies

- Results:
 - Efficiency;
 - Transfer.



Feudal hierarchies

- Non-stationarity problem

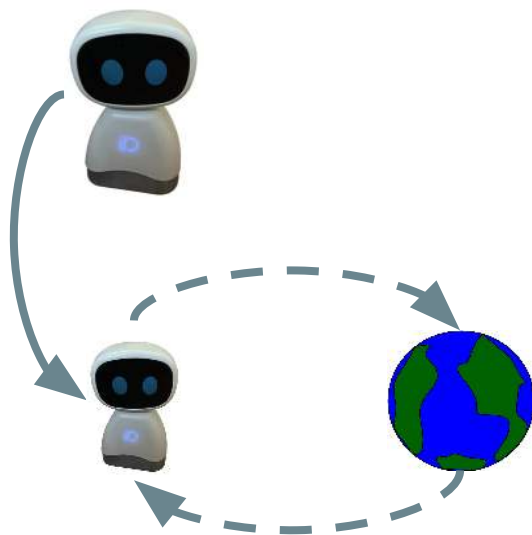
Feudal hierarchies

- Non-stationarity problem



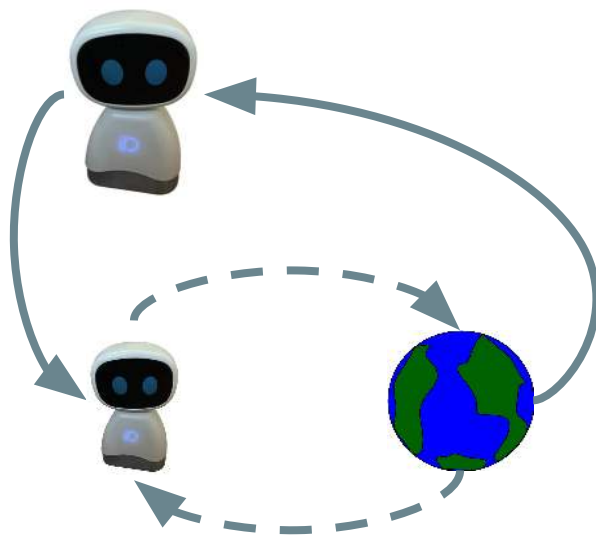
Feudal hierarchies

- Non-stationarity problem



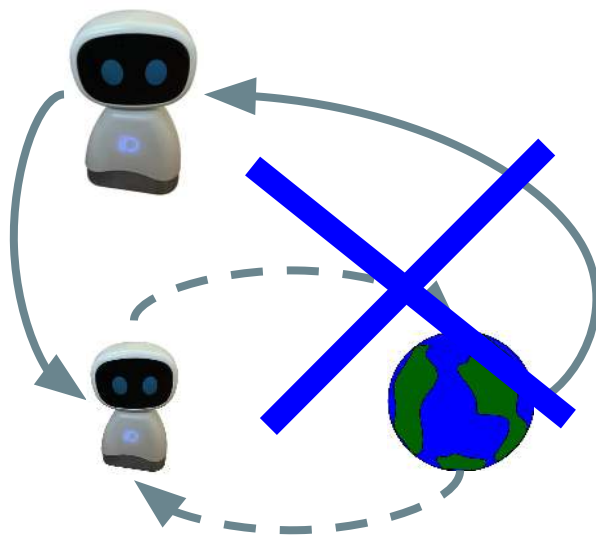
Feudal hierarchies

- Non-stationarity problem



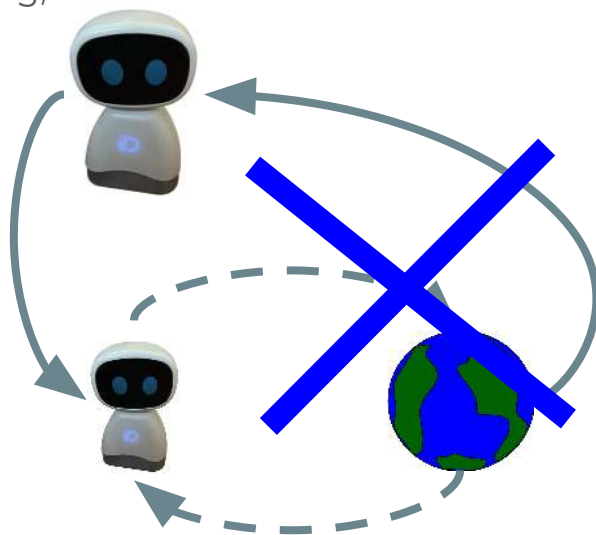
Feudal hierarchies

- Non-stationarity problem



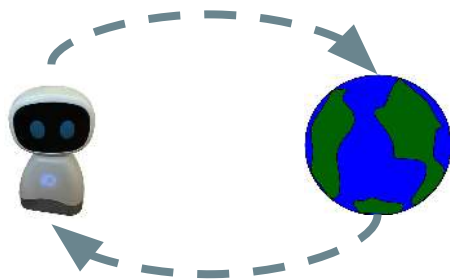
Feudal hierarchies

- Non-stationarity problem:
 - Low-level pre-training;



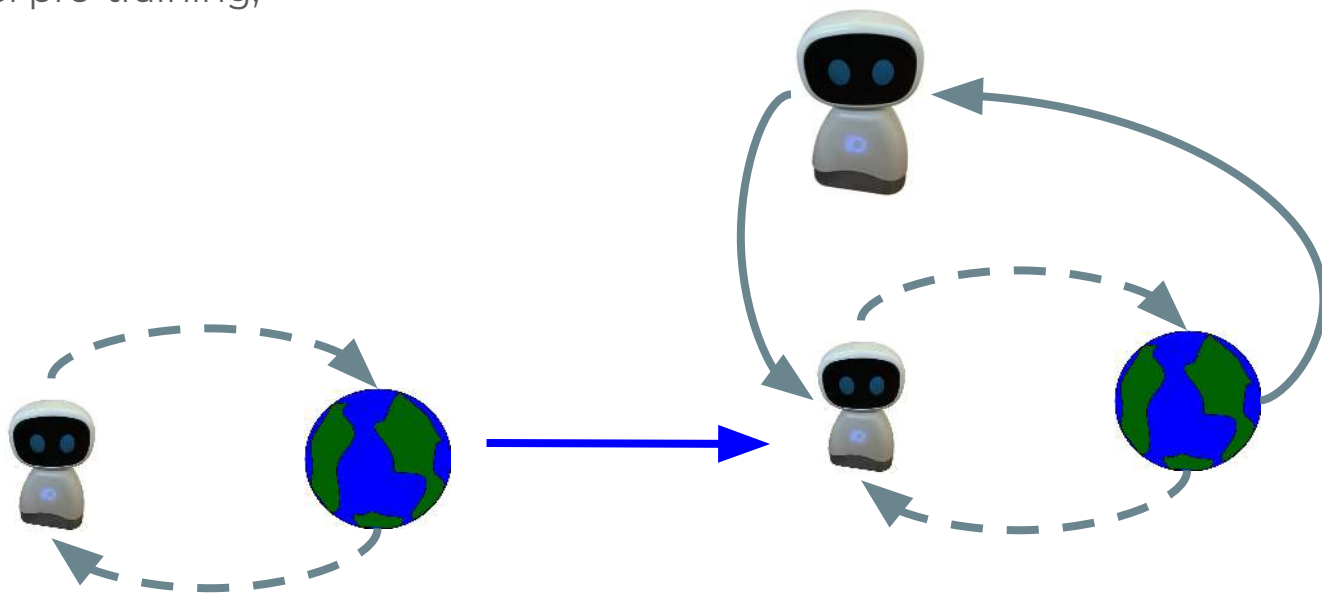
Feudal hierarchies

- Non-stationarity problem:
 - Low-level pre-training;



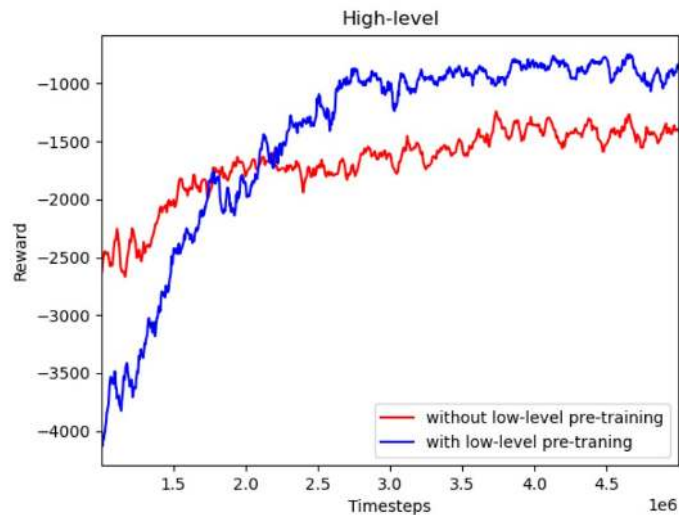
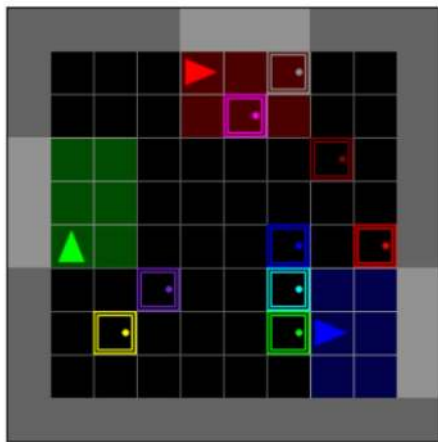
Feudal hierarchies

- Non-stationarity problem:
 - Low-level pre-training;



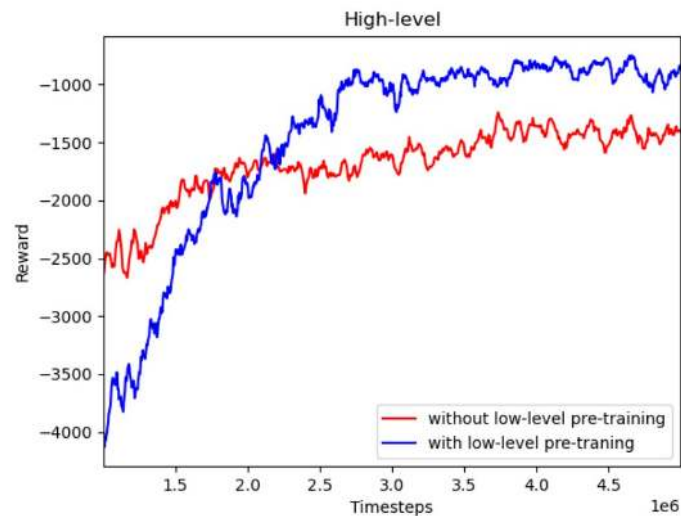
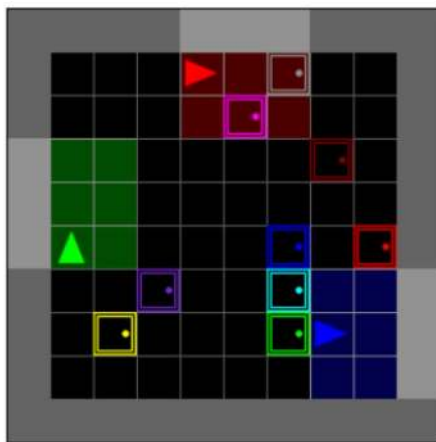
Feudal hierarchies

- Non-stationarity problem:
 - Low-level pre-training (Carvalho and Sengupta, 2022);



Feudal hierarchies

- Non-stationarity problem:
 - Low-level pre-training (Carvalho and Sengupta, 2022);
 - Two time scales.



Feudal hierarchies

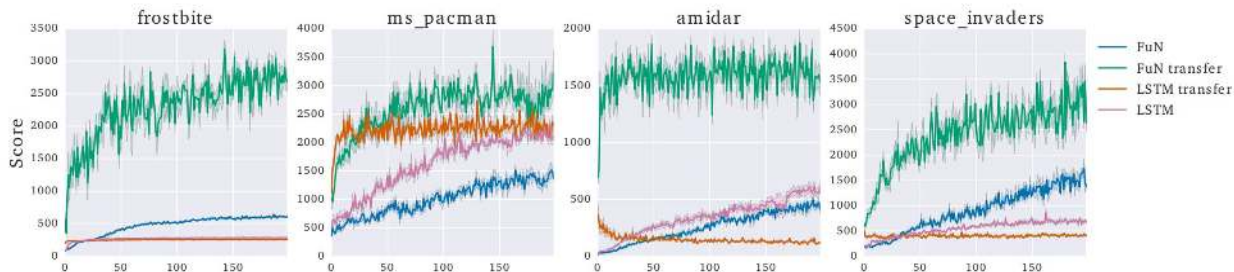
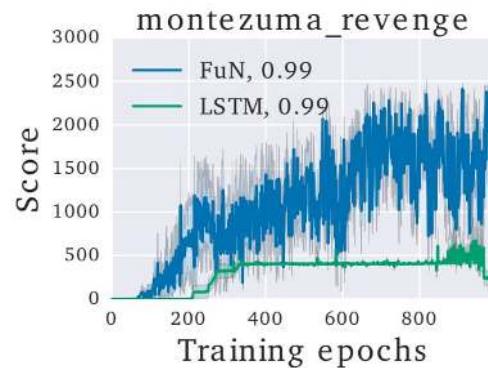
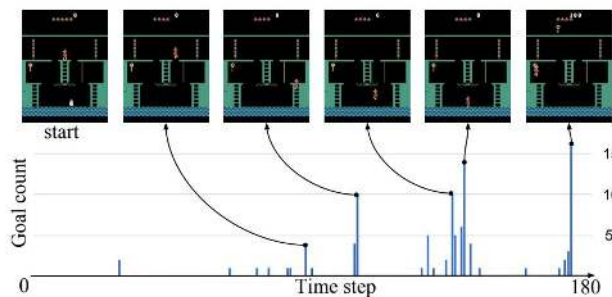
- Extensions

Feudal hierarchies

- Extensions to neural networks:

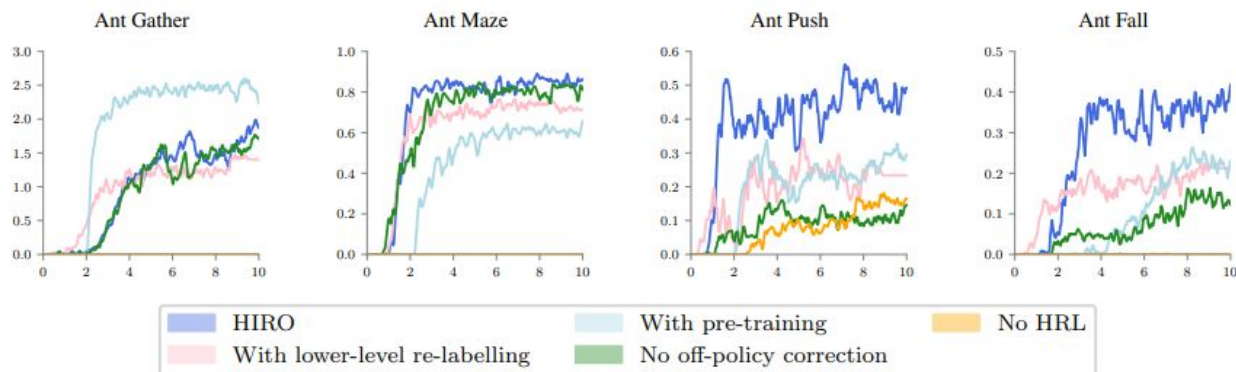
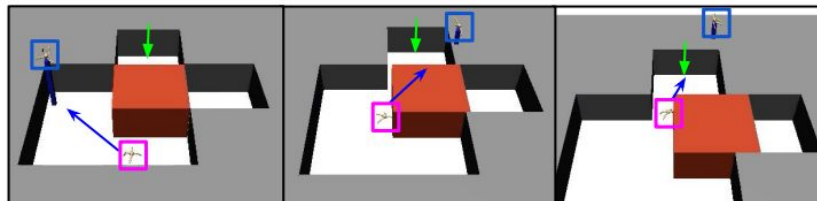
Feudal hierarchies

- Extensions to neural networks:
 - FuN (Vezhnevets et al., 2017);



Feudal hierarchies

- Extensions to neural networks:
 - HIRO (Nachum et al., 2018);

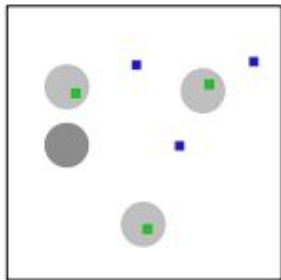


Feudal hierarchies

- Extensions

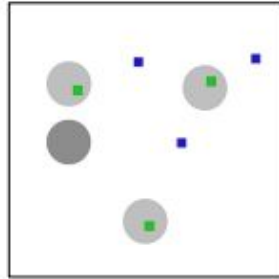
Feudal hierarchies

- Extensions to multi-agent:



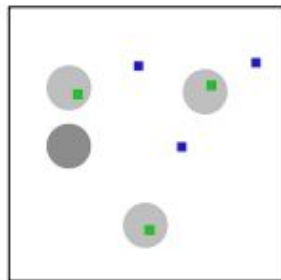
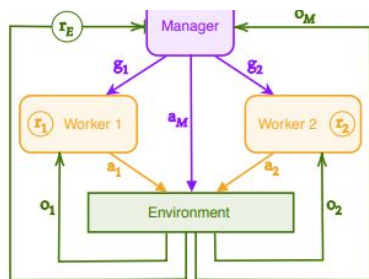
Feudal hierarchies

- Extensions to multi-agent:
 - Need to coordinate;
 - Exponential problem.



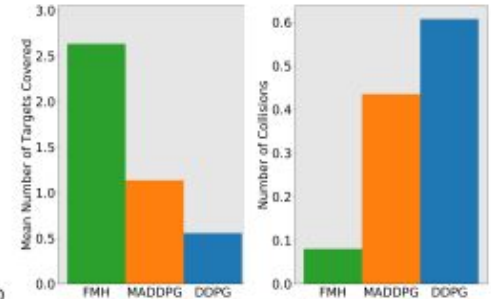
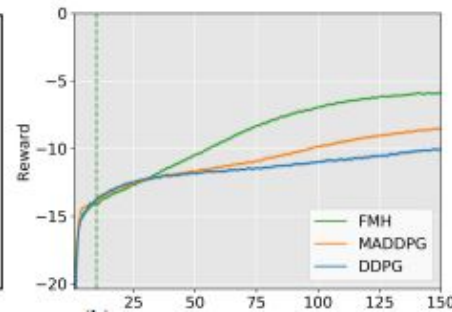
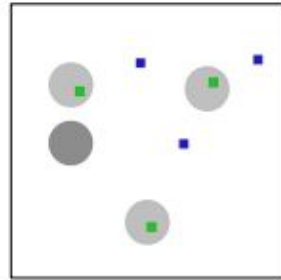
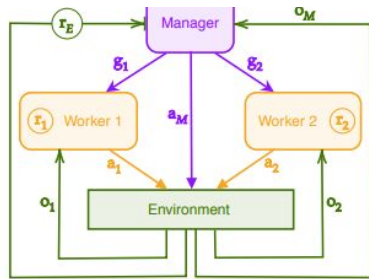
Feudal hierarchies

- Extensions to multi-agent:
 - FMH (Ahilan and Dayan, 2019);



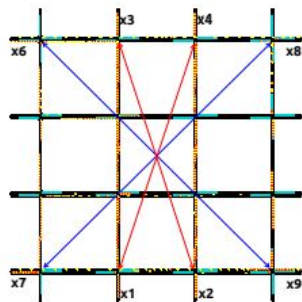
Feudal hierarchies

- Extensions to multi-agent:
 - FMH (Ahilan and Dayan, 2019);



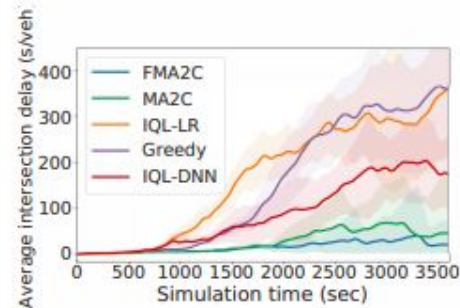
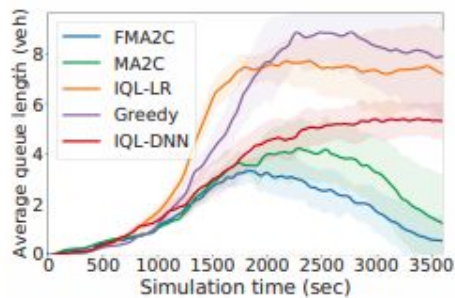
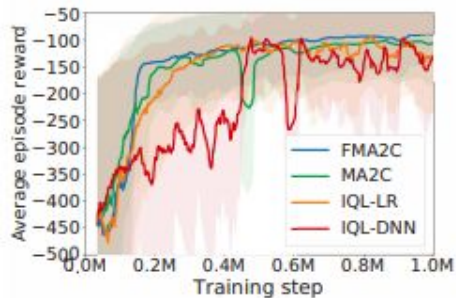
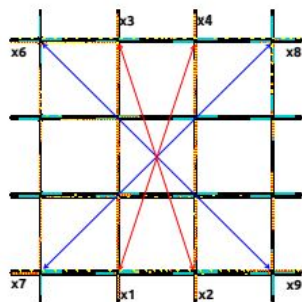
Feudal hierarchies

- Extensions to multi-agent:
 - FMA2C (Ma and Wu, 2020)



Feudal hierarchies

- Extensions to multi-agent:
 - FMA2C (Ma and Wu, 2020)



Feudal hierarchies: theory

Feudal hierarchies: theory

- Convergence of feudal reinforcement learning (Carvalho et al., 2023):

Feudal hierarchies: theory

- Convergence of feudal reinforcement learning (Carvalho et al., 2023):
 - **Assumption:** uniform stopping time:

Feudal hierarchies: theory

- Convergence of feudal reinforcement learning (Carvalho et al., 2023):
 - **Assumption:** uniform stopping time:
- **Conclusion:** Q-learning converges on feudal hierarchies.

Feudal hierarchies: theory

- Convergence of feudal reinforcement learning (Carvalho et al., 2023):
 - **Assumption:** uniform stopping time:
 - Each level is a non-stationary MDP $\mathcal{M}_t$
 - Despite, the MDPs converge $\mathcal{M}_t \rightarrow \mathcal{M}^*$
 - Q-learning converges on convergent MDPs $q_t \rightarrow q^*$
 - **Conclusion:** Q-learning converges on feudal hierarchies.

Feudal hierarchies

- Benefits

Feudal hierarchies

- Benefits:
 - Simplicity;
 - Efficiency;
 - Interpretability;

Feudal hierarchies

- Limitations:

Feudal hierarchies

- Limitations:
 - Domain knowledge;
 - Non-stationarity;
 - Theoretically under-explored.

Feudal hierarchies

- Limitations:
 - Domain knowledge;
 - Non-stationarity;
 - **Theoretically under-explored.**

Options

Options

- Introduced by Sutton, Precup and Singh (1999);

Options

- Introduced by Sutton, Precup and Singh (1999);
- Agent chooses temporally extended actions (options)

Options

- Introduced by Sutton, Precup and Singh (1999);
- Agent chooses temporally extended actions (options):
 - Proto-actions;
 - Macro-actions.

Options

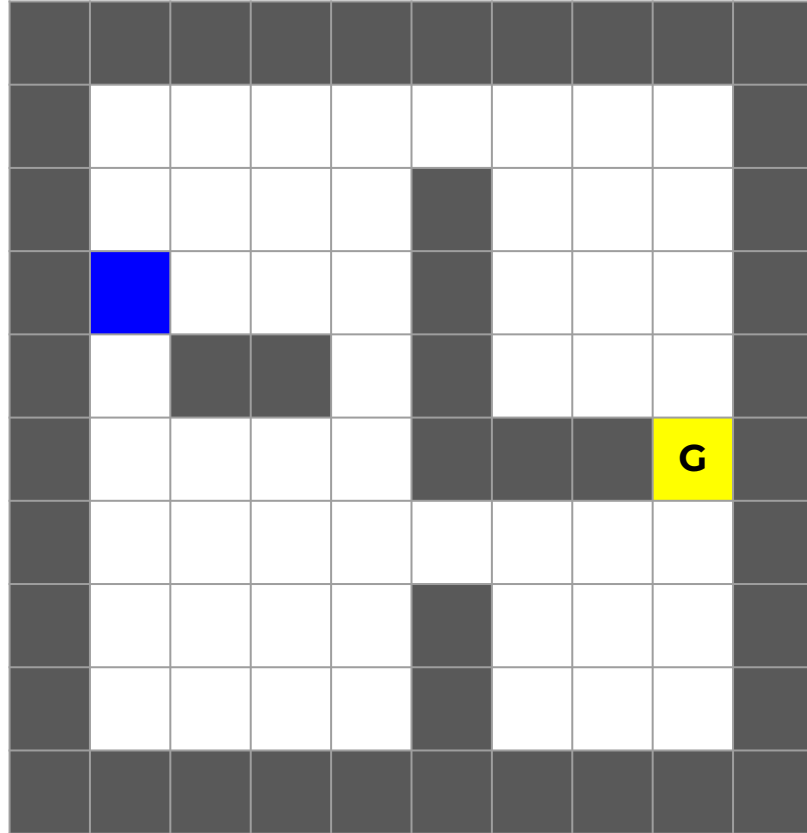
- Introduced by Sutton, Precup and Singh (1999);
- Agent chooses temporally extended actions (options):
 - Proto-actions (duration = 1 time step);
 - Macro-actions.

Options

- Introduced by Sutton, Precup and Singh (1999);
- Agent chooses temporally extended actions (options):
 - Proto-actions (duration = 1 time step);
 - Macro-actions (duration ≥ 1 time step).

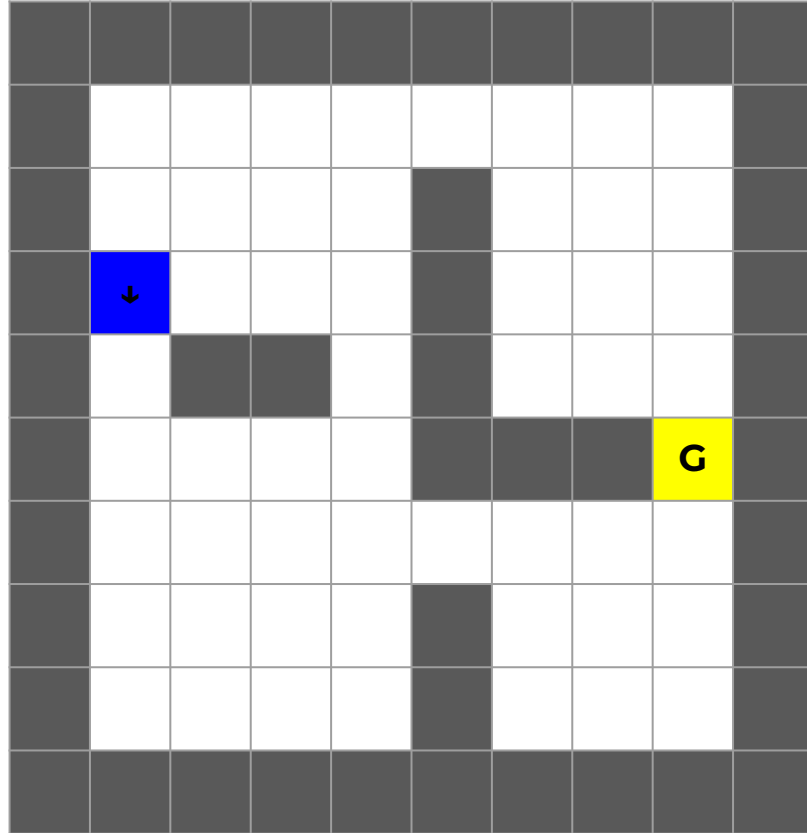
Options

- Proto-actions;



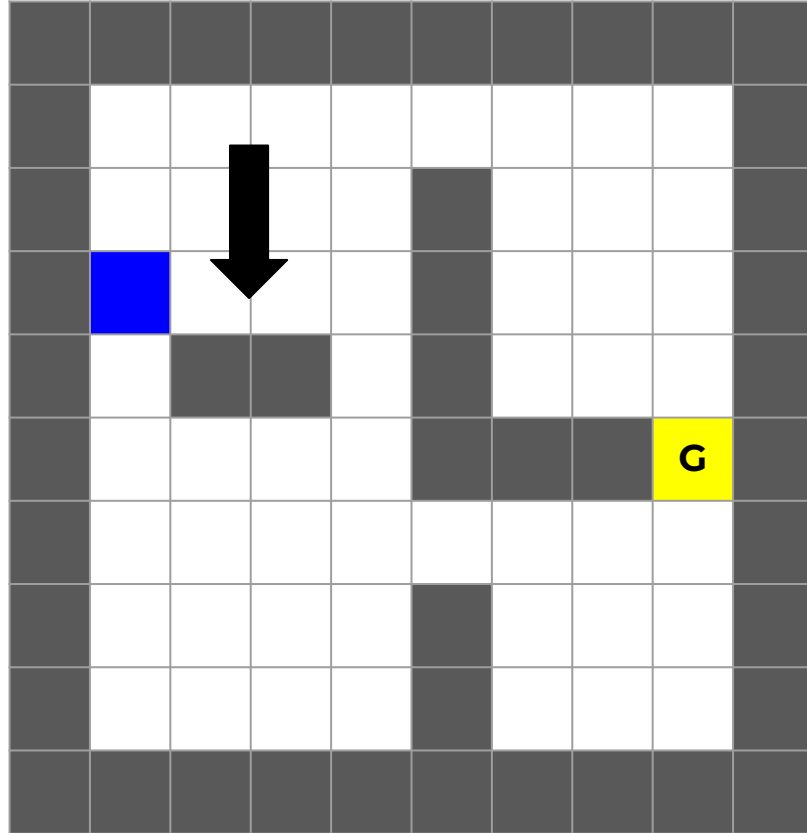
Options

- Proto-actions;



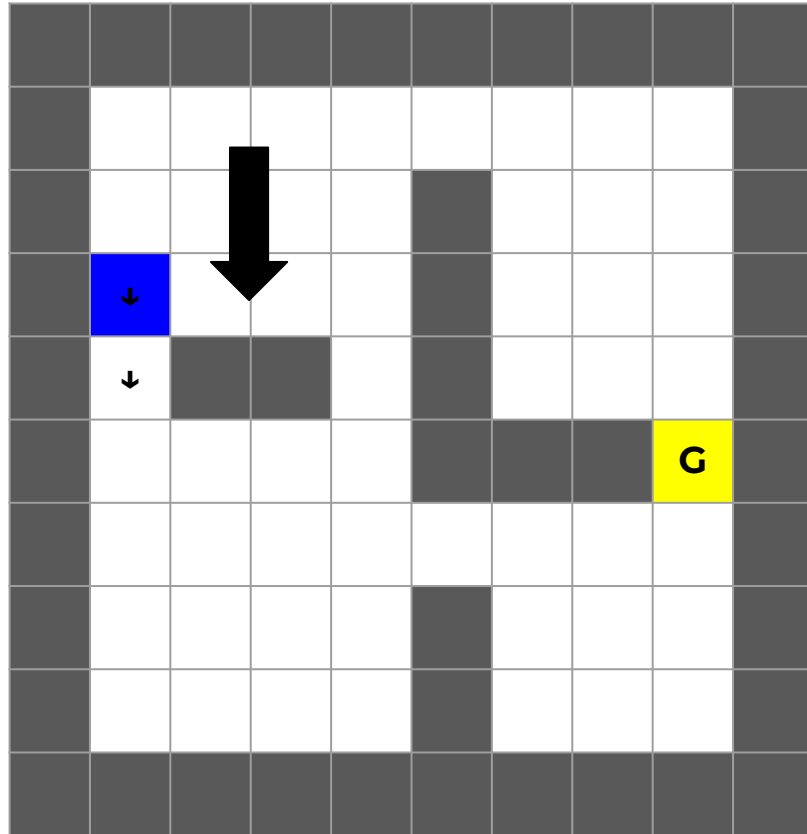
Options

- Proto-actions;
- Macro-actions.



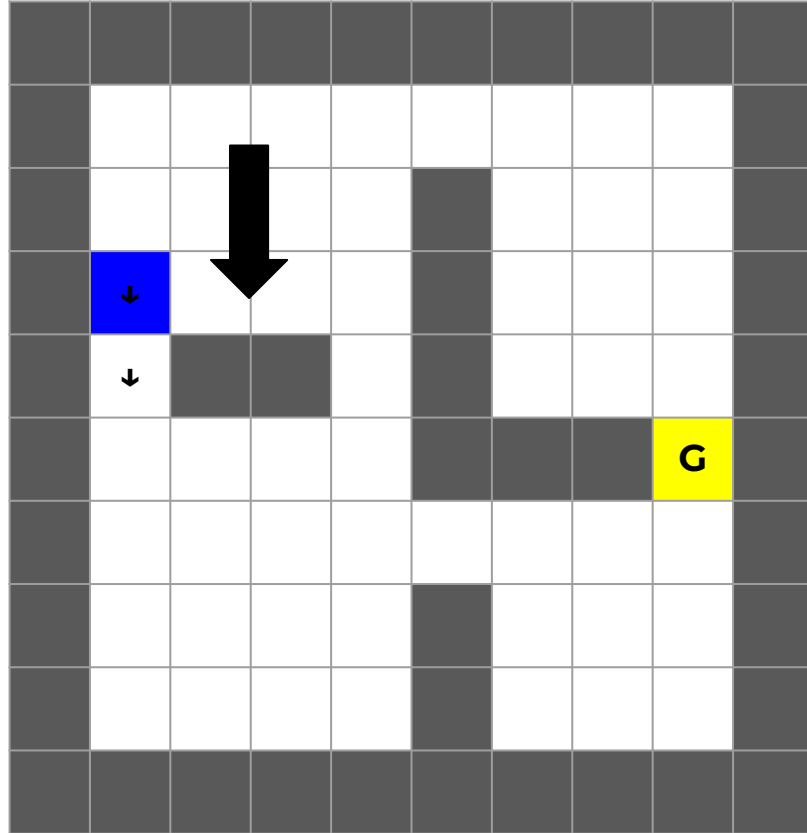
Options

- Proto-actions;
- Macro-actions.



Options

- Proto-actions;
- Macro actions.



Options: formalism

- An option is a triple $(\pi_\omega, \mathcal{I}_\omega, T_\omega)$
 - Policy
 - Initiation set
 - Termination condition

Options: formalism

- An option is a triple $(\pi_\omega, \mathcal{I}_\omega, T_\omega)$
 - Policy $\pi_\omega : \mathcal{X} \times \mathcal{A} \rightarrow [0, 1]$
 - Initiation set
 - Termination condition

Options: formalism

- An option is a triple $(\pi_\omega, \mathcal{I}_\omega, T_\omega)$
 - Policy $\pi_\omega : \mathcal{X} \times \mathcal{A} \rightarrow [0, 1]$
 - Initiation set $\mathcal{I}_\omega \subseteq \mathcal{X}$
 - Termination condition

Options: formalism

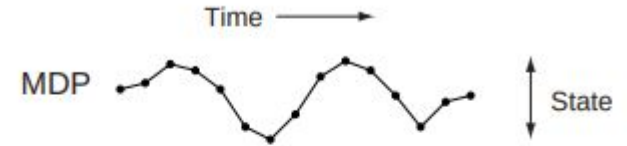
- An option is a triple $(\pi_\omega, \mathcal{I}_\omega, T_\omega)$
 - Policy $\pi_\omega : \mathcal{X} \times \mathcal{A} \rightarrow [0, 1]$
 - Initiation set $\mathcal{I}_\omega \subseteq \mathcal{X}$
 - Termination condition $T_\omega : \mathcal{X} \rightarrow [0, 1]$

Options: formalism

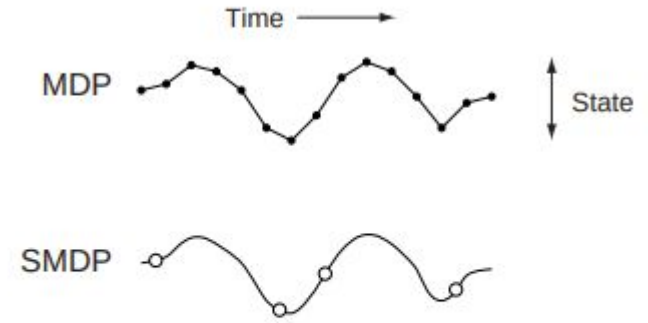
- An option is a triple $(\pi_\omega, \mathcal{I}_\omega, T_\omega)$
 - Policy $\pi_\omega : \mathcal{X} \times \mathcal{A} \rightarrow [0, 1]$
 - Initiation set $\mathcal{I}_\omega \subseteq \mathcal{X}$
 - Termination condition $T_\omega : \mathcal{X} \rightarrow [0, 1]$
- A proto-action can also be an option:
 - Policy $\pi_\omega(x, a) = 1(a = \omega)$
 - Initiation set $\mathcal{I}_\omega = \mathcal{X}$
 - Termination condition $T_\omega(x) = 1$

Options: semi-Markov decision process

Options: semi-Markov decision process

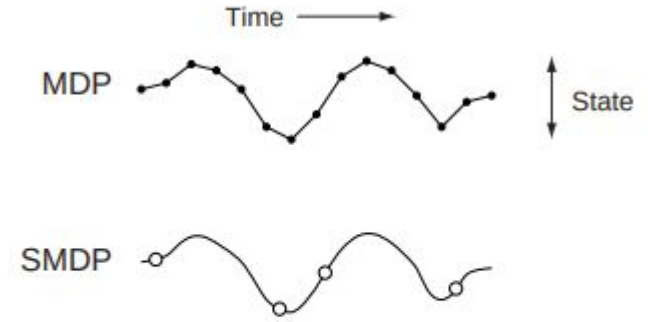


Options: semi-Markov decision process



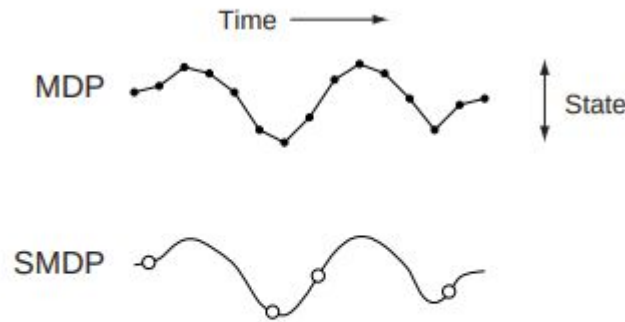
Options: semi-Markov decision process

- An SMDP (1963) is a tuple $(\mathcal{X}, \Omega, \tilde{R}, \tilde{P})$



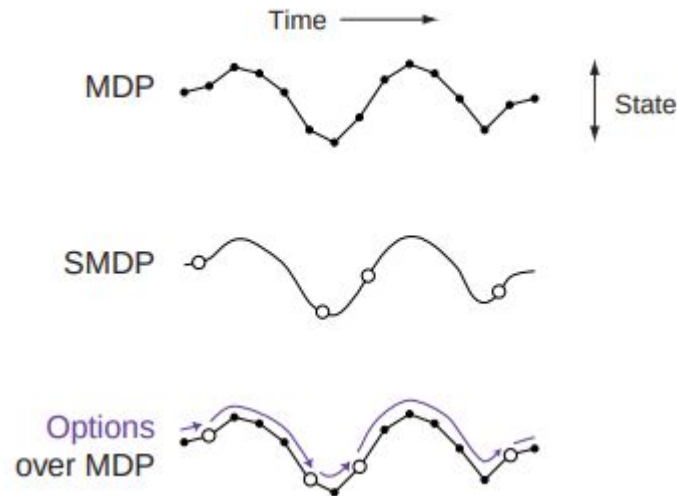
Options: semi-Markov decision process

- An SMDP (1963) is a tuple $(\mathcal{X}, \Omega, \tilde{R}, \tilde{P})$
 - Set of states $\mathcal{X}$
 - Set of actions with variable duration Ω
 - Reward function $\tilde{R}: \mathcal{X} \times \Omega \rightarrow \mathbb{R}$
 - Transition function $\tilde{P}: \mathcal{X} \times \Omega \times \mathcal{X} \times \mathbb{N} \rightarrow [0, 1]$



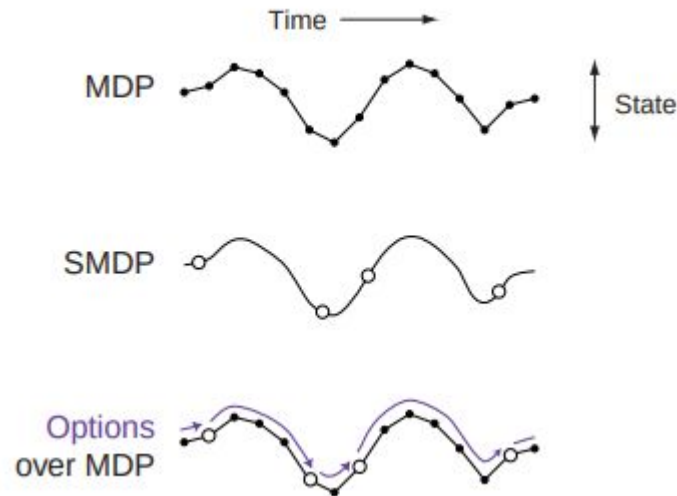
Options: semi-Markov decision process

- An SMDP (1963) is a tuple $(\mathcal{X}, \Omega, \tilde{R}, \tilde{P})$
 - Set of states $\mathcal{X}$
 - Set of actions with variable duration Ω
 - Reward function $\tilde{R}: \mathcal{X} \times \Omega \rightarrow \mathbb{R}$
 - Transition function $\tilde{P}: \mathcal{X} \times \Omega \times \mathcal{X} \times \mathbb{N} \rightarrow [0, 1]$



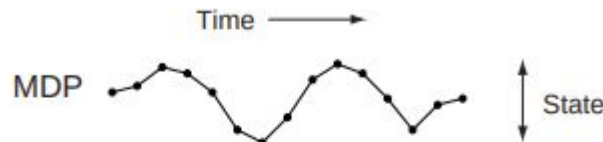
Options: semi-Markov decision process

- An SMDP (1963) is a tuple $(\mathcal{X}, \Omega, \tilde{R}, \tilde{P})$
 - Set of states $\mathcal{X}$
 - Set of actions with variable duration Ω
 - Reward function $\tilde{R}: \mathcal{X} \times \Omega \rightarrow \mathbb{R}$
 - Transition function $\tilde{P}: \mathcal{X} \times \Omega \times \mathcal{X} \times \mathbb{N} \rightarrow [0, 1]$
- Options over an MDP forms an SMDP:



Options: semi-Markov decision process

- An SMDP (1963) is a tuple $(\mathcal{X}, \Omega, \tilde{R}, \tilde{P})$
 - Set of states $\mathcal{X}$
 - Set of actions with variable duration Ω
 - Reward function $\tilde{R}: \mathcal{X} \times \Omega \rightarrow \mathbb{R}$
 - Transition function $\tilde{P}: \mathcal{X} \times \Omega \times \mathcal{X} \times \mathbb{N} \rightarrow [0, 1]$



- Options over an MDP forms an SMDP:

- Set of states $\mathcal{X}$
- Set of options Ω
- Reward function $\tilde{R}(x, \omega) = \mathbb{E}[\sum_{t=0}^{\tau} \gamma^t R(x_t, \pi_{\omega}(x_t)) \mid x_0 = x]$
- Transition function



$$\tilde{P}(x, \omega, x', n) = \sum_{x_1, \dots, x_{n-1}} P(x_{n-1}, \pi_{\omega}(x_{n-1}), x') T_{\omega}(x') \cdot \prod_{i=1}^{n-1} P(x_{i-1}, \pi_{\omega}(x_{i-1}), x_i) (1 - T_{\omega}(x_i))$$

Options: semi-Markov decision process

- Optimality equation

Options: semi-Markov decision process

- Optimality equation:

- MDP

$$q^*(x, a) = R(x, a) + \gamma \cdot \sum_{x'} P(x, a, x') \cdot \max_{a'} q^*(x', a')$$

Options: semi-Markov decision process

- Optimality equation:

- MDP

$$q^*(x, a) = R(x, a) + \gamma \cdot \sum_{x'} P(x, a, x') \cdot \max_{a'} q^*(x', a')$$

- SMDP

$$\tilde{q}^*(x, \omega) = \tilde{R}(x, \omega) + \gamma \cdot \sum_{x'} \sum_n \tilde{P}(x, \omega, x', n) \gamma^{n-1} \cdot \max_{\omega'} q^*(x', \omega')$$

Options: semi-Markov decision process

- Q-learning:

Options: semi-Markov decision process

- Q-learning:

- MDP

$$q_{t+1}(x, a) = q_t(x, a) + \alpha_t(r + \gamma \cdot \max_{a'} q_t(x', a') - q_t(x, a))$$

Options: semi-Markov decision process

- Q-learning:

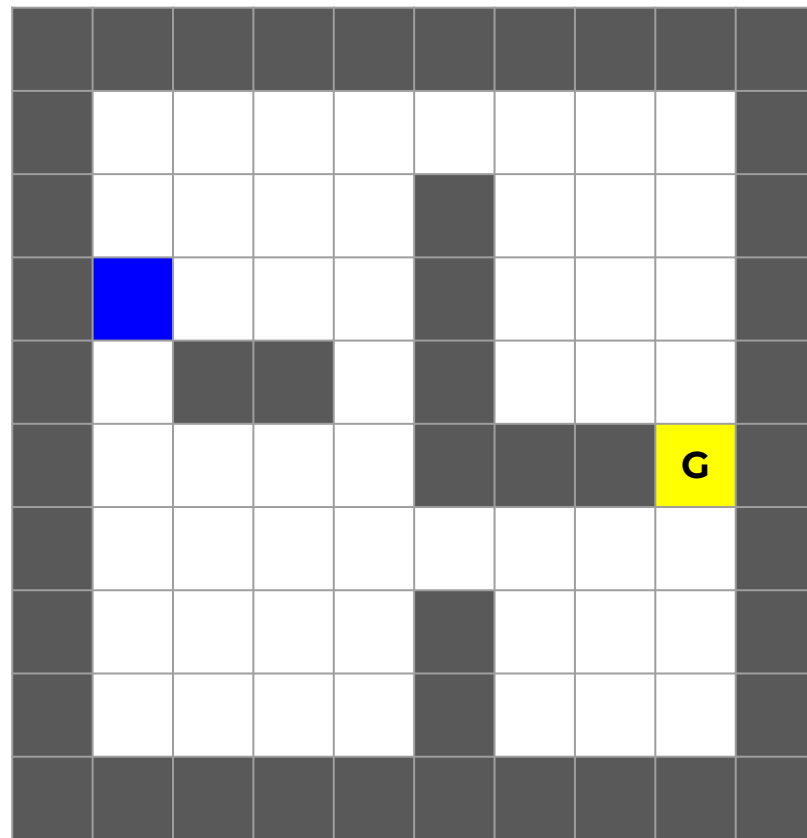
- MDP

$$q_{t+1}(x, a) = q_t(x, a) + \alpha_t(r + \gamma \cdot \max_{a'} q_t(x', a') - q_t(x, a))$$

- SMDP

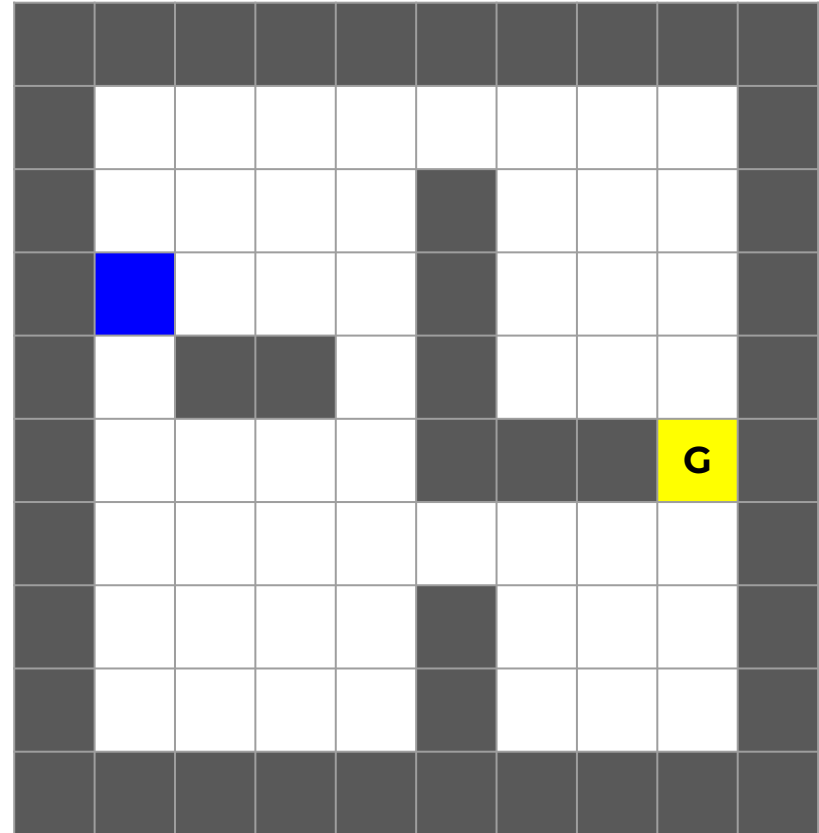
$$\tilde{q}_{t+1}(x, \omega) = \tilde{q}_t(x, \omega) + \alpha_t(\sum_{i=0, \dots, n-1} \gamma^i r_i + \gamma^n \cdot \max_{\omega'} \tilde{q}_t(x', \omega') - \tilde{q}_t(x, \omega))$$

Options: semi-Markov decision process



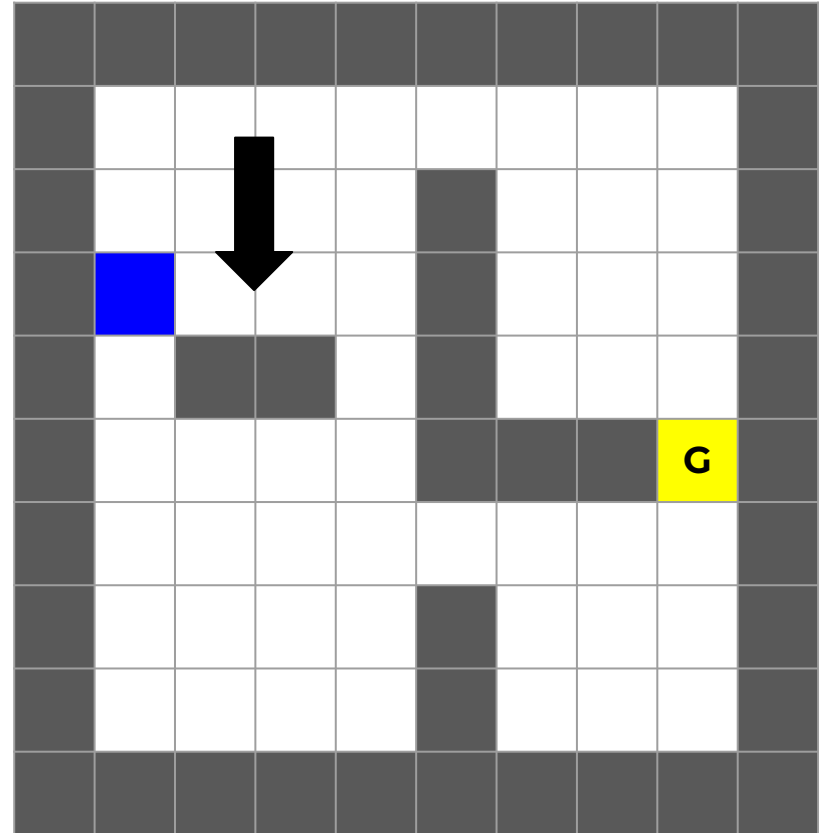
Options: semi-Markov decision process

- Agent is in state



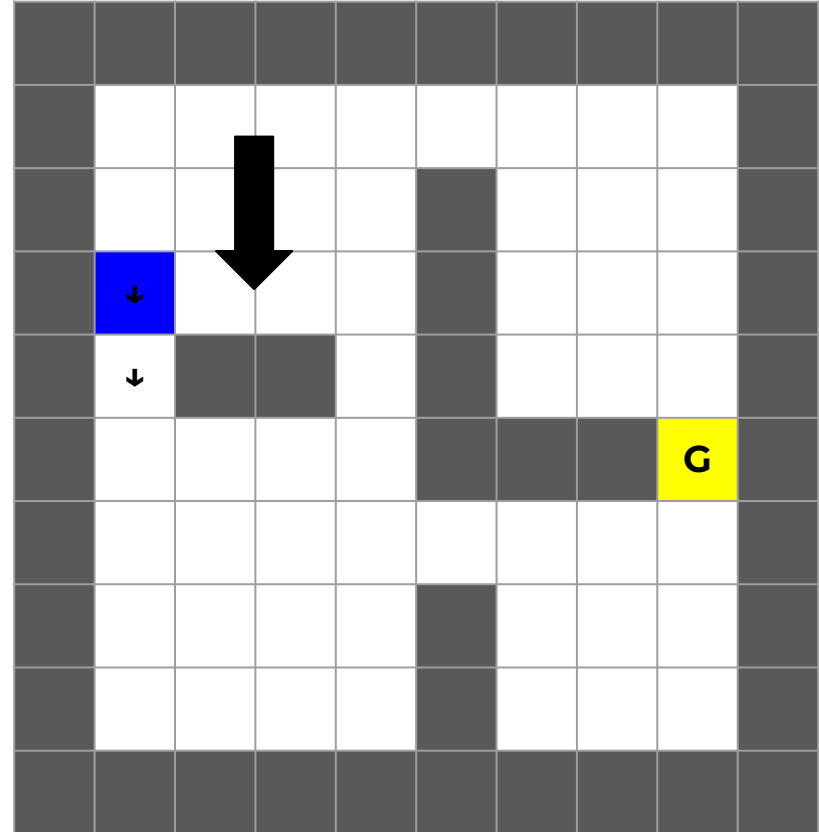
Options: semi-Markov decision process

- Agent is in state;
- Agents chooses option;



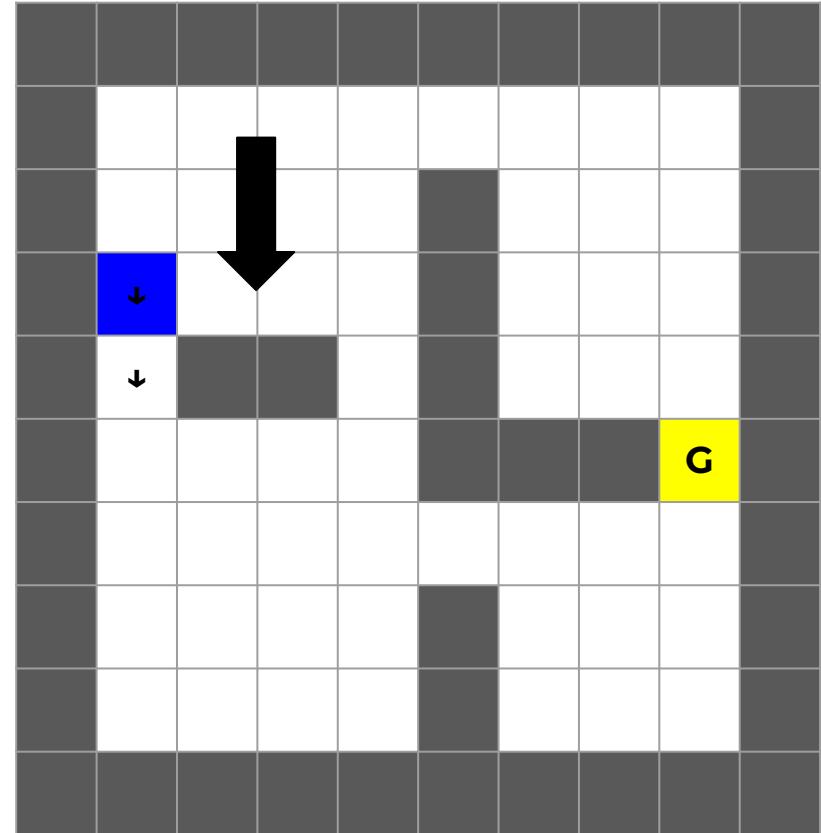
Options: semi-Markov decision process

- Agent is in state;
- Agents chooses option;
- Agent executes option;



Options: semi-Markov decision process

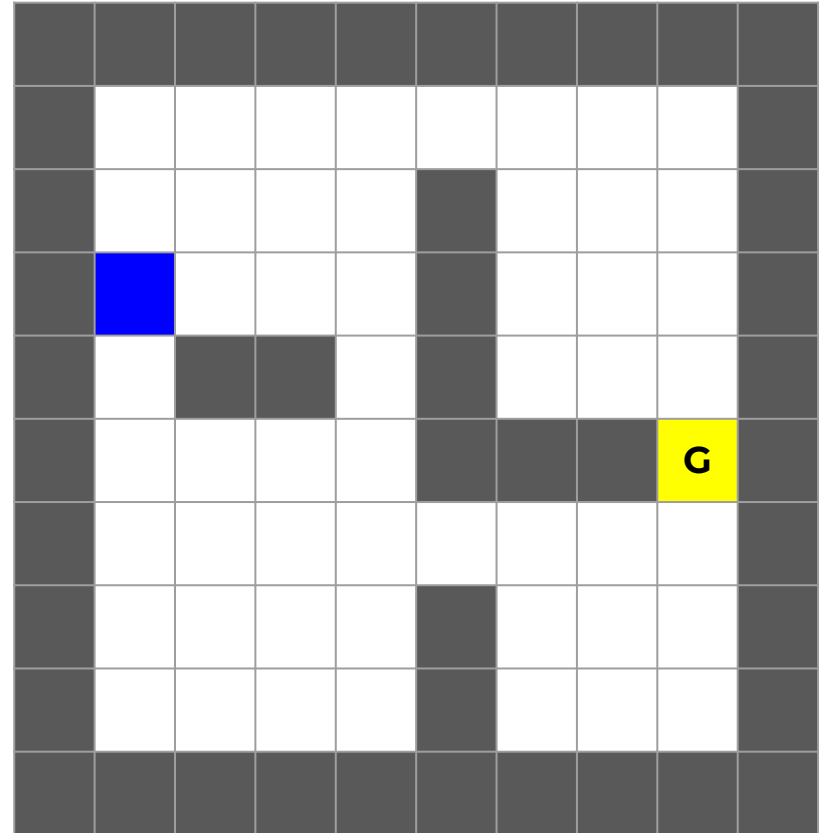
- Agent is in state;
- Agents chooses option;
- Agent executes option;
- Agent updates value.



Options: semi-Markov decision process

- Agent is in state;
- Agents chooses option;
- Agent executes option; 
- Agent updates value.

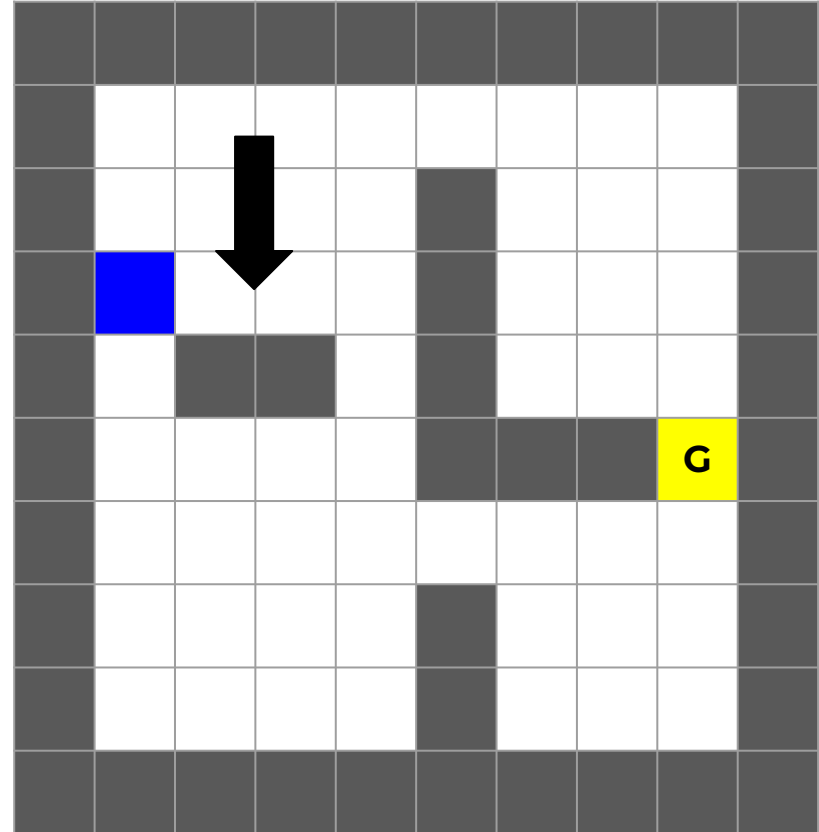
**What if the option
policy is not known?**



Options: semi-Markov decision process

- Agent is in state;
- Agents chooses option;
- Agent executes option; 
- Agent updates value.

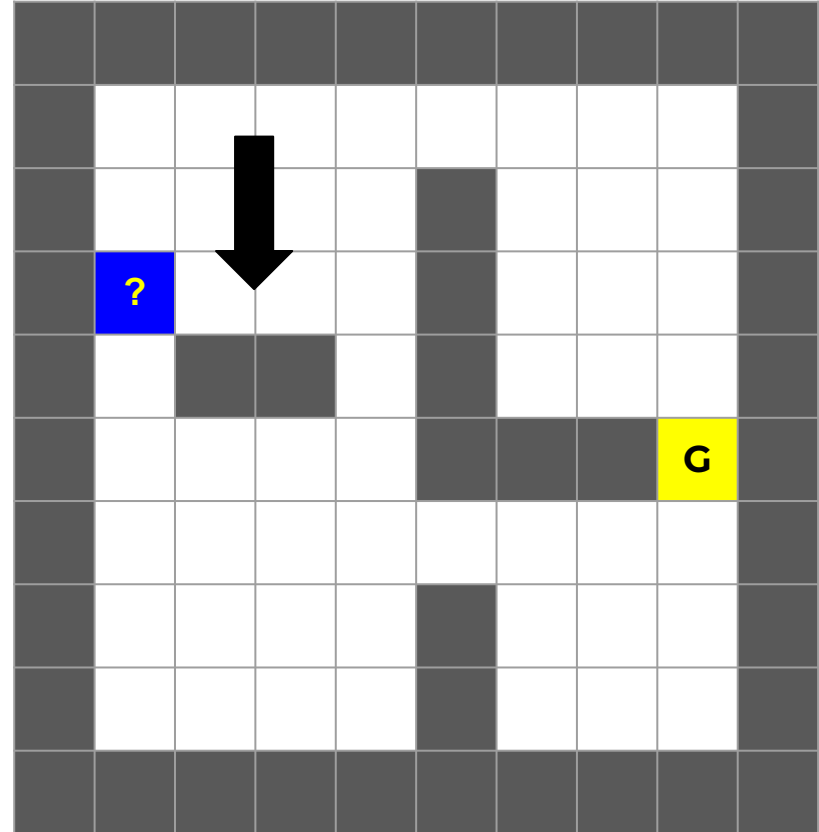
**What if the option
policy is not known?**



Options: semi-Markov decision process

- Agent is in state;
- Agents chooses option;
- Agent executes option; 
- Agent updates value.

**What if the option
policy is not known?**



Options: intra-option learning

- We can also learn the options policies

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.
 - Standard Q-learning:
 - Execute action according to option policy π_ω
 - Observe reward r_ω
 - Update Q-value $q_\omega(x, a) = q_\omega(x, a) + \alpha_t(r_\omega + \gamma \cdot \max_{a'} q_\omega(x', a') - q_\omega(x, a))$

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.
 - Standard Q-learning:
 - Execute action according to option policy π_ω
 - Observe reward r_ω
 - Update Q-value $q_\omega(x, a) = q_\omega(x, a) + \alpha_t(r_\omega + \gamma \cdot \max_{a'} q_\omega(x', a') - q_\omega(x, a))$
- Speed-up intra-option learning:

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.
 - Standard Q-learning:
 - Execute action according to option policy π_ω
 - Observe reward r_ω
 - Update Q-value $q_\omega(x, a) = q_\omega(x, a) + \alpha_t(r_\omega + \gamma \cdot \max_{a'} q_\omega(x', a') - q_\omega(x, a))$
- Speed-up intra-option learning:
 - Re-use transition to $q_{\omega'}(x, a)$

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.
 - Standard Q-learning:
 - Execute action according to option policy π_ω
 - Observe reward r_ω
 - Update Q-value $q_\omega(x, a) = q_\omega(x, a) + \alpha_t(r_\omega + \gamma \cdot \max_{a'} q_\omega(x', a') - q_\omega(x, a))$
- Speed-up intra-option learning:
 - Re-use transition to $q_{\omega'}(x, a)$
 - Share parameters as $q_\theta(x, a, \omega)$

Options: intra-option learning

- We can also learn the options policies:
 - Standard MDP $(\mathcal{X}, \mathcal{A}, R_\omega, P)$
 - We need to define the option reward:
 - For instance for reaching a goal state.
 - Standard Q-learning:
 - Execute action according to option policy π_ω
 - Observe reward r_ω
 - Update Q-value $q_\omega(x, a) = q_\omega(x, a) + \alpha_t(r_\omega + \gamma \cdot \max_{a'} q_\omega(x', a') - q_\omega(x, a))$

We can also learn both levels at the same time!

Options: end-to-end learning

Options: end-to-end learning

- Q-learning:
 - Select option $\omega = \tilde{\pi}(x)$
 - Until option termination:
 - Intra-option learning.
 - Update values

$$\tilde{q}(x, \omega) = \tilde{q}(x, \omega) + \alpha_t(\tilde{r} + \gamma^n \cdot \max_{\omega'} \tilde{q}(x', \omega') - \tilde{q}(x, \omega))$$

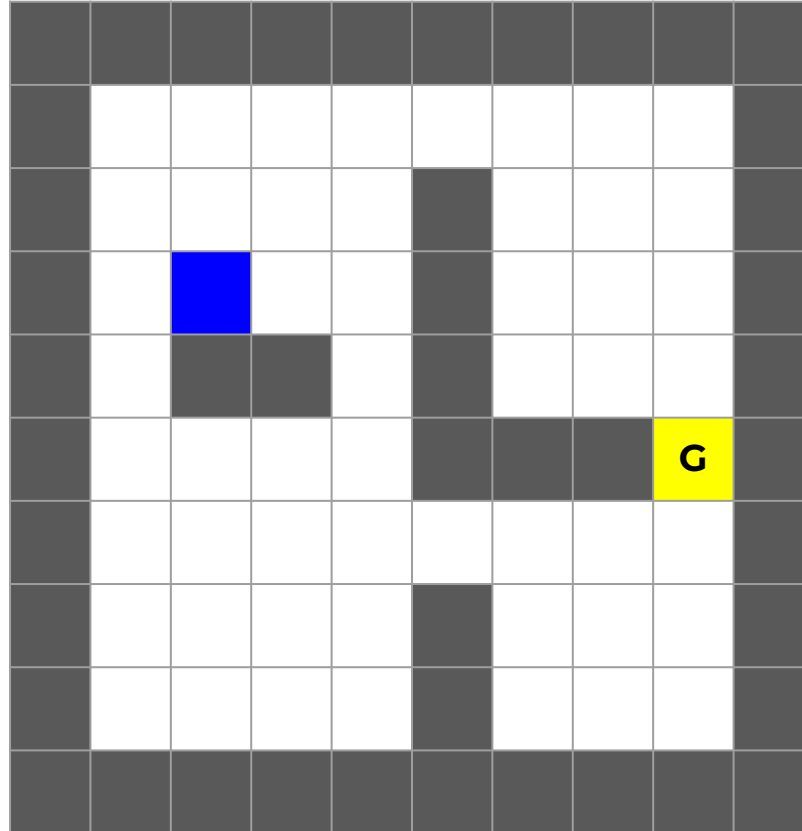
Options: desired properties

Options: desired properties

- Optimality;

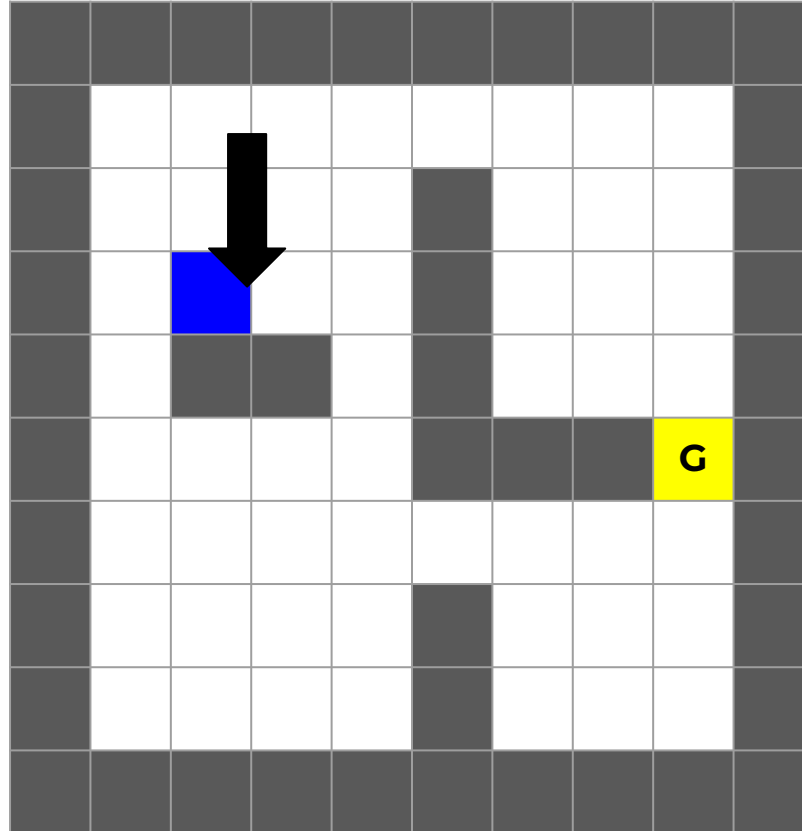
Options: desired properties

- Optimality;



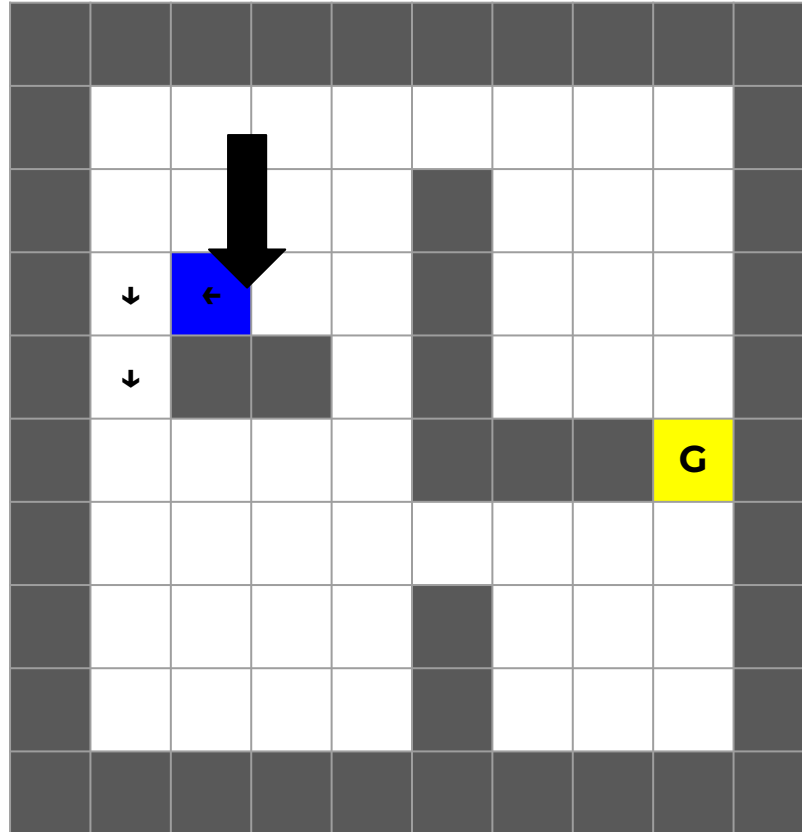
Options: desired properties

- Optimality;



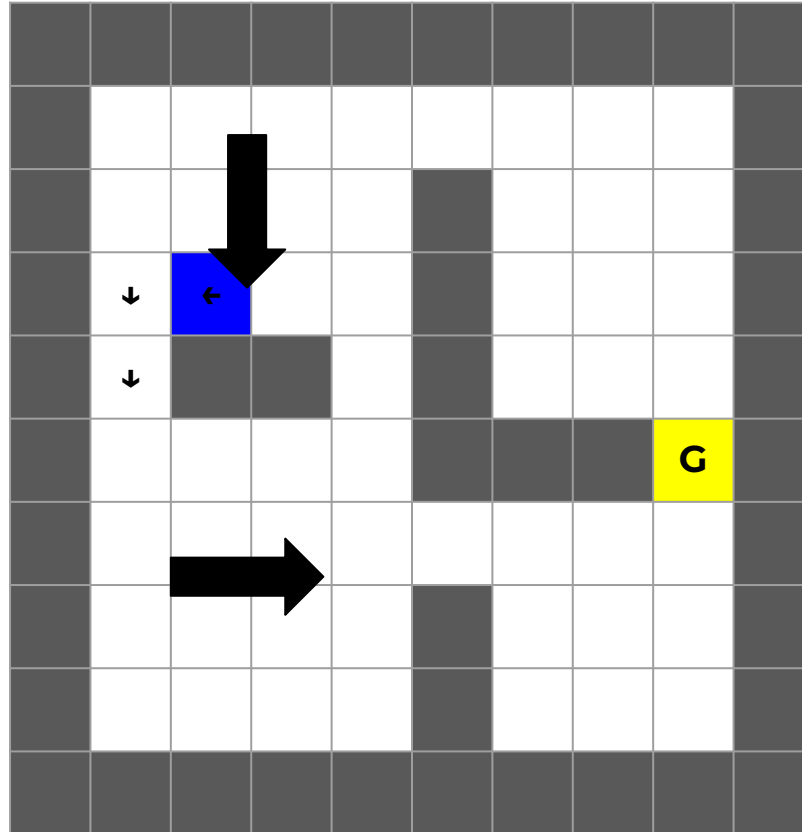
Options: desired properties

- Optimality;



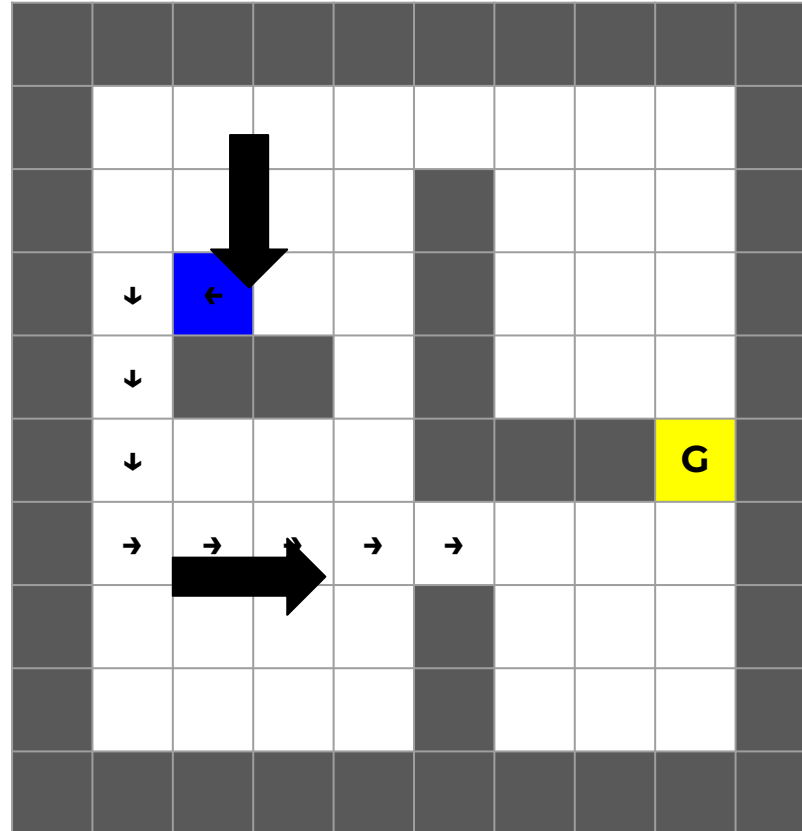
Options: desired properties

- Optimality;



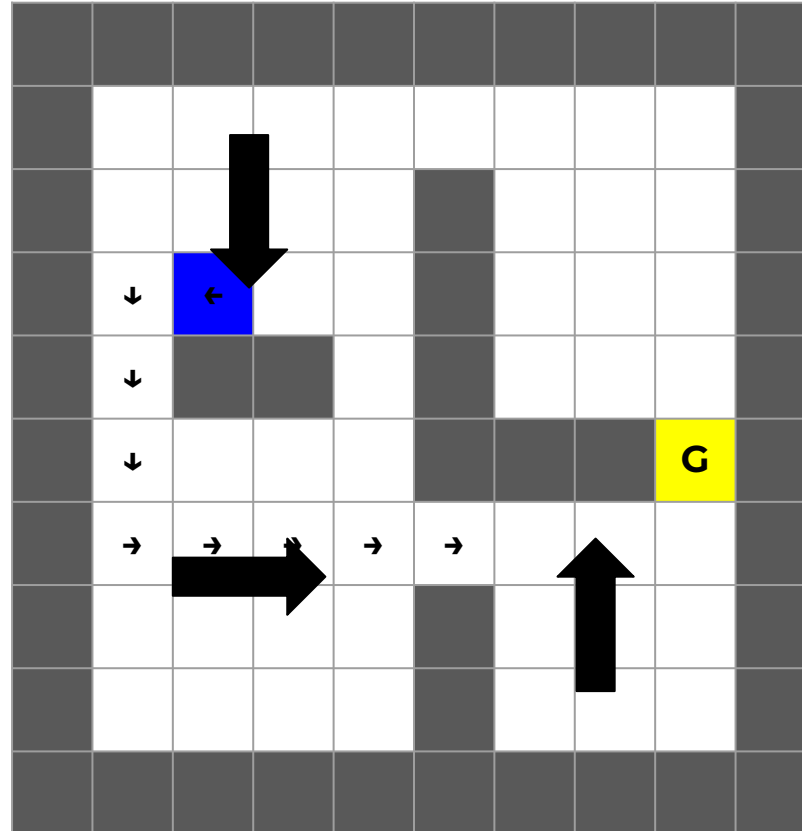
Options: desired properties

- Optimality;



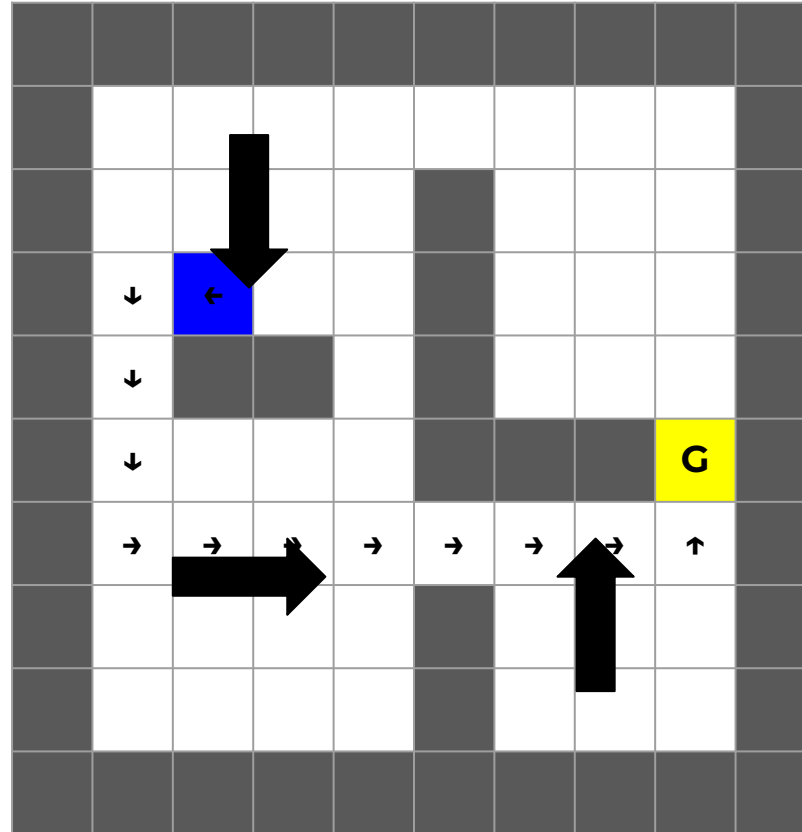
Options: desired properties

- Optimality;



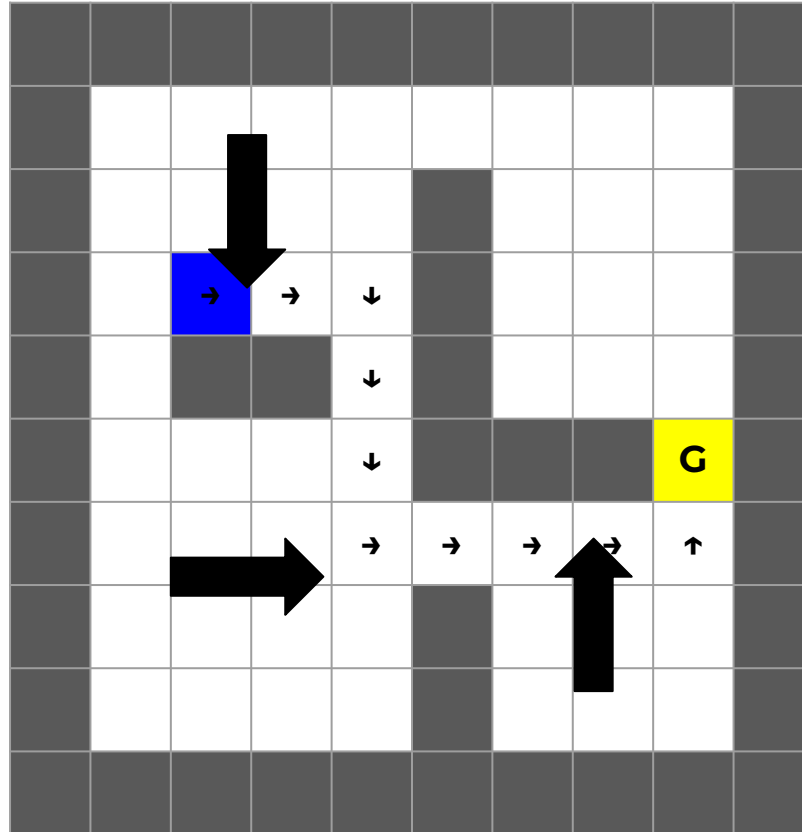
Options: desired properties

- Optimality;



Options: desired properties

- Optimality;



Options: desired properties

Options: desired properties

- Abstraction / Simplification:

Options: desired properties

- Abstraction / simplification:
 - At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$

Options: desired properties

- Abstraction / simplification:
 - At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
 - At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$

Options: desired properties

- Abstraction / simplification:
 - At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
 - At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$

Options: desired properties

- Abstraction / simplification:
 - At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
 - At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$
- Trade-off:

Options: desired properties

- Abstraction / simplification:

- At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
- At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$

- Trade-off:

- If we simplify the SMDP level too much... $|\Omega| = 1 \implies |\mathcal{X}_\omega| = |\mathcal{X}|$

Options: desired properties

- Abstraction / simplification:

- At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
- At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$

- Trade-off:

- If we simplify the SMDP level too much... $|\Omega| = 1 \implies |\mathcal{X}_\omega| = |\mathcal{X}|$
- If we simplify the option level too much... $|\mathcal{X}_\omega| = 1 \implies |\Omega| = |\mathcal{A}|$

Options: desired properties

- Abstraction / simplification:

- At the SMDP level: $|\mathcal{X} \times \Omega| < |\mathcal{X} \times \mathcal{A}|$
- At the option level: $|\mathcal{X}_\omega \times \mathcal{A}| < |\mathcal{X} \times \mathcal{A}|$

- Trade-off:

- If we simplify the SMDP level too much... $|\Omega| = 1 \implies |\mathcal{X}_\omega| = |\mathcal{X}|$
- If we simplify the option level too much... $|\mathcal{X}_\omega| = 1 \implies |\Omega| = |\mathcal{A}|$

Simplify the state-space!

Options: desired properties

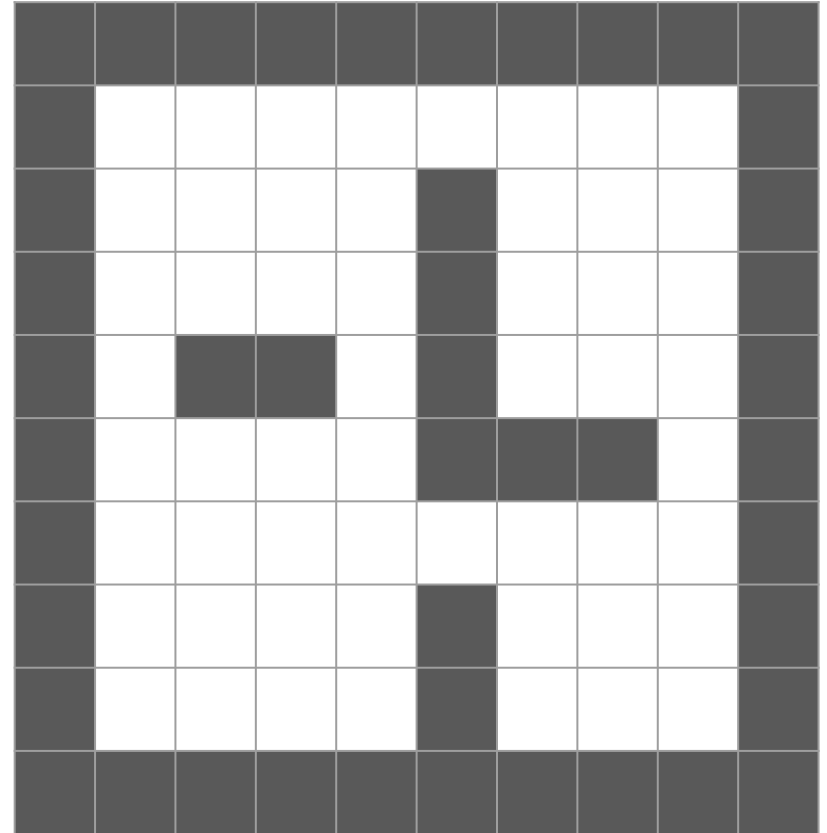
- State abstraction

Options: desired properties

- State abstraction $|\tilde{\mathcal{Z}}| < |\mathcal{X}|$

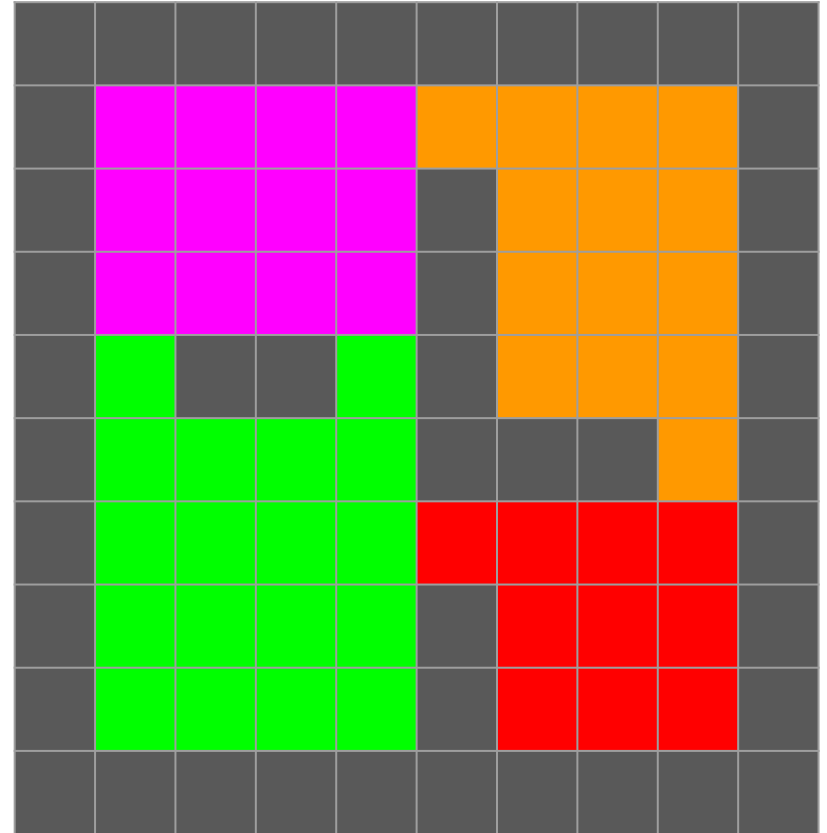
Options: desired properties

- State abstraction $|\tilde{\mathcal{Z}}| < |\mathcal{X}|$



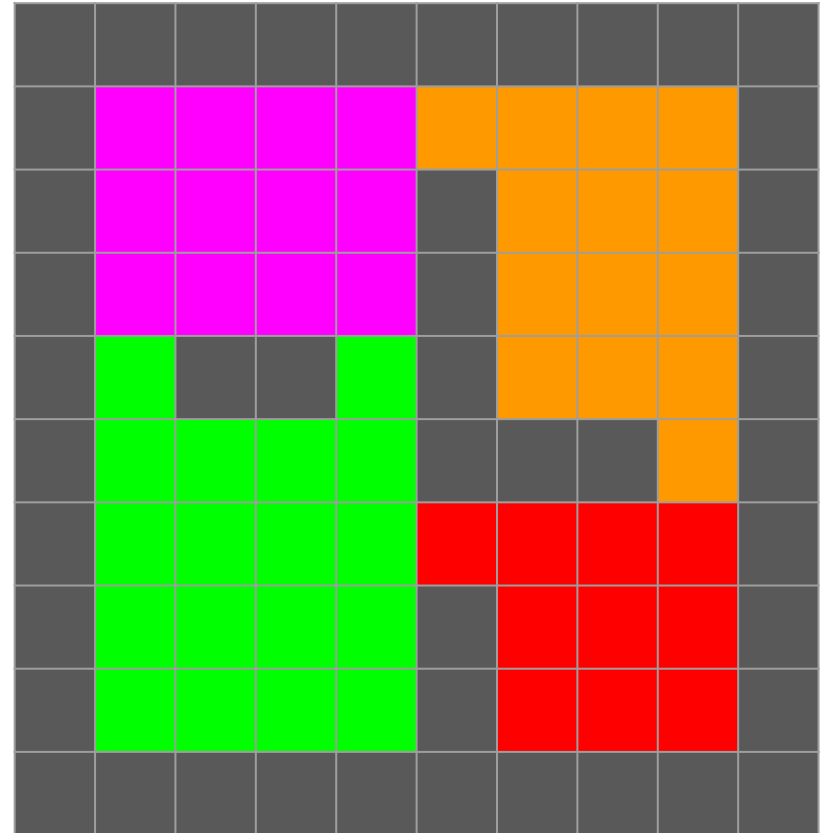
Options: desired properties

- State abstraction $|\tilde{\mathcal{Z}}| < |\mathcal{X}|$



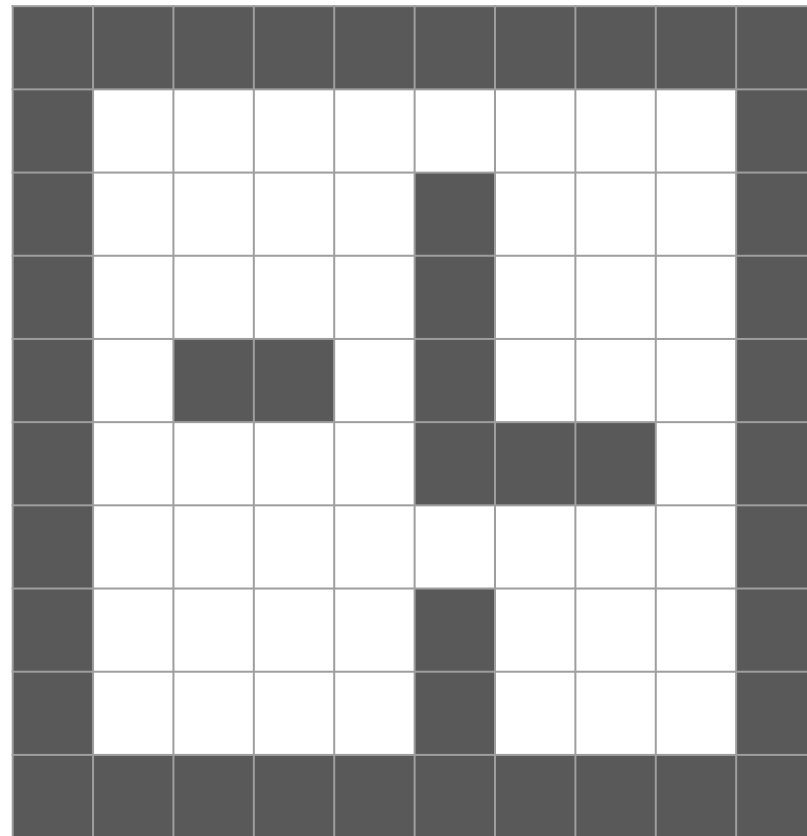
Options: desired properties

- State abstraction $|\tilde{\mathcal{Z}}| < |\mathcal{X}|$
- Can also be used at the option-level



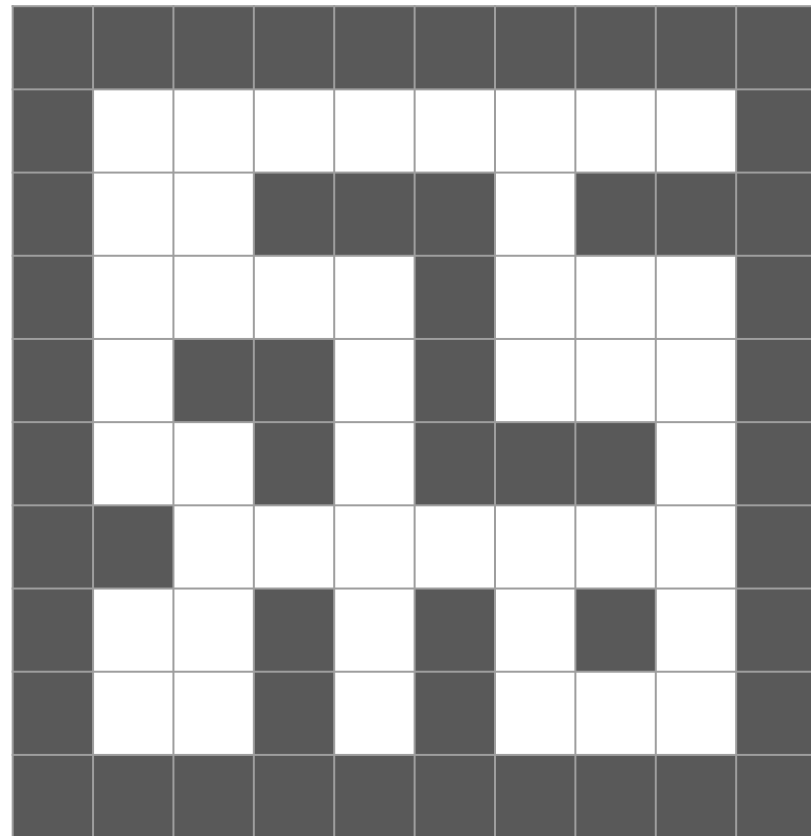
Options: design choices

Options: design choices



Options: design choices

?



Options: design choices

?



Options: design choices

- What are the tasks?



Options: design choices

- What are the tasks?
- Are the option policies known?



Options: design choices

- What are the tasks?
- Are the option policies known?
 - What is the option reward function?



Options: design choices

- What are the tasks?
- Are the option policies known?
 - What is the option reward function?
- Primitive actions are options?
- Speed-up intra-option learning?
- Use state abstraction?
 - Predefined or function approximation?
- Prevent non-stationarity?



Options: design choices

- **What are the tasks?**
- Are the option policies known?
 - What is the option reward function?
- Primitive actions are options?
- Speed-up intra-option learning?
- Use state abstraction?
 - Predefined or function approximation?
- Prevent non-stationarity?



Task discovery

Task discovery

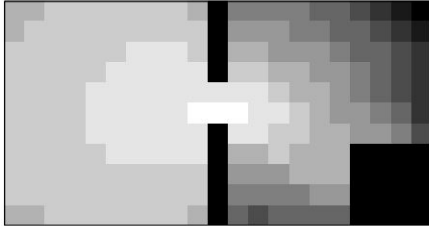
- Early works

Task discovery

- Early works:
 - Bottleneck states (McGovern and Barto 2001);
 - Novel-states (Simsek and Barto 2004);
 - State clustering (Mannor et al. 2004);
 - Graph analysis (Simsek and Barto 2008).

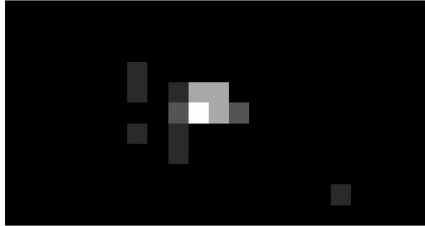
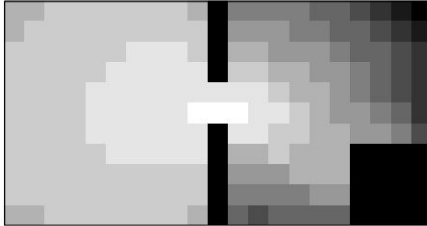
Task discovery

- Early works:
 - Bottleneck states (McGovern and Barto 2001);
 - Novel-states (Simsek and Barto 2004);
 - State clustering (Mannor et al. 2004);
 - Graph analysis (Simsek and Barto 2008).



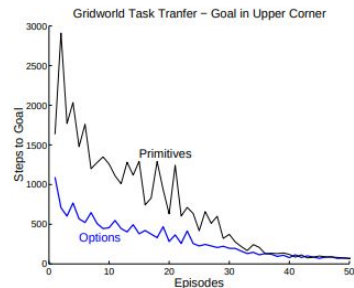
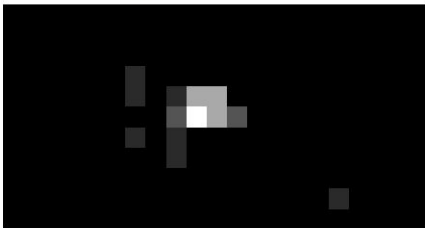
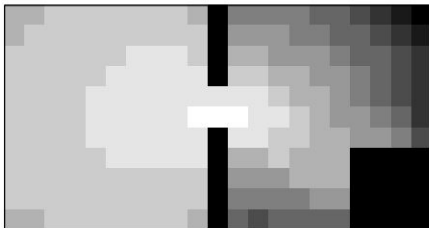
Task discovery

- Early works:
 - Bottleneck states (McGovern and Barto 2001);
 - Novel-states (Simsek and Barto 2004);
 - State clustering (Mannor et al. 2004);
 - Graph analysis (Simsek and Barto 2008).



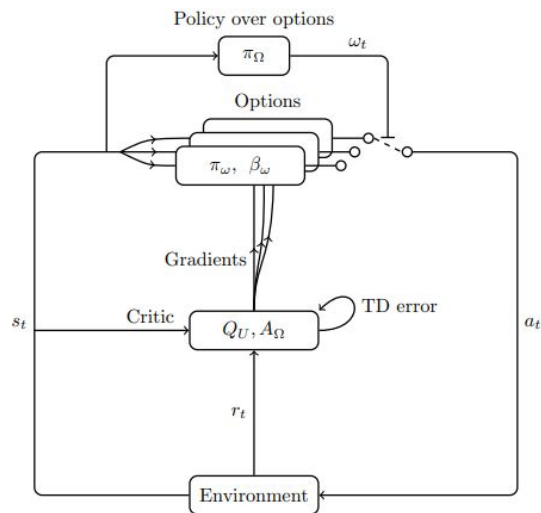
Task discovery

- Early works:
 - Bottleneck states (McGovern and Barto 2001);
 - Novel-states (Simsek and Barto 2004);
 - State clustering (Mannor et al. 2004);
 - Graph analysis (Simsek and Barto 2008).



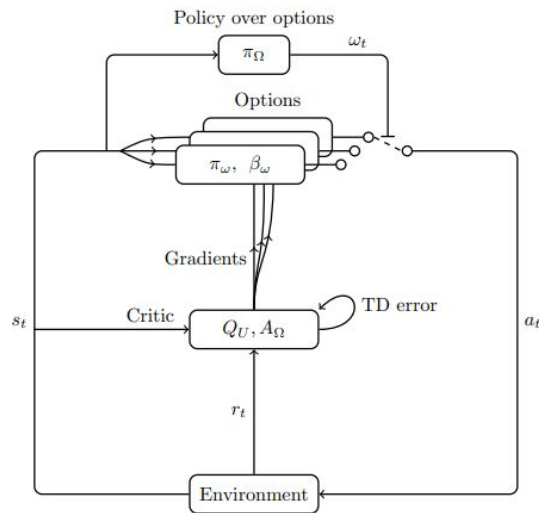
Task discovery

- Option-critic framework (Bacon et al. 2017)



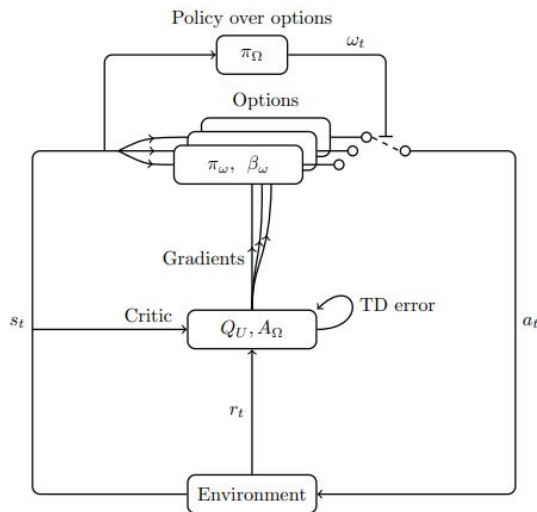
Task discovery

- Option-critic framework (Bacon et al. 2017)
 - Similar to the actor-critic framework;
 - Learn the hierarchical policy and termination functions for k options:



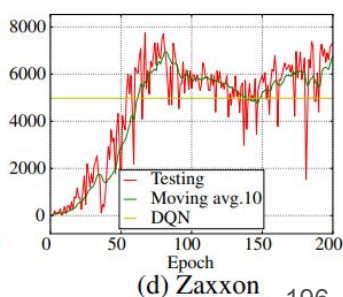
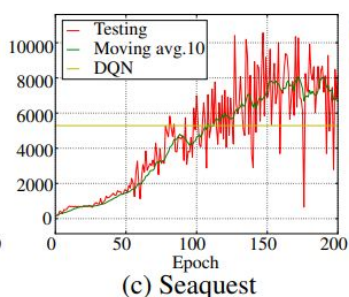
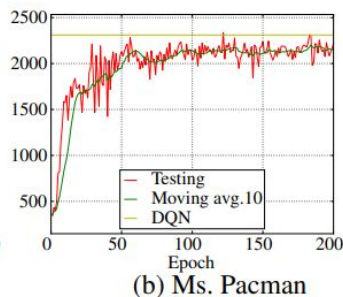
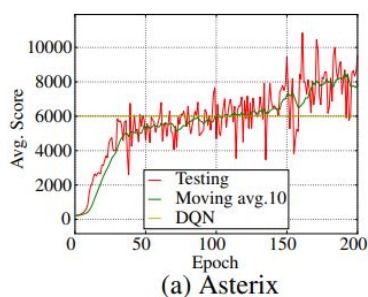
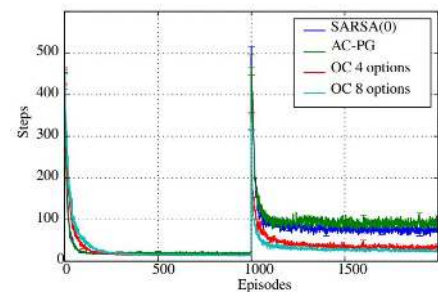
Task discovery

- Option-critic framework (Bacon et al. 2017)
 - Similar to the actor-critic framework;
 - Learn the hierarchical policy and termination functions for k options:
 - Gradient descent.



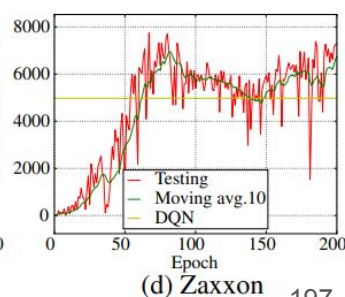
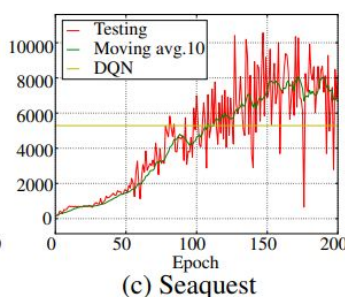
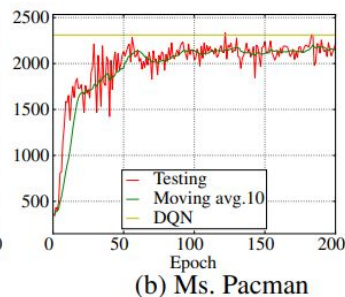
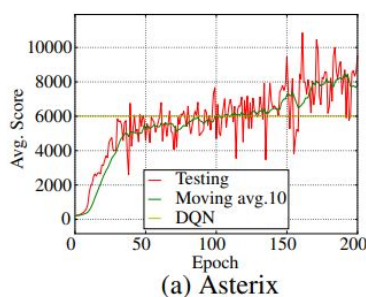
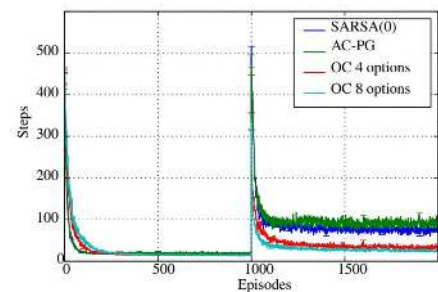
Task discovery

- Option-critic framework (Bacon et al. 2017)
 - Similar to the actor-critic framework;
 - Learn the hierarchical policy and termination functions for K options:
 - Gradient descent.



Task discovery

- Option-critic framework (Bacon et al. 2017)
 - Similar to the actor-critic framework;
 - Learn the hierarchical policy and termination functions for K options:
 - Gradient descent.
- Limitations:
 - Tendency to converge to a trivial policy;
 - Not easy to interpret;
 - Computationally expensive.



Task discovery

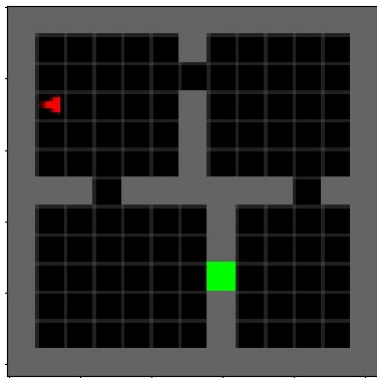
- Eigenoptions (Machado et al. 2017):

Task discovery

- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$

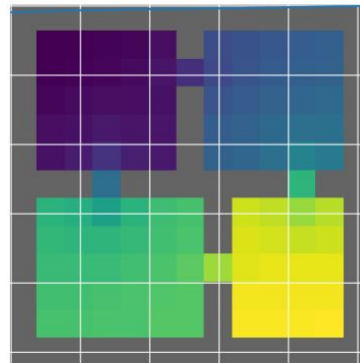
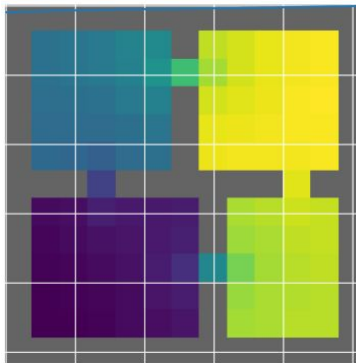
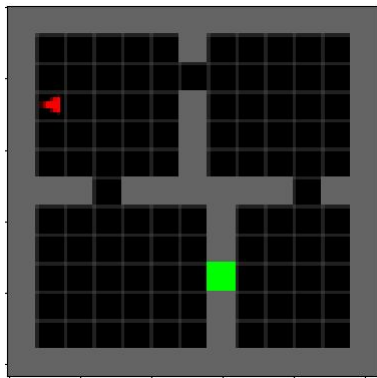
Task discovery

- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$



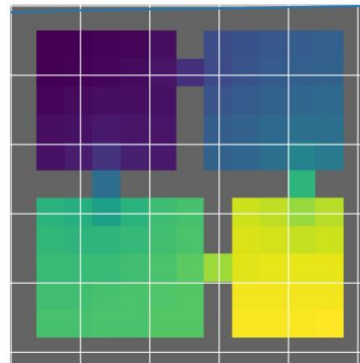
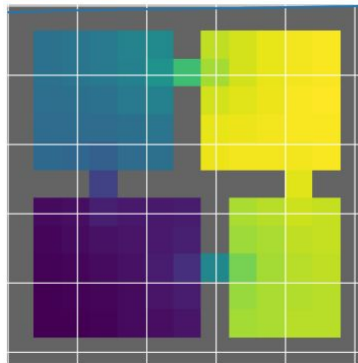
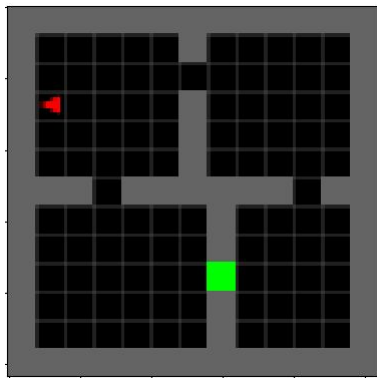
Task discovery

- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$



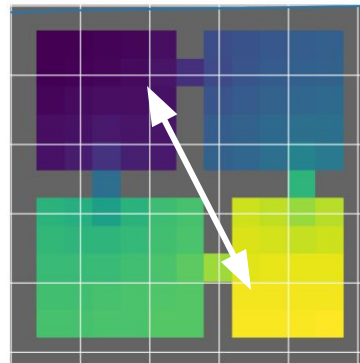
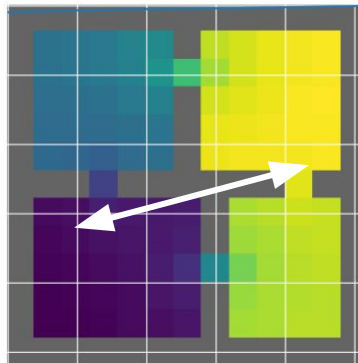
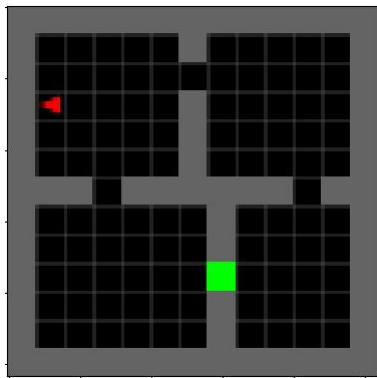
Task discovery

- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$
 - Maximize the eigenpurpose $r_i(s) = \phi_i(s)$



Task discovery

- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$
 - Maximize the eigenpurpose $r_i(s) = \phi_i(s)$

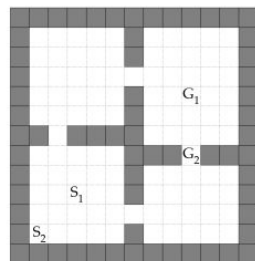


Task discovery

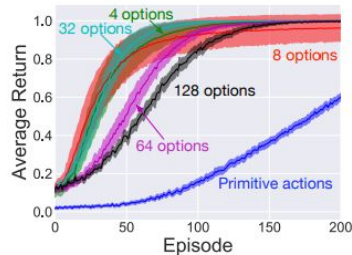
- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$
 - Maximize the eigenpurpose $r_i(s) = \phi_i(s)$
- Exploration

Task discovery

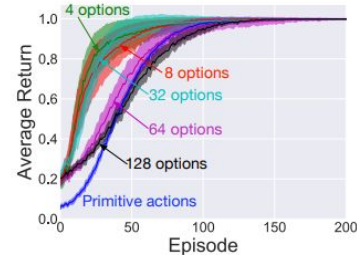
- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$
 - Maximize the eigenpurpose $r_i(s) = \phi_i(s)$
- Exploration:



(a) Used environments



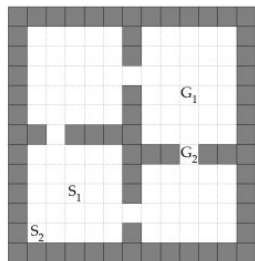
(b) Results in env. with S_1, G_1



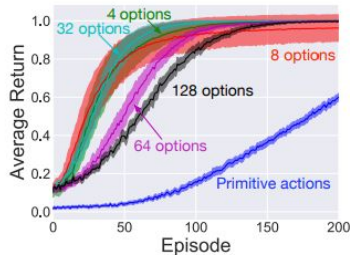
(c) Results in env. with S_2, G_2

Task discovery

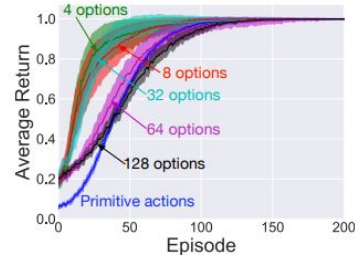
- Eigenoptions (Machado et al. 2017):
 - Based on the graph Laplacian $I - P_\pi$
 - Take the k smallest eigenvectors $\phi_i(x)$
 - Maximize the eigenpurpose $r_i(s) = \phi_i(s)$
- Limitations:
 - Requires knowledge of P_π
 - Eigenvectors are specific of policy.
 - Ignores reward;
 - Expensive.



(a) Used environments



(b) Results in env. with S_1, G_1



(c) Results in env. with S_2, G_2

Task discovery

- Covering options (Jinnai et al. 2019):

Task discovery

- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ

Task discovery

- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ
 - The option initiates at $\arg \min_x \phi(x)$
 - The option terminates at $x' = \arg \max_x \phi(x)$

Task discovery

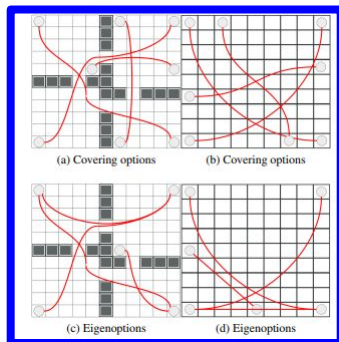
- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ
 - The option initiates at $\arg \min_x \phi(x)$
 - The option terminates at $x' = \arg \max_x \phi(x)$
 - Learn the option policy from $r(x) = \mathbf{1}(x = x')$

Task discovery

- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ
 - The option initiates at $\arg \min_x \phi(x)$
 - The option terminates at $x' = \arg \max_x \phi(x)$
 - Learn the option policy from $r(x) = \mathbf{1}(x = x')$
 - Recompute the Laplacian with the new option;
 - Repeat for k iterations.

Task discovery

- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ
 - The option initiates at $\arg \min_x \phi(x)$
 - The option terminates at $x' = \arg \max_x \phi(x)$
 - Learn the option policy from $r(x) = \mathbf{1}(x = x')$
 - Recompute the Laplacian with the new option;
 - Repeat for k iterations.

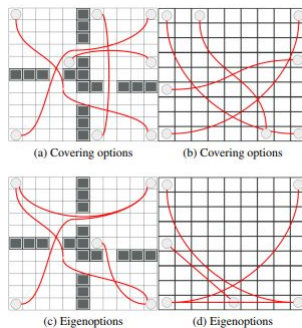


four-room	λ_2	Expected Cover Time
Covering options	0.065	672.0
Eigenoptions	0.054	695.9
No options	0.023	1094.8

9x9 grid	λ_2	Expected Cover Time
Covering options	0.24	258.6
Eigenoptions	0.19	261.5
No options	0.12	460.5

Task discovery

- Covering options (Jinnai et al. 2019):
 - Take the smallest eigenvector of the graph Laplacian, ϕ
 - The option initiates at $\arg \min_x \phi(x)$
 - The option terminates at $x' = \arg \max_x \phi(x)$
 - Learn the option policy from $r(x) = \mathbf{1}(x = x')$
 - Recompute the Laplacian with the new option;
 - Repeat for k iterations.



four-room	λ_2	Expected Cover Time
Covering options	0.065	672.0
Eigenoptions	0.054	695.9
No options	0.023	1094.8
9x9 grid	λ_2	Expected Cover Time
Covering options	0.24	258.6
Eigenoptions	0.19	261.5
No options	0.12	460.5

Task discovery

- Covering eigenoptions (Machado et al. 2023):

Task discovery

- Covering eigenoptions (Machado et al. 2023):
 - Iteratively add option from the smallest eigenvector of the Laplacian.

Task discovery

- Covering eigenoptions (Machado et al. 2023):
 - Iteratively add option from the smallest eigenvector of the Laplacian.
 - Combines the benefits of:
 - Covering options:
 - Iterative;
 - Low cover time;
 - Eigenoptions:
 - Dense reward;
 - Broad initiation sets.

Task discovery

- Covering eigenoptions (Machado et al. 2023):
 - Iteratively add option from the smallest eigenvector of the Laplacian.
 - Combines the benefits of:
 - Covering options:
 - Iterative;
 - Low cover time;
 - Eigenoptions:
 - Dense reward;
 - Broad initiation sets.

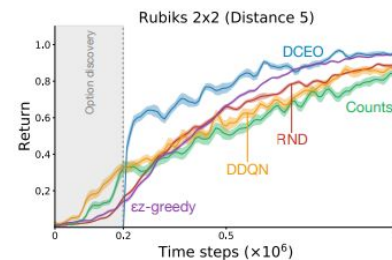
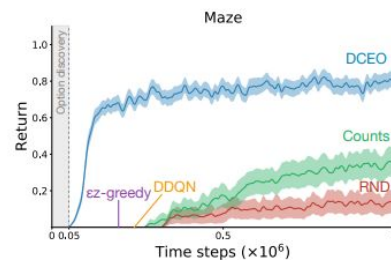
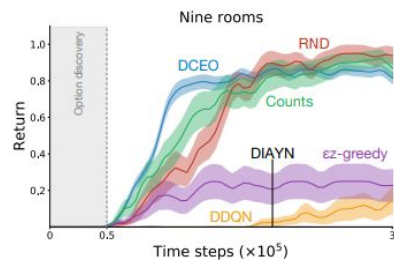
Can be combined with function approximation!

Task discovery

- Deep covering eigenoptions for exploration (Klissarov and Machado 2023):

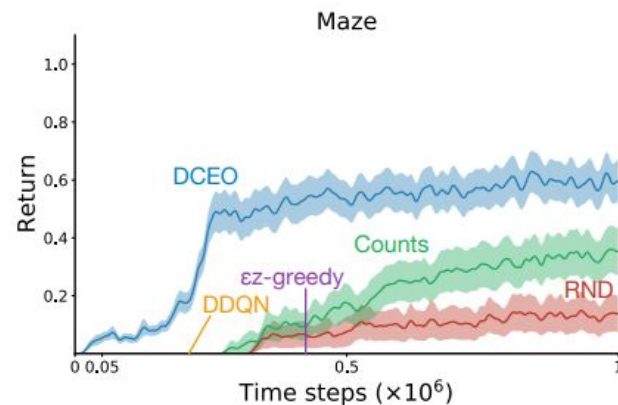
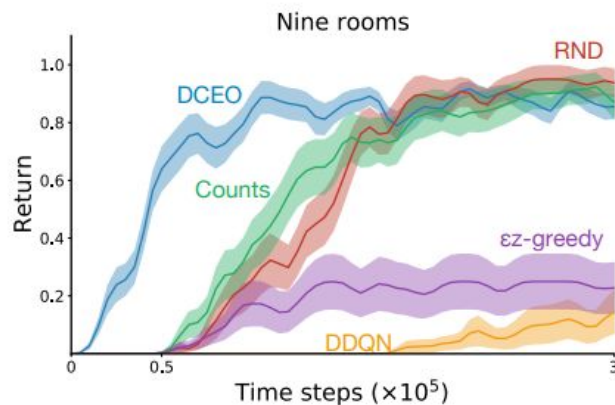
Task discovery

- Deep covering eigenoptions for exploration (Klissarov and Machado 2023):
 - Pre-trained;



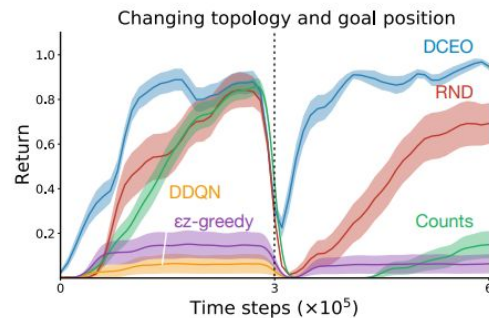
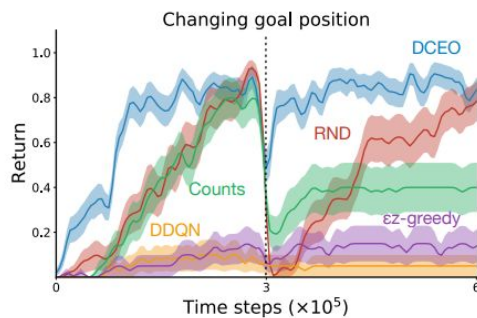
Task discovery

- Deep covering eigenoptions for exploration (Klissarov and Machado 2023):
 - Pre-trained;
 - Online;



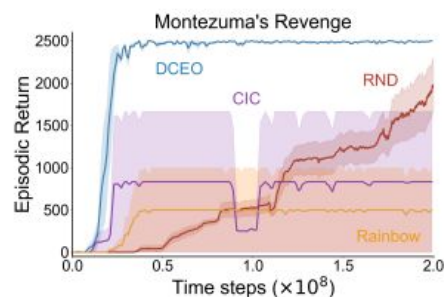
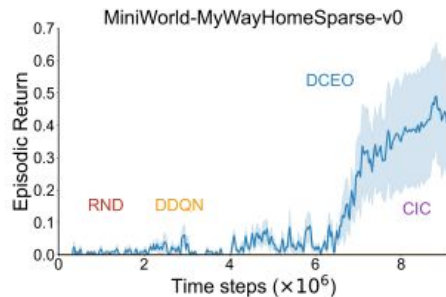
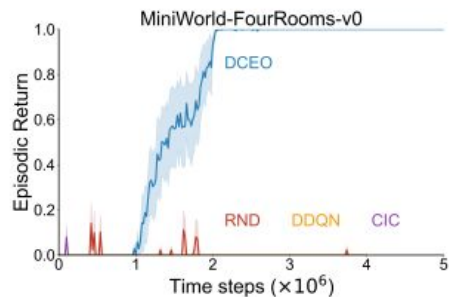
Task discovery

- Deep covering eigenoptions for exploration (Klissarov and Machado 2023):
 - Pre-trained;
 - Online;
 - Transfer;



Task discovery

- Deep covering eigenoptions for exploration (Klissarov and Machado 2023):
 - Pre-trained;
 - Online;
 - Transfer;
 - High-dimensions.



Task discovery: option keyboard

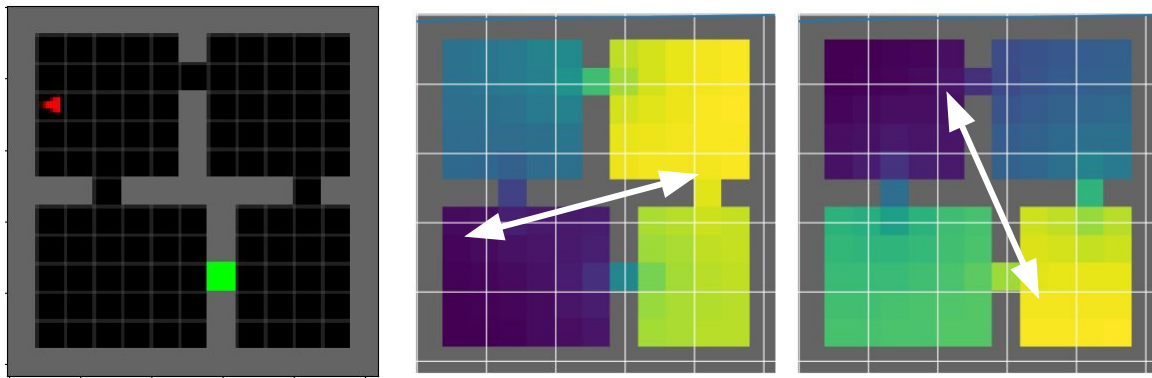
- Introduced by Barreto et al. (2019);

Task discovery: option keyboard

- Introduced by Barreto et al. (2019);
- Combine options to new ones without additional learning.

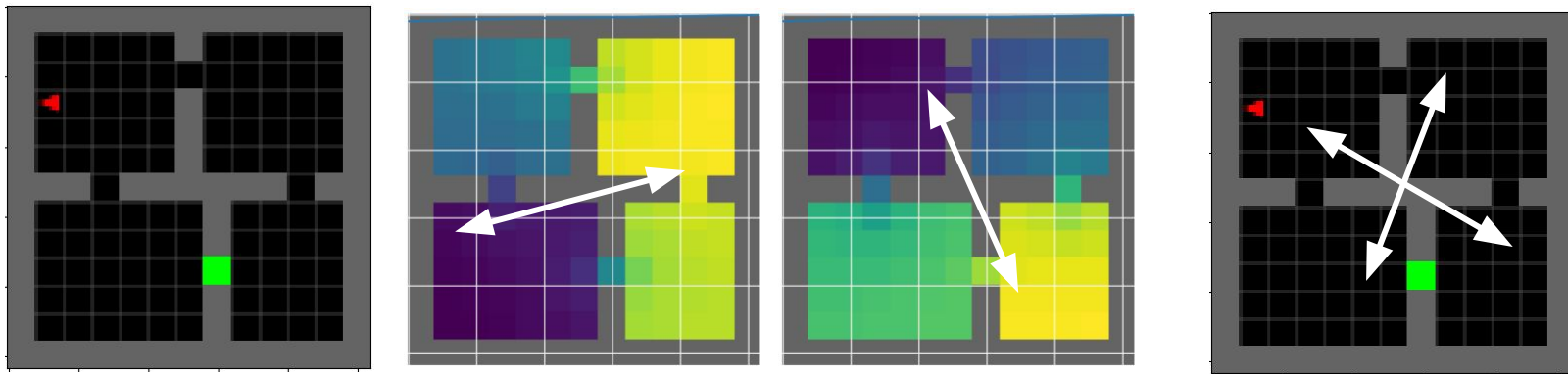
Task discovery: option keyboard

- Introduced by Barreto et al. (2019);
- Combine options to new ones without additional learning.



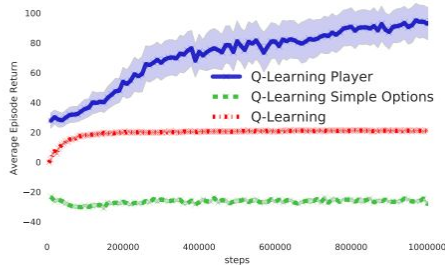
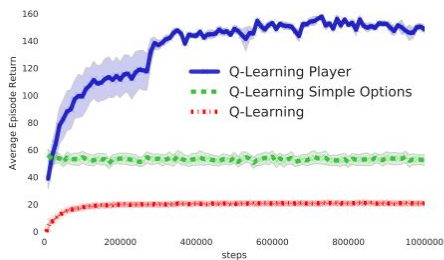
Task discovery: option keyboard

- Introduced by Barreto et al. (2019);
- Combine options to new ones without additional learning.



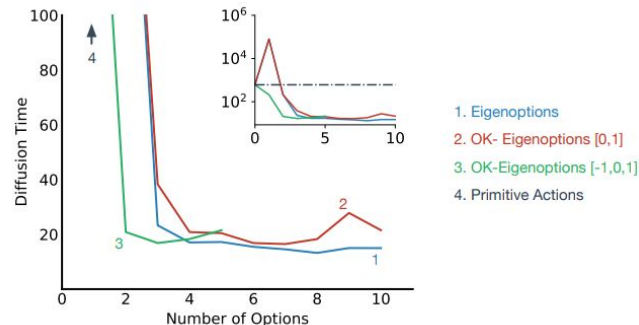
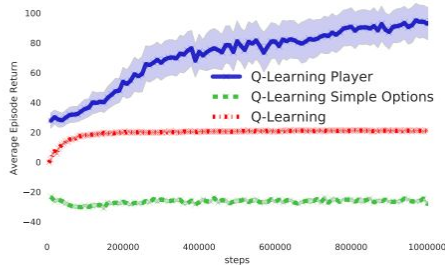
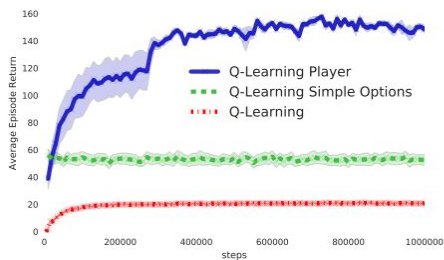
Task discovery: option keyboard

- Introduced by Barreto et al. (2019);
- Combine options to new ones without additional learning.
 - Approximate option policy, but still with benefits.



Task discovery: option keyboard

- Introduced by Barreto et al. (2019);
- Combine options to new ones without additional learning.
 - Approximate option policy, but still with benefits.
- Can be combined with eigenoptions (Machado et al. 2023).



Conclusion

Benefits

Benefits

- Exploration;
- Transfer;
- Efficiency;
- Interpretability;
- Simplification.

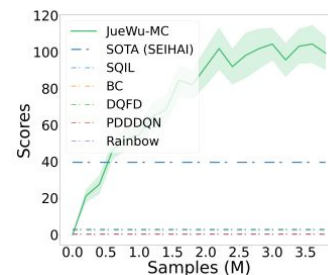
Success

- Minecraft competition (Lin et al. 2021)

Success

- Minecraft competition (Lin et al. 2021)

Baselines		2019 Competition Results		2020 Competition Results		2021 Competition Results	
Name	Score	Team Name	Score	Team Name	Score	Team Name	Score
SQIL	2.94	CDS (ForGER)	61.61	HelloWorld (SEIHAI)	39.55	X3 (JueWu-MC)	76.97
DQfD	2.39	mc_rl	42.41	michal_opanowicz	13.29	WinOrGoHome	22.97
Rainbow	0.42	I4DS	40.8	NoActionWasted	12.79	MCagent	18.98
PDDDQN	0.11	CraftRL	23.81	Rabbits	5.16	sneakysquids	14.35
BC	2.40	UEFDRL	17.9	MajiManji	2.49	JBR_HSE	10.33
		TD240	15.19	BeepBoop	1.97	zhongguodui	8.84



Limitations

Limitations

- Design choices;
- No unified terminology and framework;
- Not many big success stories;
- Few theoretical guarantees;
- No common benchmarks.

Open problems

Open problems

- Theory:
 - End-to-end learning (non-stationarity);
 - Optimality and efficiency;
 - Trade-off optimality to abstraction;
 - Unification of the frameworks.

Open problems

- Application:
 - High-dimensional / hard / real-world problem;
 - Show that one approach achieves several benefits;
 - Compare different approaches in several benefits;
 - Define important hierarchical testbeds.

Resources

Resources

- Classes:
 - Hierarchical reinforcement learning, RL summer school 2023, Anders Jonsson;
 - Hierarchical RL and skill discovery, Stanford computer science 2020, Karol Hausman.

Resources

- Classes:
 - Hierarchical reinforcement learning, RL summer school 2023, Anders Jonsson;
 - Hierarchical RL and skill discovery, Stanford computer science 2020, Karol Hausman.
- Surveys:
 - Hierarchical reinforcement learning: a survey and open research challenges, Hutsebaut-Buisse et al. 2022;
 - Hierarchical reinforcement learning: a comprehensive survey, Pateria et al. 2021.
- Seminal papers:
 - Feudal reinforcement learning, Dayan and Hinton 1992;
 - Between MDPs and semi-MDPs, Sutton et al. 1999.

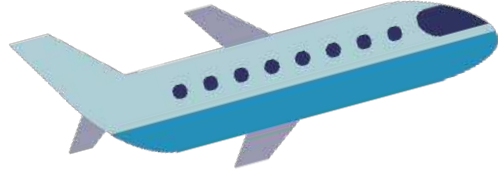
More recently

More recently

- Important papers:
 - Feudal networks for hierarchical reinforcement learning, Vezhnevets 2017;
 - Data-efficient hierarchical reinforcement learning, Nachum et al. 2018;
 - The option-critic architecture, Bacon et al. 2017;
 - Temporal abstraction in RL with the successor representation, Machado et al. 2023;
 - Deep laplacian-based options for temporally extended exploration, Klissarov and Machado 2023.

More recently

- Important papers:
 - Feudal networks for hierarchical reinforcement learning, Vezhnevets 2017;
 - Data-efficient hierarchical reinforcement learning, Nachum et al. 2018;
 - The option-critic architecture, Bacon et al. 2017;
 - Temporal abstraction in RL with the successor representation, Machado et al. 2023;
 - Deep laplacian-based options for temporally extended exploration, Klissarov and Machado 2023.
- Also:
 - Hierarchically structured scheduling and execution of tasks in a multi-agent warehouse environment, Carvalho and Sengupta 2022;
 - Theoretical remarks on feudal hierarchies and reinforcement learning, Carvalho et al. 2023



Hierarchical reinforcement learning

or abstraction in sequential decision making

